

PARADIGMAS DE PROGRAMACIÓN · LABORATORIO 1

Guía de Estudio Completa

Proyecto HackerRank1 (LibraryService API)

Guía de Estudio Completa

Proyecto analizado: Laboratorio 1 · Paradigmas de Programación **Objetivo de la guía:** Llevar a un desarrollador desde cero hasta ser capaz de comprender **completamente** este proyecto y de diseñar e implementar **desde cero** una API Web en .NET similar, de manera autónoma.

Horas totales estimadas: 215 horas (rango recomendado: 150–250).

Proyecto HackerRank1 (LibraryService API) — .NET 8

Cómo usar esta guía

1. No estudies los módulos en orden arbitrario. Sigue la **Ruta de aprendizaje** (sección final) — cada módulo indica los conocimientos previos que exige.
2. Por cada módulo: - Lee el objetivo y los conceptos fundamentales. - Ubica cada concepto en el proyecto real usando "Cómo aparece dentro del proyecto". - Haz los ejercicios sugeridos **con código real**, no solo leyendo. - Responde las preguntas de autoevaluación **sin mirar el proyecto**. - Construye el proyecto práctico del módulo (el mismo proyecto puede crecer módulo a módulo).
3. Marca tu progreso en el **Checklist** final.
4. Al terminar toda la ruta, realiza la **Evaluación final** en un solo bloque de 4 horas.
5. Si un tema aparece en "Conocimientos previos" y no lo dominas, detente y regresa antes de continuar.

Regla de oro de esta guía: **no mires código antes de entender el concepto**. Primero la teoría, luego el código, luego escribirlo tú mismo.

Contexto real del proyecto que debes tener presente

Antes de cada módulo, este es el estado del arte que la guía pretende explicar:

- **Solución:** `HackerRank1.sln` , con 2 proyectos → `HackerRank1` (Web API, `net8.0`) y `LibraryService.Integration.Test` (xUnit, `net8.0`).
 - **API:** `LibraryService` — gestiona `Libraries` y `Books` (un libro pertenece a una librería).
 - **Patrón:** arquitectura en capas clásica → `Controller` → `Service` → `DbContext` (EF Core) → `PostgreSQL` (Supabase) . **No usa Repository Pattern.**
 - **Hosting:** modelo "legacy" → `Program.cs` con `Host.CreateDefaultBuilder` + `UseStartup<Startup>()` .
 - **Datos:** EF Core 8.0.2, proveedor Npgsql, base PostgreSQL en la nube (Supabase), migración `InitialCreate` , FK con `ON DELETE CASCADE` .
 - **Seguridad:** JWT Bearer (HS256), endpoint `POST /login` , credenciales hardcodeadas (`admin` / `1234`), secretos fuera del repo (User Secrets).
 - **Docs API:** Swagger/Swashbuckle.
 - **Tests:** 3 tests de integración (xUnit + FluentAssertions) que sustituyen el `LibraryContext` real por SQLite in-memory.
 - **Herramientas:** `dotnet` CLI, `dotnet-ef` (global tool), `dotnet user-secrets` , Git/GitHub.
-

MÓDULO 1 — Fundamentos de Internet y HTTP

1. Nombre del módulo

Fundamentos de Internet, HTTP/HTTPS y APIs REST

2. Objetivo de aprendizaje

El estudiante comprenderá cómo viajan los datos entre un cliente y un servidor: qué es una petición HTTP, su estructura completa (método, URL, headers, body, status code), qué significa que HTTP sea *stateless*, y por qué una API REST escribe y lee JSON. Al terminar, debe ser capaz de leer una petición HTTP cruda y predecir la respuesta de un endpoint simple.

3. Conceptos fundamentales

3.1 TCP/IP y el modelo cliente-servidor

- **Qué es:** conjunto de protocolos que permite a dos máquinas comunicarse por red. TCP garantiza que los bytes lleguen en orden y sin pérdida; IP asigna direcciones (`172.16.0.1` , `127.0.0.1` = localhost). El puerto identifica el servicio dentro de la máquina (`80` HTTP, `443` HTTPS, `5432` PostgreSQL, `7098` esta API).
- **Para qué sirve:** todo HTTP corre *sobre* TCP. Entenderlo explica conceptos como conexiones, latencia, puertos y firewalls.
- **Cuándo se utiliza:** siempre que una app hable con otra, aunque normalmente no lo ves porque el framework lo esconde.
- **Ventajas:** comunicación confiable, universal, abstraída por los frameworks.
- **Desventajas:** overhead (latencia) y complejidad de red que el desarrollador debe respetar (timeouts, retries).
- **Relación:** el modelo cliente-servidor es la base del patrón "Controller responde a una petición".

3.2 URL y URI

- **Qué es:** una URL (`Uniform Resource Locator`) identifica *dónde* está un recurso. Estructura: `esquema://host:puerto/ruta?query#fragmento`.
- **En el proyecto:** `https://localhost:7098/api/libraries/1/books?x=1` → esquema `https` , host `localhost` , puerto `7098` , ruta `/api/libraries/1/books` . La **ruta** mapea al controller/acción; el **query string** a parámetros opcionales.
- **Relación:** el routing de ASP.NET Core se construye sobre estas piezas.

3.3 DNS (Domain Name System)

- **Qué es:** el "directorio telefónico" de Internet; traduce `supabase.com` a una IP.
- **Para qué sirve:** los humanos escriben nombres, las máquinas usan IPs. `localhost` se resuelve a `127.0.0.1` sin red.
- **En el proyecto:** el host de Supabase `aws-1-us-west-2.pooler.supabase.com` se resuelve por DNS.

3.4 HTTP

- **Qué es:** el protocolo de transferencia de hipertexto. Un **request** (petición) y un **response** (respuesta). Cada uno tiene: línea de inicio, headers y body (opcional).
- **Para qué sirve:** es el idioma que hablan navegadores, Postman, cURL y las APIs. Todo el proyecto es HTTP.
- **Cuándo se utiliza:** cada llamada a un endpoint. No puedes escribir una Web API sin entender HTTP.
- **Ventajas:** simple, humano-lectible, soportado por todo, sin estado (ver 3.8).
- **Desventajas:** sin cifrado por defecto (HTTPS lo resuelve), mensajes verbosos, *no apto para streaming pesado en tiempo real* (para eso hay WebSockets/gRPC).
- **Relación:** REST es un *estilo* de diseño que se implementa *con* HTTP.

3.5 Métodos HTTP

- **Qué son:** verbos que indican la *intención*: `GET` (leer), `POST` (crear), `PUT` (reemplazar), `PATCH` (actualizar parcial), `DELETE` (borrar).

- **Semántica clave:** `GET` y `DELETE` son **idempotentes** (ejecutarlos N veces produce el mismo resultado); `POST` **no** lo es (cada POST crea un recurso nuevo). `PUT` es idempotente; `PATCH` no necesariamente.
- **En el proyecto:**
 - `GET /api/libraries` → lista librerías.
 - `GET /api/libraries/{id}` → una librería.
 - `POST /api/libraries` → crea librería.
 - `PUT /api/libraries/{id}` → reemplaza librería.
 - `DELETE /api/libraries/{id}` → borra librería.
 - `GET|POST /api/libraries/{libraryId}/books` → libros de una librería.
 - `POST /login` → emite token.
- **Error común:** usar `GET` para acciones que modifican datos, o `POST` para lecturas (rompe la semántica y la caché).

3.6 Códigos de estado (Status Codes)

- **Qué son:** la respuesta HTTP lleva un código de 3 dígitos que resume el resultado.
- `1xx` : informativo.
- `2xx` : éxito → `200 OK`, `201 Created`, `204 No Content`.
- `3xx` : redirección → `301`, `304`.
- `4xx` : error del cliente → `400 Bad Request` (body malformado o validación fallida), `401 Unauthorized` (sin autenticar), `403 Forbidden` (sin permisos), `404 Not Found` (recurso inexistente), `409 Conflict`.
- `5xx` : error del servidor → `500 Internal Server Error`, `503`.
- **En el proyecto** (contrato impuesto por README y tests):
 - `POST /api/libraries/{id}/books` → `201` éxito, `404` si la librería no existe.
 - `GET /api/libraries/{id}/books` → `200` con lista, `404` si no existe la librería.
 - `DELETE /api/libraries/{id}` → `204` éxito, `404` si no existe.
 - `POST /login` sin `role` → `400`; credenciales malas → `401`.
 - `GET /api/libraries/{id}/books` sin token → `401` (cuando el endpoint estaba protegido).
- **Por qué importa:** los tests afirman códigos exactos (`Status201Created`, `Status404NotFound`). Elegir el código correcto es parte del contrato de la API, no un detalle.

3.7 Headers

- **Qué son:** metadatos del mensaje. Ejemplos: `Content-Type` (formato del body), `Content-Length`, `Authorization` (credenciales/token), `Accept` (formato deseado), `Set-Cookie`, `CORS` (`Access-Control-Allow-*`).
- **Para qué sirven:** negocian formato, autenticación, caché, controladores de acceso y cookies.
- **En el proyecto:**
 - `Content-Type: application/json` en los POST (lo genera ASP.NET Core).
 - `Authorization: Bearer <token>` — el header que activa el middleware JWT.
 - `Access-Control-Allow-Origin: http://localhost:5173` — producido por la política CORS.

3.8 Stateless (sin estado)

- **Qué es:** HTTP no recuerda nada entre peticiones. Cada request es independiente; el servidor no guarda la "sesión" de quién eres.
- **Para qué sirve:** permite escalar horizontalmente (cualquier servidor puede atender cualquier petición) y simplifica el protocolo.
- **Desventaja:** el servidor necesita otro mecanismo para saber *quién* eres → cookies con sesión o **tokens** (JWT). Esto explica por qué existe la autenticación por token del Módulo 7.
- **En el proyecto:** el JWT es la solución stateless elegida. El servidor no guarda sesión; valida la firma del token en cada request.

3.9 JSON

- **Qué es:** formato de datos de texto: `{ "clave": valor }`, con `null`, números, booleanos, strings, arreglos `[...]` y objetos anidados.

- **Para qué sirve:** es el "idioma" del body de las APIs REST. Los DTOs del proyecto (`BookForm` , `User`) se serializan/deserializan a JSON.
- **Reglas que importan:** nombres de propiedad en `camelCase` por convención REST (`libraryId`), no existen comentarios, `null` es distinto de "ausente".
- **En el proyecto:** `BookForm` usa `[JsonPropertyName("id")]` para fijar los nombres exactos en JSON; Newtonsoft (`[JsonProperty]`) se usa en los tests para serializar/deserializar.

3.10 Serialización / Deserialización

- **Qué es:** convertir objeto ↔ texto. Serializar: objeto → JSON. Deserializar: JSON → objeto.
- **En el proyecto:** ASP.NET Core deserializa el body del request en el parámetro del action (`BookForm bookForm`), y serializa la respuesta de `Ok(objeto)` a JSON automáticamente.

3.11 REST (Representational State Transfer)

- **Qué es:** estilo arquitectónico para APIs: recursos identificados por URL, operados con métodos HTTP, representaciones en JSON, stateless, respuestas con códigos de estado.
- **Ventajas:** predecible, cacheable, usable por cualquier cliente, descubre la estructura del dominio en las URLs.
- **Desventajas:** no prescribe estándares estrictos (¿plural?, ¿anidación?), lo que genera debates; no ideal para operaciones que no son CRUD.
- **En el proyecto:** URLs como `/api/libraries/{id}/books` muestran anidación de recursos; `LibrariesController` y `BooksController` son recursos REST.

3.12 HTTPS, TLS/SSL y Certificados

- **Qué es:** HTTPS = HTTP cifrado con TLS. Cifra todo el mensaje; el navegador verifica la identidad del servidor con certificados firmados por CA.
- **Para qué sirve:** confidencialidad (nadie lee tus datos), integridad (nadie los modifica) y autenticación del servidor.
- **En el proyecto:** la app expone `https://localhost:7098` (certificado de desarrollo). La connection string de Supabase incluye `SSL Mode=Require;Trust Server Certificate=true` → la conexión a la BD va cifrada.

3.13 Cookies vs Tokens

- **Qué son:** cookies: pares clave/valor que el navegador almacena y reenvía. Tokens: cadenas autónomas (JWT) que el cliente envía en el header.
- **Diferencia clave:** las cookies dependen del navegador y del servidor (suelen ser stateful); los tokens son stateless y funcionan en cualquier cliente (web, móvil, script).
- **En el proyecto:** se usa el modelo de tokens (`Authorization: Bearer`).

3.14 Clientes HTTP (cURL, Postman, Swagger UI)

- **Qué son:** herramientas para disparar requests manualmente y ver la respuesta.
- **En el proyecto:** son la forma de probar la API localmente; Swagger UI genera requests a partir del OpenAPI.

4. Conocimientos previos necesarios

- Uso básico de una terminal (crear carpetas, ejecutar comandos).
- Concepto de "programa que corre en una computadora" y "otra computadora en la red".
- No se requiere ningún lenguaje de programación para este módulo, aunque ayuda saber qué es una variable.

5. Cómo aparece dentro del proyecto

Archivo	Dónde aparece el concepto
<code>Controllers/LibrariesController.cs</code>	<code>[Route("api/[controller]")]</code> , <code>[HttpGet]</code> , <code>[HttpPost]</code> , <code>[HttpPut]</code> , <code>[HttpDelete]</code> , <code>return Ok(...)</code> , <code>NotFound()</code> , <code>NoContent()</code>

Archivo	Dónde aparece el concepto
Controllers/BooksController.cs	[Route("api/libraries/{libraryId}/{controller}")] , CreatedAtAction (→ 201 Created)
Controllers/AuthController.cs	[HttpPost("/login")] , return Unauthorized() (→ 401)
Startup.cs	app.UseCors("Frontend") → headers CORS; env.IsDevelopment()
appsettings.json	ConnectionStrings:DefaultConnection incluye host/puerto de Supabase
Properties/launchSettings.json	puertos https://localhost:7098 , http://localhost:5219
docs/DOCUMENTACION-TECNICA.md	tabla de pruebas en vivo con códigos 200/400/401/404
LibraryService.Integration.Test/IntegrationTest.cs	afirma Status201Created , Status200OK , Status404NotFound , Status204NoContent

6. Nivel de importancia

Fundamental — no existe ninguna comprensión de este proyecto sin HTTP.

7. Tiempo recomendado de estudio

8 horas (teoría 4 h + práctica 4 h).

8. Recursos recomendados

- **Documentación oficial:** RFC 7230-7235 (HTTP/1.1); MDN Web Docs "HTTP" (<https://developer.mozilla.org/es/docs/Web/HTTP>).
- **Microsoft Learn:** "Explore the basics of web API" (module `build-web-api-net`).
- **Libros:** *HTTP: The Definitive Guide* (Gourley & Totty, O'Reilly).
- **Videos:** "REST API concepts and examples" (WebConcepts, YouTube); "HTTP Crash Course" (Traversy Media).
- **Cursos:** Postman Learning Center; *API Security for the absolute beginner*.
- **Repositorios:** `public-apis/public-apis` (listas reales de APIs para probar).
- **Herramienta:** Postman Desktop o Thunder Client (VS Code), cURL.

9. Ejercicios sugeridos

1. (Fácil) Abre Swagger de la app en `https://localhost:7098/swagger` y ejecuta `GET /api/libraries` . Anota método, URL, headers y status code de la petición y de la respuesta.
2. (Fácil) Con cURL, haz `curl -i https://localhost:7098/api/libraries` y explica cada línea del header de respuesta.
3. (Fácil) Haz `POST /api/libraries` con un JSON válido y otro inválido; compara `200` vs `400` .
4. (Medio) Dibuja la anatomía de un request `POST /api/libraries/1/books` y de su respuesta `201` (línea de inicio, headers y body).
5. (Medio) Prueba los 4 verbos sobre `/api/libraries` y completa una tabla: método, URL, código, body.
6. (Medio) Verifica la idempotencia: dos `DELETE /api/libraries/1` → primero `204` , segundo `404` . Explica por qué.
7. (Difícil) Usa F12 del navegador → pestaña Network mientras haces requests en Swagger; identifica los headers de CORS en la respuesta.
8. (Difícil) Explica por qué `POST /login` con credenciales malas devuelve `401` y no `404` .

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Confundir <code>401</code> con <code>403</code>	<code>401</code> llega sin token o con token inválido; <code>403</code> con token válido pero sin permiso	Estudiar la diferencia: <i>autenticación</i> vs <i>autorización</i>

Error	Cómo detectarlo	Cómo solucionarlo
Usar <code>POST</code> para todo	URL con verbo ("/deleteBook"), semántica rota	Usar el verbo HTTP correcto
Ignorar el body en <code>DELETE</code> / <code>204</code>	Respuestas <code>204</code> no llevan body	No escribir JSON en un <code>204</code>
Creer que HTTP "recuerda" al usuario	Peticiones repetidas sin sesión	Comprender stateless; la identidad va en el header <code>Authorization</code>
No distinguir query string de ruta	<code>{libraryId}</code> en la URL vs <code>?name=...</code>	Ruta identifica el recurso; query filtra/opciones
Olvidar <code>Content-Type</code>	El servidor responde <code>415 Unsupported Media Type</code>	Enviar <code>Content-Type: application/json</code>

MÓDULO 2 — C# y la plataforma .NET

1. Nombre del módulo

C# moderno, .NET, el SDK, la CLI y la estructura de una solución

2. Objetivo de aprendizaje

El estudiante leerá y escribirá código C# con fluidez: tipos, objetos, interfaces, records, `async/await`, nullable reference types, LINQ, genéricos y manejo de errores. Además entenderá la plataforma .NET: CLR, runtime, SDK, CLI, NuGet, soluciones, proyectos y ensamblados — la infraestructura exacta sobre la que corre esta API.

3. Conceptos fundamentales

3.1 C# (el lenguaje)

- **Qué es:** lenguaje de programación orientado a objetos, tipado estático, de propósito general, creado por Microsoft.
- **Para qué sirve:** escribir todo el código del proyecto (controllers, servicios, entidades).
- **Cuándo se utiliza:** desde scripts pequeños hasta sistemas empresariales. Es el lenguaje de este proyecto.
- **Ventajas:** rendimiento, ecosistema enorme (NuGet), soporte de Microsoft, sintaxis moderna, `async/await` nativo, null-safety progresiva.
- **Desventajas:** curva de conceptos (interfaces, genéricos, DI), dependencia del ecosistema Microsoft.
- **Relación:** es el lenguaje; .NET es la plataforma que lo ejecuta.

3.2 .NET (la plataforma)

- **Qué es:** plataforma unificada (runtime + bibliotecas de clases + herramientas) para construir apps de consola, web, móvil, escritorio y cloud.
- **Versiones:** el proyecto usa **.NET 8.0** (LTS — soporte a largo plazo). La versión define el `TargetFramework` (`net8.0` en el `.csproj`).
- **Para qué sirve:** provee el BCL (Base Class Library): colecciones, LINQ, IO, HTTP, JSON, etc.
- **Ventaja:** un solo runtime para todos los tipos de app; fuerte performance.
- **Desventaja:** versiones mayores rompen compatibilidad (por eso 6.0 ≠ 8.0 causó el bug del Módulo 9).

3.3 CLR (Common Language Runtime)

- **Qué es:** la máquina virtual que ejecuta el código .NET: JIT (compilación just-in-time), gestión de memoria (GC — garbage collector), tipos, seguridad, threading.
- **Para qué sirve:** tu C# se compila a IL (Intermediate Language) dentro de un **ensamblado** (`.dll` / `.exe`), y el CLR lo ejecuta y gestiona la memoria.
- **Relación:** GC explica por qué no liberas memoria a mano (a diferencia de C++). La interoperabilidad entre lenguajes .NET se da porque todos compilan a IL.

3.4 SDK vs Runtime

- **SDK** (Software Development Kit): compilador + herramientas + runtimes (desarrollo). Instalación para *crear*.
- **Runtime:** solo ejecuta apps ya compiladas (producción/servidores).
- **Para qué sirve:** en tu máquina necesitas el SDK (tienes .NET 8); en un servidor de producción basta el runtime o una app self-contained.
- **En el proyecto:** la app se publica compilada; la máquina dev requiere SDK 8.

3.5 CLI de .NET (dotnet)

- **Qué es:** herramienta de terminal para todo el ciclo: `dotnet new`, `build`, `run`, `test`, `restore`, `add package`, `tool`, `user-secrets`, `ef`.

- **En el proyecto** (comandos reales de la documentación):
- `dotnet build` → compila la solución.
- `dotnet run --project HackerRank1` → arranca la API (obligatorio `--project`, hay 2 proyectos).
- `dotnet test` → ejecuta los tests de integración.
- `dotnet tool install --global dotnet-ef --version 8.0.2` → instala la herramienta de migraciones.
- `dotnet user-secrets set "ConnectionStrings:DefaultConnection" "..."` → guarda la credencial.
- `dotnet ef migrations list --project HackerRank1` → consulta estado de migraciones.
- **Error común:** `dotnet run` sin `--project` en la raíz de una solución de 2 proyectos falla con "No se ha podido encontrar un proyecto para ejecutar".

3.6 NuGet

- **Qué es:** el gestor de paquetes de .NET (equivalente a npm/pip). Los paquetes se referencian en el `.csproj` dentro de `<PackageReference>`.
- **Para qué sirve:** incorporar bibliotecas de terceros y oficiales.
- **En el proyecto** (`HackerRank1.csproj`):
- `Microsoft.EntityFrameworkCore` y `.Design` 8.0.2
- `Npgsql.EntityFrameworkCore.PostgreSQL` 8.0.2
- `Microsoft.AspNetCore.Authentication.JwtBearer` 8.0.16
- `Swashbuckle.AspNetCore` 6.5.0
- En tests: `xunit`, `FluentAssertions`, `Microsoft.AspNetCore.Mvc.Testing`, `Microsoft.EntityFrameworkCore.Sqlite`, `Microsoft.EntityFrameworkCore.InMemory`, `Newtonsoft.Json`.
- **Conceptos:** `PackageReference` con versión explícita; **dependencias transitivas** (un paquete trae otros) — causa del bug de versiones 6.0 vs 8.0 en tests.

3.7 Solución (.sln), Proyecto (.csproj), Ensamblado (Assembly)

- **Solución:** contenedor de uno o más proyectos (`HackerRank1.sln`).
- **Proyecto:** unidad de compilación con su `.csproj` (SDK, framework, paquetes). Este proyecto tiene 2: la Web API y el proyecto de tests.
- **Ensamblado:** resultado compilado de un proyecto (`.dll` + `.exe` de host). `Program.cs` de la API se compila en `HackerRank1.dll`.
- **Relación:** la referencia del proyecto de tests a la API (`<ProjectReference Include="..\HackerRank1\HackerRank1.csproj" />`) hace que los tests puedan usar `Program` y `Startup`.

3.8 Namespaces

- **Qué es:** contenedor lógico de tipos, análogo a carpetas. Evita colisiones de nombres.
- **En el proyecto:** `LibraryService.WebAPI` (base), `.Controllers`, `.Services`, `.Data`, `.DTO`, `HackerRank1.Entities`, `HackerRank1.Helpers`. El `using` en cada archivo importa los namespaces usados.

3.9 Tipos de valor vs tipos de referencia

- **Valor** (`int`, `bool`, `struct`): copia del dato; vive en el stack.
- **Referencia** (`class`, `string`, `array`): el "objeto" vive en el heap; la variable guarda una referencia.
- **Por qué importa:** pasar una `Library` por parámetro pasa la *misma* instancia (los cambios dentro del método persisten). Comparar con `==` compara referencias, no contenido.

3.10 Orientación a objetos: clases, objetos, herencia, encapsulamiento, polimorfismo

- **Clase** (`Library`, `Book`, `User`): plantilla con propiedades y métodos.
- **Encapsulamiento:** ocultar estado (propiedades `private set`, campos privados).
- **Herencia:** reutilizar comportamiento (menos usada aquí; prefieren composición/DI).
- **Polimorfismo:** "misma firma, distinto comportamiento" — clave en interfaces (`ILibrariesService` implementada por `LibrariesService`).

- **Abstracción:** las interfaces (`IAuthenticationService` , `ILibrariesService` , `IBooksService`) definen *contratos*; el DI resuelve la implementación concreta.

3.11 Interfaces

- **Qué es:** contrato de miembros (métodos, propiedades) sin implementación.
- **En el proyecto:**
 - `IAuthenticationService.AuthenticateAsync(string email, string password) → Task<User>` .
 - `ILibrariesService → Get, Add, AddRange, Update, Delete` .
 - `IBooksService → Get, Add, Update, Delete` .
- **Por qué importan:** son el corazón de la Inyección de Dependencias (Módulo 3): registras "interfaz → clase" y el contenedor construye la clase.

3.12 Records

- **Qué es:** tipo de referencia inmutable diseñado para datos; comparación por valor, sintaxis concisa.
- **En el proyecto:** `public record TokenResponse(string token);` — el objeto de respuesta del login. `Newtonsoft.Json` lo serializa como `{ "token": "..."} .`

3.13 Nullable Reference Types

- **Qué es:** con `<Nullable>enable</Nullable>` , el compilador marca tipos de referencia como no-nulables por defecto y te avisa si un `string` puede ser `null` .
- **En el proyecto:** `public string? Category` (BookForm) — el `?` declara que sí puede ser `null` . `public string Name` sin `?` significa "no debe ser null", y el compilador emite warning `CS8618` si no se inicializa.
- **Relación con ASP.NET Core:** con `[ApiController]` , una propiedad `string` no-nulable en un DTO se vuelve **obligatoria** en el model binding → el `400 "The Role field is required."` del bug #4 de la bitácora. Por eso `Role` y `Category` son `string?` .
- **Herramientas:** el operador `??` (`bookForm.Category ?? string.Empty` = "si es null, usa string vacío") y `?.` .

3.14 Async/Await y Task

- **Qué es:** `async/await` permite código no bloqueante: mientras se espera I/O (BD, red), el hilo queda libre. Un método `async` devuelve `Task` (void) o `Task<T>` (valor).
- **En el proyecto:** casi todo es async:
 - `await _libraryContext.Libraries.AddAsync(library)` — I/O de BD.
 - `await _librariesService.Get(new[] { libraryId })` .
 - `Task<IActionResult>` en todos los actions de los controllers.
- **Reglas:** no usar `async void` salvo handlers; no bloquear con `.Result` / `.Wait()` (¡el test usa `.Result` y provoca el warning `xUnit1031` !). El flujo async requiere que toda la cadena (controller → service → EF) sea async.
- **Ventaja:** escalabilidad — una API async atiende miles de requests con pocos hilos.

3.15 LINQ (Language Integrated Query)

- **Qué es:** sintaxis integrada al lenguaje para consultar colecciones y datos: `Where` , `Select` , `Any` , `ToListAsync` , `FirstOrDefault` , `Contains` , etc.
- **En el proyecto:**
 - `_libraryContext.Libraries.AsQueryable() + .Where(x => ids.Contains(x.Id)) + .ToListAsync()` .
 - `(await _librariesService.Get(new[] { libraryId })).FirstOrDefault()` .
 - `services.Any(d => d.ServiceType == typeof(JwtSettings))` (en Startup).
 - `db.Database.GetPendingMigrations().Any()` .
- **Importante:** LINQ sobre `IQueryable` se traduce a SQL (Módulo 5); LINQ sobre listas en memoria (`List`) ejecuta en el proceso.

3.16 Genéricos

- **Qué es:** tipos parametrizables: `List<T>`, `Task<T>`, `IEnumerable<T>`, `DbSet<T>`, `ServiceCollection`, `DbContextOptions<LibraryContext>`.
- **En el proyecto:** `Task<IEnumerable<Library>>`, `AddDbContextPool<LibraryContext>(...)`, `GetRequiredService<LibraryContext>()`.

3.17 Manejo de excepciones

- **Qué es:** mecanismo para propagar errores: `try/catch/finally`, tipos de excepción (`InvalidOperationException`, `DbUpdateException`, etc.), `throw`.
- **En el proyecto:** `?? throw new InvalidOperationException("Invalid JWT Settings")` en `Startup.ConfigureServices` — si falta la config de JWT, la app **falla al arrancar** en vez de correr con valores malos.
- **Uso estratégico:** excepciones para errores *imprevistos*; códigos de estado HTTP (`NotFound()`) para resultados *esperados* (recurso inexistente).

3.18 Expresiones Lambda y delegates

- **Qué es:** funciones anónimas: `x => x.Id == library.Id`. Los delegates son tipos de función (`Func<>`, `Action<>`).
- **En el proyecto:** en cada `.Where(...)`, `.AddTransient<TInterface, TClass>(...)`, `webBuilder.UseStartup<Startup>()`.

3.19 `static` y métodos de extensión

- **Qué es:** `static` = miembro de la clase, no de la instancia. Métodos de extensión: métodos estáticos que "parecen" métodos de instancia (firma `this Tipo param`).
- **En el proyecto:** `TokenGenerator.GenerateToken(...)` es estático (no necesita estado). `StringComparison.OrdinalIgnoreCase` usa un método de extensión del framework. Los middleware `app.UseRouting()`, `app.UseAuthentication()` son métodos de extensión.

4. Conocimientos previos necesarios

- Módulo 1 (HTTP) para el vocabulario de red, aunque es opcional para este módulo.
- Fundamentos de programación (variables, condicionales, bucles, funciones).
- Si nunca programaste: estudiar primero un curso de lógica y luego regresar.

5. Cómo aparece dentro del proyecto

Archivo	Conceptos C#/.NET
<code>HackerRank1.csproj</code>	<code>net8.0</code> , <code>Nullable</code> , <code>ImplicitUsings</code> , <code>PackageReference</code> , <code>UserSecretsId</code>
<code>Program.cs</code>	<code>Main</code> , <code>IHostBuilder</code> , métodos estáticos, <code>using</code> directives
<code>Services/*.cs</code>	interfaces + clases, <code>async Task<T></code> , LINQ, DI por constructor
<code>Helpers/TokenGenerator.cs</code>	clase estática, <code>static</code> , arrays de <code>Claim</code>
<code>Controllers/AuthController.cs</code>	<code>record TokenResponse(string token)</code>
<code>DTO/BookForm.cs</code>	<code>string? Category</code> , atributos <code>[JsonPropertyName]</code>
<code>Data/LibraryContext.cs</code>	clases <code>Book</code> / <code>Library</code> con propiedades auto-implementadas
<code>Startup.cs</code>	genéricos (<code>IServiceCollection</code> , <code>DbSet</code>), excepciones, lambdas
<code>LibraryService.Integration.Test/*</code>	xUnit <code>[Fact]</code> , <code>FluentAssertions</code> , <code>JsonConvert</code> (Newtonsoft)

6. Nivel de importancia

Fundamental — es el idioma en que está escrito todo.

7. Tiempo recomendado de estudio

25 horas (15 h teoría + 10 h práctica).

8. Recursos recomendados

- **Documentación oficial:** Microsoft Learn "C# Guide"; "Tour of C#".
- **Microsoft Learn:** "Write your first C# code" (path C# en learn.microsoft.com).
- **Libros:** *C# in a Nutshell* (Albahari, O'Reilly); *C# 12 in a Nutshell*; *Head First C#* (para empezar de cero).
- **Videos:** "C# Fundamentals for Beginners" (Microsoft Developer, 60+ capítulos gratis).
- **Cursos:** Microsoft Learn path "C# for Absolute Beginners"; Pluralsight "C# Language Fundamentals".
- **Repositorios:** `dotnet/samples` ; `JustArchiNET/ArchiSteamFarm` (código C# real, grande).
- **Herramienta:** `dotnet script` para probar snippets sin crear proyectos.

9. Ejercicios sugeridos

1. (Fácil) Consola "Hola mundo" con `dotnet new console` . Agrega un `record` , una clase, una interfaz y un método async.
2. (Fácil) Reescribe manualmente el DTO `BookForm` con y sin `[JsonPropertyName]` ; compara el JSON resultante con `System.Text.Json` .
3. (Medio) Crea `List<Library>` con 5 elementos y usa LINQ: `Where` , `FirstOrDefault` , `Any` , `OrderBy` , `Select` , `Contains` .
4. (Medio) Escribe una versión tuya de `TokenResponse` y serialízala con Newtonsoft y con System.Text.Json; compara la salida.
5. (Medio) Explica por qué `books.Count().Should().Be(2)` funciona si `books` es `List<Book>` (pista: extension methods, genéricos).
6. (Difícil) Simula la lógica de `AuthenticationService.AuthenticateAsync` como método async real que use `Task.Delay(100)` ; invócalo con `await` y compara con usar `.Result` (observa el bloqueo).
7. (Difícil) Reproduce el bug de nullable: una clase `User` con `public string Role` (sin `?`) + `[ApiController]` → observa el `400` cuando falta `role` ; luego cámbiala a `string?` y observa el cambio.
8. (Difícil) Usa `dotnet ef migrations script` para ver el SQL que genera EF; explica las palabras SQL a partir del C# de la entidad.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Bloquear el hilo con <code>.Result</code> / <code>.Wait()</code>	Warnings <code>xUnit1031</code> , apps "colgadas", deadlocks en UI	Usar <code>await</code> en toda la cadena async
Usar <code>async void</code>	Excepciones que crashean el proceso	Usar <code>Task</code> / <code>Task<T></code>
Propiedades no-nulables sin inicializar	Warning <code>CS8618</code>	Inicializar o declarar <code>string?</code>
Comparar clases con <code>==</code>	Dos "iguales" distintos	Usar records o comparar propiedades
Ignorar el <code>TargetFramework</code>	NuGet de 6.0 vs 8.0 rompiendo build	Alinear versiones de paquetes y framework (bug real del proyecto)
<code>dotnet run</code> sin <code>--project</code>	Error "no se encontró un proyecto para ejecutar"	Especificar <code>--project HackerRank1</code>

MÓDULO 3 — ASP.NET Core

1. Nombre del módulo

ASP.NET Core: hosting, pipeline, middleware, DI, configuración, logging, routing y controllers

2. Objetivo de aprendizaje

El estudiante comprenderá cómo arranca esta API, cómo se registran los servicios, en qué orden se ejecuta el middleware y cómo los controllers se convierten en endpoints HTTP. Podrá explicar línea por línea `Program.cs` y `Startup.cs` y sabrá construir una Web API minimalista desde cero.

3. Conceptos fundamentales

3.1 ASP.NET Core

- **Qué es:** framework de Microsoft para construir aplicaciones web y APIs. Es **multi-plataforma**, de alto rendimiento, y se ejecuta sobre .NET.
- **Para qué sirve:** aquí construye la Web API LibraryService.
- **Cuándo se utiliza:** APIs REST, MVC, Razor Pages, Blazor, SignalR, gRPC.
- **Ventajas:** alto rendimiento (Kestrel), DI integrada, configuración por niveles, middleware extensible, soporte nativo de OpenAPI.
- **Desventajas:** ecosistema grande con muchas formas de hacer lo mismo (versionado, minimal APIs vs controllers), curva de conceptos (middleware, pipeline, lifetimes).

3.2 El modelo de Hosting

- **Qué es:** el *host* es el objeto que posee el runtime de la app: el servidor, el contenedor de DI, la configuración y el pipeline de peticiones.
- **Modelo moderno vs legacy:**
- **Moderno:** `Program.cs` con top-level statements: `var builder = WebApplication.CreateBuilder(args);` → `var app = builder.Build();` → `app.Run();`.
- **Legacy (este proyecto):** `Host.CreateDefaultBuilder(args).ConfigureWebHostDefaults(webBuilder => webBuilder.UseStartup<Startup>())`. `Program` solo construye el host; toda la configuración vive en `Startup.cs`.
- **Por qué importa:** saber distinguir ambos modelos te permite leer proyectos viejos y nuevos. Este proyecto es el modelo legacy con `ConfigureServices` (registrar servicios) y `Configure` (montar pipeline).

3.3 Program.cs en el proyecto

- `Main(string[] args)` es el punto de entrada.
- `CreateHostBuilder(args).Build().Run()` : crea el host, lo construye y lo ejecuta (bloquea hasta parar).
- `Host.CreateDefaultBuilder(args)` : aplica convenciones por defecto — carga configuración, registra logging, y en `Development` agrega **User Secrets** (Módulo 6).
- `.ConfigureWebHostDefaults(...)` : habilita Kestrel, IIS Integration, y por defecto agrega `appsettings.json`, `appsettings.{Environment}.json`, variables de entorno, argumentos de línea de comandos.
- `.UseStartup<Startup>()` : instruye al host a usar `Startup` para `ConfigureServices` y `Configure`.

3.4 Startup.cs — estructura y contratos

- `Startup(IConfiguration configuration)` : recibe la configuración ya cargada por el host (constructor injection en el propio Startup).
- `ConfigureServices(IServiceCollection services)` : **registra** servicios en el contenedor de DI. No se usa el contenedor aquí.
- `Configure(IApplicationBuilder app, IWebHostEnvironment env)` : **monta** el pipeline de peticiones (middleware) y usa el contenedor (los servicios registrados se inyectan como parámetros si lo pides).

- **Detalle importante del proyecto:** `ConfigureServices` incluye un **guard de idempotencia** (`if (services.Any(d => d.ServiceType == typeof(JwtSettings))) return;`) para soportar que el host de tests lo invoque dos veces (bug real "Scheme already exists: Bearer").

3.5 Middleware

- **Qué es:** piezas de software que procesan cada request en cadena. Cada middleware decide si maneja el request o lo pasa al siguiente (y opcionalmente procesa la respuesta al volver). Forman el *pipeline*.
- **Para qué sirve:** cross-cutting concerns: logging, autenticación, autorización, CORS, manejo de errores, endpoints.
- **En el proyecto** (orden del pipeline en `Configure`): 1. `UseDeveloperExceptionPage()` (solo Development) — página de errores detallada. 2. `UseSwagger()` / `UseSwaggerUI()` (solo Development). 3. Auto-migración de BD (scope + `Migrate()`), **antes** de montar el pipeline de requests. 4. `UseRouting()` — selecciona el endpoint. 5. `UseCors("Frontend")` — política CORS. 6. `UseAuthentication()` — **valida el token JWT** y construye el `User`. 7. `UseAuthorization()` — **aplica las políticas de permisos** (`[Authorize]`). 8. `UseEndpoints(...)` → `MapControllers()` — despacha al controller/acción.
- **Orden crítico:** `Authentication` **antes** de `Authorization`, y ambos **antes** de `Endpoints`. Si se invierte, `[Authorize]` no funciona.

3.6 Dependency Injection (DI)

- **Qué es:** patrón y contenedor integrado: no construyes dependencias con `new`; las *declaras* como parámetros de constructor y el contenedor las instancia y destruye por ti.
- **Conceptos:**
- *Interfaz* → contrato. *Implementación* → clase concreta.
- *Registro:* `services.AddScoped<IAuthenticationService, AuthenticationService>()`.
- *Resolución:* el constructor `LibrariesController(ILibrariesService librariesService)` recibe automáticamente la implementación.
- **Lifetimes** (ciclos de vida):
- **Transient** (`AddTransient`): nueva instancia por resolución. `ILibrariesService`, `IBooksService`.
- **Scoped** (`AddScoped`): una por request. `IAuthenticationService`, `LibraryContext` (via pooling).
- **Singleton** (`AddSingleton`): una para toda la vida de la app. `JwtSettings`.
- **Regla de oro:** un servicio no puede depender de otro con vida *más corta* (singleton no puede depender de scoped — "cannot consume scoped service from singleton").
- **En el proyecto:**
- `services.AddSingleton(jwtSettings)` — la config de JWT es inmutable y global.
- `services.AddScoped<IAuthenticationService, AuthenticationService>()`.
- `services.AddTransient<ILibrariesService, LibrariesService>()` y `AddTransient<IBooksService, BooksService>()`.
- `AddDbContextPool<LibraryContext>` — variante *scoped* con pool de 20 contextos.
- **Resolución manual** (en Startup): `scope.ServiceProvider.GetRequiredService<LibraryContext>()` — usado para la auto-migración.

3.7 Configuration (IConfiguration)

- **Qué es:** sistema de configuración por *proveedores en cadena* (el último gana): `appsettings.json` → `appsettings.{Environment}.json` → User Secrets (Development) → variables de entorno → args.
- **En el proyecto:** `Configuration.GetSection("JwtSettings").Get<JwtSettings>()` (binding a clase tipada) y `Configuration.GetConnectionString("DefaultConnection")`.
- **Error común:** la connection string real en `appsettings.json` (debería ir en secretos/env) — ver Módulos 6 y 13.

3.8 Entornos (Development / Production)

- **Qué es:** `ASPNETCORE_ENVIRONMENT` define el entorno; `env.IsDevelopment()` activa bloques condicionales.
- **En el proyecto:** `launchSettings.json` fija `Development`; solo en Development: Swagger, página de errores dev, User Secrets.
- **Por qué importa:** User Secrets **no** se cargan en Production (causa de "BD no conecta en prod" documentado en el repo).

3.9 Kestrel y puertos

- **Qué es:** Kestrel es el servidor web embebido de ASP.NET Core.

- **En el proyecto:** `launchSettings.json` define `https://localhost:7098;http://localhost:5219`. El perfil "HackerRank1" (`commandName: Project`) usa Kestrel. El perfil "IIS Express" existe para Visual Studio.

3.10 Routing y Controllers

- **Qué es:** el routing mapea una URL + verbo HTTP → una acción de un controller.
- **Attribute routing:** `[Route("api/[controller]")]` donde `[controller]` se sustituye por el nombre del controller sin sufijo → `LibrariesController` → `api/libraries`.
- **En el proyecto:**
 - `LibrariesController`: `[Route("api/[controller]")]` → `api/libraries`; `[HttpGet("{libraryId}")]` → `api/libraries/{id}`.
 - `BooksController`: `[Route("api/libraries/{libraryId}/[controller]")]` → `api/libraries/{libraryId}/books` (ruta anidada).
 - `AuthController`: `[HttpPost("/login")]` → ruta absoluta `/login`.
- **Model binding:** los parámetros se llenan desde la ruta (`int libraryId`), el body JSON (`BookForm bookForm`, `Library l`, `User user`) o el query string.
- **Convenciones de nombre:** `LibrariesController` → controlador de `Libraries`; las acciones `GetAll`, `Get`, `Add`, `Update`, `Delete` se mapean por atributos HTTP.

3.11 [ApiController]

- **Qué es:** atributo que activa comportamiento automático de API: inferencia de binding, **validación automática del modelo** (si el body no cumple los atributos → `400` automático con `ProblemDetails`), respuesta de error estructurada.
- **En el proyecto:** `[ApiController]` en los 3 controllers. La validación automática explica el `400 "The Category field is required."` cuando `Category` era no-nulable.
- **Detalle:** con `[ApiController]`, `Ok(...)`, `NotFound()` etc. devuelven `ActionResult<T>` con negociación de contenido.

3.12 ActionResult e IActionResult

- **Qué es:** tipos de retorno de las acciones: `Ok(obj)` → `200`, `NotFound()` → `404`, `NoContent()` → `204`, `Unauthorized()` → `401`, `CreatedAtAction(...)` → `201`.
- **En el proyecto:**
 - `Ok(libraries)`, `NotFound()`, `NoContent()` en `LibrariesController`.
 - `CreatedAtAction(nameof(GetAll), new { libraryId = createdBook.LibraryId }, createdBook)` en `BooksController` — responde `201` e incluye la Location del recurso.
 - `Unauthorized()` en `AuthController` cuando falla el login.

3.13 Servicios y Asincronía en el flujo

- **Qué es:** el flujo completo: request → routing → middleware → controller → service → DbContext → BD → respuesta.
- **En el proyecto:** `LibrariesController` no toca EF; delega en `ILibrariesService`. `BooksController` coordina dos servicios (librería existe + libro). Esto es la capa de aplicación.

3.14 Logging

- **Qué es:** `ILogger<T>` y el sistema de logs por niveles (`Trace/Debug/Information/Warning/Error/Critical`), configurable en `appsettings.Development.json`.
- **En el proyecto:** configurado `Default: Information`, `Microsoft.AspNetCore: Warning`. Los controllers no loguean (mala práctica, ver Módulo 12), pero la infraestructura ya está lista.

3.15 launchSettings.json

- **Qué es:** archivo de Visual Studio/`dotnet run` con perfiles de arranque: puertos, entorno, navegador.
- **En el proyecto:** perfil `HackerRank1` → `launchUrl: swagger`, puertos 7098/5219, `ASPNETCORE_ENVIRONMENT=Development`.

4. Conocimientos previos necesarios

- Módulo 1 (HTTP) — imprescindible para entender requests, rutas y status codes.
- Módulo 2 (C#/.NET) — los controllers y Startup están escritos en C#.

5. Cómo aparece dentro del proyecto

Archivo	Concepto
<code>Program.cs</code>	hosting legacy: <code>Host.CreateDefaultBuilder</code> , <code>ConfigureWebHostDefaults</code> , <code>UseStartup<Startup></code>
<code>Startup.cs</code>	<code>ConfigureServices</code> (DI, JWT, CORS, DbContextPool, Swagger), <code>Configure</code> (pipeline: <code>DeveloperExceptionPage</code> , Swagger, Migrate, Routing, CORS, Authentication, Authorization, Endpoints)
<code>Controllers/*.cs</code>	<code>[ApiController]</code> , <code>[Route]</code> , <code>[HttpGet]</code> , <code>[HttpPost]</code> , DI por constructor, <code>ActionResult</code>
<code>Properties/launchSettings.json</code>	perfiles, puertos, entorno
<code>appsettings.json</code> / <code>.Development.json</code>	Configuration providers, logging
<code>HackerRank1.csproj</code>	<code>Microsoft.NET.Sdk.Web</code> (el SDK de web)
<code>LibraryService.Integration.Test</code>	<code>WebApplicationFactory<Program></code> reutiliza el host real

6. Nivel de importancia

Fundamental — es la plataforma sobre la que vive el proyecto.

7. Tiempo recomendado de estudio

20 horas (teoría 10 h + práctica 10 h).

8. Recursos recomendados

- **Documentación oficial:** "ASP.NET Core fundamentals", "ASP.NET Core Middleware", "Dependency injection in ASP.NET Core", "Configuration in ASP.NET Core".
- **Microsoft Learn:** "Create a web API with ASP.NET Core controllers" (module `build-web-api-minimal-api`).
- **Libros:** *Pro ASP.NET Core 8* (Adam Freeman, APress); *Ultimate ASP.NET Core Web API* (Practical ASP.NET Core).
- **Videos:** "ASP.NET Core Middleware" y "ASP.NET Core Dependency Injection" (Nick Chapsas, YouTube).
- **Cursos:** freeCodeCamp "ASP.NET Core Web API course" (YouTube); Pluralsight "Building a Web API with ASP.NET Core".
- **Repositorios:** `dotnet/aspnetcore` (samples); `KevinDockx/AspNetCore8WebAPIFundamentals`.

9. Ejercicios sugeridos

1. (Fácil) `dotnet new webapi -n MiApi` → abre `Program.cs` y compara con el modelo legacy de `HackerRank1`. Señala las diferencias.
2. (Fácil) Agrega al pipeline un middleware propio con `app.Use(async (ctx, next) => {...})` que loguee el tiempo de cada request.
3. (Medio) Registra un servicio Transient vs Scoped vs Singleton y observa con un GUID cómo cambia entre requests.
4. (Medio) Reproduce el bug "Scheme already exists: Bearer": registra JWT dos veces en `ConfigureServices` y observa el error.
5. (Medio) Crea un controller con acciones GET/POST/PUT/DELETE que devuelvan `Ok/NotFound/NoContent/CreatedAtAction` y prueba cada status con cURL.
6. (Difícil) Explica por qué `UseAuthentication()` debe ir antes de `UseAuthorization()` y demuéstalo invirtiendo el orden con un endpoint `[Authorize]`.
7. (Difícil) Crea una política CORS restrictiva y una abierta; prueba con un cliente que envíe `Origin` y observa los headers `Access-Control-Allow-Origin`.
8. (Difícil) Mueve la app del modelo legacy al modelo moderno (top-level statements + builder) manteniendo el comportamiento; documenta qué cambia.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Middleware en orden incorrecto	401 aunque el token sea válido	<code>UseAuthentication</code> antes de <code>UseAuthorization</code>
Singleton que depende de scoped	<code>InvalidOperationException: Cannot consume scoped service</code>	Bajar el lifetime o usar un factory
Olvidar <code>UseRouting</code> / <code>UseEndpoints</code>	404 en todas las rutas	Montar el pipeline completo
Registrar JWT dos veces	<code>InvalidOperationException: Scheme already exists: Bearer</code>	Guard de idempotencia (como el del proyecto) o registrar una sola vez
<code>[ApiController]</code> + <code>string</code> no-nulable	400 The X field is required.	Declarar <code>string?</code> cuando el campo es opcional
Configurar servicios después de <code>Build()</code>	El contenedor está "congelado"	Registrar todo en <code>ConfigureServices</code> (o antes de <code>Build()</code> en el modelo moderno)
Creer que <code>Program.cs</code> moderno y legacy son iguales	Confusión al migrar	Saber que legacy separa <code>ConfigureServices</code> / <code>Configure</code>

MÓDULO 4 — Arquitectura del proyecto

1. Nombre del módulo

Arquitectura de software: capas, Clean Architecture, Vertical Slice y el patrón Repository

2. Objetivo de aprendizaje

El estudiante entenderá *por qué* el proyecto está organizado como está, qué alternativas existen (Clean Architecture, Vertical Slice, Repository Pattern), **por qué este proyecto NO usa Repository**, y sabrá evaluar los tradeoffs de una decisión arquitectónica — la habilidad más importante de un arquitecto.

3. Conceptos fundamentales

3.1 Arquitectura de software

- **Qué es:** decisiones estructurales de alto nivel sobre un sistema: cómo se organizan sus partes, cómo se comunican, qué responsabilidades tiene cada una.
- **Para qué sirve:** mantener el sistema comprensible, testeable y evolucionable a medida que crece.
- **Cuándo se utiliza:** siempre que diseñas un sistema. La decisión arquitectónica más barata de cambiar es la que tomas al inicio.
- **Ventajas de una buena arquitectura:** testeabilidad, mantenibilidad, separación de responsabilidades, despliegue independiente.
- **Desventajas/costos:** complejidad, más archivos, más abstracción, más tiempo inicial.
- **Relación:** la arquitectura determina cómo se distribuyen los controllers, servicios y datos (los Módulos 3 y 5).

3.2 Separación de responsabilidades (SoC)

- **Qué es:** dividir el sistema en partes con una sola responsabilidad clara.
- **En el proyecto:** Controller (HTTP) → Service (reglas de aplicación/acceso a datos) → DbContext (persistencia). Cada pieza hace una cosa.

3.3 Arquitectura en capas (Layered / N-tier)

- **Qué es:** organiza el software en capas horizontales: Presentación → Aplicación/Servicios → Persistencia → Base de datos. Las dependencias fluyen hacia abajo; cada capa depende de la inmediata inferior.
- **En el proyecto:** `Controller (Presentación) → Service (Aplicación) → DbContext (Persistencia) → PostgreSQL`
- **Ventajas:** simple, familiar, fácil de razonar, cada capa puede testearse.
- **Desventajas:** tendencia a que la lógica de negocio "se filtre" hacia arriba (controllers gordos) o hacia abajo (servicios que son mero CRUD de BD); a gran escala se vuelve "anemia" y el cruce de dependencias se complica.
- **Relación:** es la base del proyecto; las alternativas (Clean/Hexagonal/Vertical Slice) atacan sus debilidades.

3.4 Clean Architecture

- **Qué es:** capas *concéntricas* con dependencias hacia adentro (hacia el dominio): Entities → Use Cases → Interface Adapters → Frameworks. El dominio no conoce a los frameworks.
- **Ventajas:** máxima independencia del framework, testabilidad, el dominio sobrevive a cambios de infraestructura (cambiar de BD no toca la lógica de negocio).
- **Desventajas:** más boilerplate (interfaces + implementaciones para cada adaptador), puede resultar sobre-ingeniería para proyectos pequeños.
- **Cuándo elegirla:** sistemas grandes con dominio complejo y vida larga. Para un lab/CRUD simple es excesiva.

3.5 Vertical Slice Architecture

- **Qué es:** en vez de capas horizontales, organiza por *funcionalidades* (slices verticales). Cada slice (crear libro, borrar librería) contiene su propio request, handler, DTO y acceso a datos.

- **Ventajas:** cada feature es autónoma, cambios locales, menos "saltar entre capas", alinea con CQRS/MediatR (Módulo 15).
- **Desventajas:** requiere disciplina para no duplicar lógica compartida; puede generar repetición de patrones entre slices.
- **Relación:** es la evolución natural para proyectos que crecen; no se usa aquí.

3.6 Repository Pattern

- **Qué es:** capa intermedia entre el servicio y el DbContext que encapsula la persistencia con una interfaz tipo colección (`GetById` , `Add` , `Remove`). Esconde EF Core.
- **Ventajas históricas:** abstraer el acceso a datos, permitir mockear fácilmente en unit tests, estandarizar consultas.
- **Desventajas actuales:** con EF Core moderno, el `DbContext` ya es un repositorio (Unit of Work) + mapeador + Change Tracker. Añadir otra capa es **puro boilerplate** que no aporta: más código, más abstracción, y limita las capacidades de EF (Include, ThenInclude, Project, tracking) si la interfaz es genérica pobre.
- **Por qué este proyecto NO usa Repository** (decisión explícita, documentada): el `DbContext` ya provee `DbSet<Library>` con `AddAsync` , `Remove` , `Where` , `ToListAsync` , `SaveChangesAsync` . Los servicios (`LibrariesService` , `BooksService`) **son** la capa de acceso a datos contra `LibraryContext` . Añadir `IRepository<T>` aquí añadiría indirección sin beneficio.
- **Tradeoffs de la decisión:**
 - A favor: menos código, uso directo de toda la potencia de EF Core (IQueryable, tracking, eager loading), menos mantenimiento.
 - En contra: los unit tests de servicios no pueden mockear el contexto fácilmente (requiere mocks de `DbSet` , doloroso); el acoplamiento a EF Core está presente en la capa de servicios (aunque dentro del mismo proyecto está bien).
- **Cuando Sí usar Repository:** necesidad real de soportar múltiples proveedores, o de desacoplar el dominio de la persistencia (Clean/Hexagonal), o equipos que lo exijan. La lección: **no uses patrones "porque sí"; usa patrones cuando resuelven un problema real.**

3.7 DTO (Data Transfer Object)

- **Qué es:** objeto plano (sin lógica) que transporta datos entre capas o por la red. Desacopla el contrato externo del modelo interno.
- **En el proyecto:** `BookForm` (contrato de entrada del POST), `User` (body del login), `TokenResponse` (salida del login). El controller mapea `BookForm` → entidad `Book` a mano (`new Book { Name = ..., Category = ..., LibraryId = ... }`).
- **Ventajas:** no expones entidades completas; controlas exactamente qué campos entran/salen (y con qué nombre JSON).
- **Desventajas:** mapeo manual repetitivo (se usa AutoMapper cuando crece).
- **Relación:** con el Módulo 12 (DTOs y validación).

3.8 Inversión de Dependencias (DIP, la "D" de SOLID)

- **Qué es:** los módulos de alto nivel no deben depender de módulos de bajo nivel, sino de **abstracciones**.
- **En el proyecto:** los controllers dependen de `ILibrariesService` / `IBooksService` (abstracciones), no de `LibrariesService` concreto. El DI (Módulo 3) inyecta la implementación. Esto permite sustituir la implementación en los tests (Módulo 9) — aunque en ese caso se sustituye el `DbContext` .

3.9 Capa de aplicación vs capa de dominio (concepto de fondo)

- **En el proyecto:** no hay dominio rico; las entidades (`Library` , `Book`) son POCOs sin reglas de negocio (anémicas). Para este CRUD está bien. En dominios complejos, la lógica de negocio debe vivir en el dominio, no en los servicios.

3.10 Acoplamiento y cohesión

- **Acoplamiento:** qué tan conectadas están las piezas. Bajo es deseable.
- **Cohesión:** qué tan relacionadas están las responsabilidades dentro de una pieza. Alta es deseable.
- **En el proyecto:** `BookService` solo toca libros; `BooksController` solo orquesta libros/librerías. Alta cohesión, bajo acoplamiento vía interfaces.

4. Conocimientos previos necesarios

- Módulo 2 (C#/interfaces/classes).
- Módulo 3 (DI, controllers, servicios) — imprescindible para ver la arquitectura "en vivo".

5. Cómo aparece dentro del proyecto

Archivo	Concepto
<code>Controllers/</code>	Capa de presentación
<code>Services/LibraryService.cs</code>	Capa de aplicación + acceso a datos (por decisión de no usar Repository)
<code>Services/BookService.cs</code>	Ídem, con <code>IBooksService</code> / <code>BooksService</code>
<code>Services/AuthenticationService.cs</code>	Servicio con lógica de autenticación (credenciales hardcodeadas)
<code>Data/LibraryContext.cs</code>	Capa de persistencia (DbSet + entidades en el mismo archivo)
<code>DTO/BookForm.cs</code>	DTO de entrada
<code>Controllers/AuthController.cs</code>	<code>record TokenResponse</code> como DTO de salida
<code>docs/DOCUMENTACION-TECNICA.md</code>	"Patrón de diseño: Layered clásica, sin capa de repositorios"
<code>HackerRank1.csproj</code>	Un solo proyecto → no hay separación física de capas (Domain/Application/Infrastructure)

6. Nivel de importancia

Importante — para entender el proyecto basta; para diseñar uno similar es fundamental.

7. Tiempo recomendado de estudio

12 horas.

8. Recursos recomendados

- **Documentación:** "Repository pattern in EF Core — is it obsolete?" (artículo y video de Nick Chapsas); "Clean Architecture" (Robert C. Martin); "Vertical Slice Architecture" (Jimmy Bogard).
- **Microsoft Learn:** "Design a layered architecture" (module `design-multi-tier-data-application`).
- **Libros:** *Clean Architecture* (Robert C. Martin); *Architecture Patterns with Python* (Percival & Gregory, conceptos aplicables); *Domain-Driven Design* (Eric Evans, referencia).
- **Videos:** "Repository pattern: Should you use it?" (Nick Chapsas); "The Clean Architecture" (Robert C. Martin talks); "Vertical Slice Architecture" (Jimmy Bogard, NDC).
- **Repositorios:** `ardalis/CleanArchitecture`; `jbogard/ContosoUniversity` (Vertical Slice); `dotnet-architecture/eShopOnWeb` (Clean-ish).

9. Ejercicios sugeridos

1. (*Fácil*) Dibuja el flujo de una petición `GET /api/libraries` atravesando las capas; marca dónde vive cada responsabilidad.
2. (*Fácil*) Encuentra en el proyecto 3 dependencias hacia abstracciones (interfaces) y explica el beneficio.
3. (*Medio*) Refactoriza el proyecto agregando un Repository Pattern; luego redacta una conclusión de 1 página sobre si mejoró o empeoró el código y por qué.
4. (*Medio*) Propón cómo sería la misma feature (libros por librería) en Vertical Slice: lista los archivos que tendrías.
5. (*Difícil*) Evalúa: ¿qué pasaría si mañana la BD cambia de PostgreSQL a SQL Server? ¿Cuántos archivos cambiarían con la arquitectura actual? ¿Y con Repository? Argumenta.
6. (*Difícil*) Diseña una arquitectura Clean para este proyecto (proyectos Domain, Application, Infrastructure, API) y describe el contenido de cada proyecto.
7. (*Difícil*) Debate con un colega: "¿Cuándo Repository es la elección correcta en 2026?" Escribe 3 argumentos a favor y 3 en contra.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Aplicar Repository sin justificación	Capas de indirección que solo reenvían llamadas	Evaluar si el <code>DbContext</code> ya cubre la necesidad
Controllers gordos con lógica de BD	<code>_context</code> dentro del controller	Mover la lógica al service
Entidades con responsabilidades de red	<code>[JsonProperty]</code> en entidades	Usar DTOs para el contrato externo
Acoplar el contrato externo a la BD	Cambiar la BD rompe el JSON de la API	Separar entidad persistida y DTO de API
Capa "Domain" vacía de reglas	Todo es CRUD anémico	Decidir conscientemente: dominio rico o anémico según complejidad
Creer que Clean Architecture es gratis	Cientos de interfaces para 3 endpoints	Elegir arquitectura según el tamaño del problema

MÓDULO 5 — Entity Framework Core

1. Nombre del módulo

Entity Framework Core: ORM, DbContext, entidades, LINQ, Change Tracker, migraciones y Npgsql

2. Objetivo de aprendizaje

El estudiante comprenderá cómo EF Core convierte clases C# en tablas y consultas SQL, cómo funciona el seguimiento de cambios, por qué las consultas son diferidas (IQueryable), cómo aplicar y revertir migraciones, y cómo conectar el contexto a PostgreSQL vía Npgsql. Es el módulo más importante junto con HTTP.

3. Conceptos fundamentales

3.1 ORM (Object-Relational Mapper)

- **Qué es:** herramienta que mapea objetos (C#) a tablas relacionales (SQL) y viceversa, evitando escribir SQL a mano.
- **Para qué sirve:** productividad (escribes C# y LINQ, no SQL), seguridad (evita SQL Injection), mantenibilidad.
- **Desventajas:** abstracción que puede generar SQL ineficiente (N+1, consultas enormes); requiere entender el SQL que *genera* para optimizar.
- **Relación:** EF Core es el ORM; Npgsql es el *provider* (el puente a PostgreSQL).

3.2 DbContext

- **Qué es:** clase central de EF Core. Representa una **unidad de trabajo**: una sesión hacia la BD, un conjunto de entidades rastreadas y la coordinación de cambios.
- **Para qué sirve:** expone `DbSet`s, ejecuta queries, aplica cambios y gestiona el ciclo de vida de las entidades.
- **En el proyecto** (`Data/LibraryContext.cs`):

```
csharp public class LibraryContext : DbContext { public LibraryContext(DbContextOptions<LibraryContext> options) : base(options) { } public DbSet<Library> Libraries { get; set; } public DbSet<Book> Books { get; set; } }
```
- **Lifecycle y lifetime:** registrado como **scoped** vía `AddDbContextPool` (pool de 20). Cada request tiene su propio `LibraryContext` (o uno del pool reutilizado); al terminar el request se descarta. **Nunca** lo guardes en un singleton.
- **DbContextPool:** `AddDbContextPool` reutiliza instancias del contexto entre requests (rendimiento) pero exige que el contexto no guarde estado mutable entre peticiones.
- **Unit of Work:** `SaveChangesAsync()` aplica *todos* los cambios rastreados en una sola transacción. El DbContext ES el Unit of Work y el Repository a la vez.

3.7 Entidades

- **Qué son:** clases que mapean 1:1 a tablas. Propiedades → columnas.
- **En el proyecto:** `Library` → tabla `Libraries` (Id, Name, Location); `Book` → tabla `Books` (Id, Name, Category, LibraryId).
- **Convenciones:** `[Key]` en `Id` (primaria, autogenerada). La FK `Book.LibraryId` se infiere por convención + navegación `public virtual Library Library`.
- **virtual:** habilita proxies de lazy loading (carga diferida). Aquí `Book.Library` es `virtual` pero no hay lazy loading configurado (ver 3.10).

3.8 Convenciones vs configuración explícita

- **Qué es:** EF aplica convenciones (nombres de tablas = plural de DbSet, PK = `Id`, FKs por nombre `XId`). Puedes anularlas con Data Annotations (`[Key]`, `[Required]`) o Fluent API (`OnModelCreating`).
- **En el proyecto:** se usa la convención + `[Key]`. La relación 1:N y el `ON DELETE CASCADE` se definieron por convención/convención de la FK y quedaron en la migración.

3.9 LINQ sobre entidades: IQueryable y ejecución diferida

- **IQueryable**: expresión *no ejecutada* (plan). Se traduce a SQL en el momento de la ejecución (ToListAsync, FirstOrDefault, etc.). Esto permite componer consultas (`Where(...)` luego otro `Where(...)`).
- **En el proyecto**: `csharp var query = _libraryContext.Books.AsQueryable().Where(b => b.LibraryId == libraryId); if (ids != null && ids.Any()) query = query.Where(b => ids.Contains(b.Id)); return await query.ToListAsync();` El `if` compone la consulta **antes** de ejecutarla — imposible con listas ya materializadas.
- **Diferencia crítica**: **IQueryable** (SQL en el servidor) vs **IEnumerable** (memoria). Aplicar `.ToListAsync()` a mitad de camino rompe la optimización.
- **Cómo se traduce**: `ids.Contains(x.Id)` → `WHERE "Id" IN (1,2,3)` . `x => x.Id == id` → `WHERE "Id" = @p0` .

3.10 Cargas de navegación: Lazy vs Eager vs Explicit

- **Lazy Loading**: las navegaciones se cargan al accederlas (requiere proxies + configuración). Ventaja: simple. Desventaja: N+1 queries.
- **Eager Loading**: `.Include(x => x.Books)` . Carga todo en una query con JOIN. Ventaja: eficiente. Desventaja: puede traer de más.
- **Explicit Loading**: `context.Entry(book).Reference(x => x.Library).LoadAsync()` .
- **En el proyecto**: `Book.Library` es `virtual` (soporta lazy) pero no hay `UseLazyLoadingProxies` , así que la navegación solo se rellena si la consulta hace `Include` . Los services no usan `Include` → el proyecto prácticamente no usa navegaciones.
- **N+1 Problem**: consultar N libros y disparar 1 query extra por cada uno (acceso a `book.Library` sin Include) = N+1 queries. Síntoma clásico de Lazy Loading.

3.11 Change Tracker (seguimiento de cambios)

- **Qué es**: el contexto vigila cada entidad y le asigna un `EntityState` : `Added` , `Modified` , `Deleted` , `Detached` , `Unchanged` .
- **Cómo funciona**: al hacer `SaveChangesAsync()` , EF compara estados y emite los comandos SQL necesarios (INSERT/UPDATE/DELETE) **en una transacción**.
- **En el proyecto**:
 - `AddAsync` → `Added` → INSERT.
 - `Remove` → `Deleted` → DELETE.
 - `Update(projectForChanges)` + modificar propiedades → `Modified` → UPDATE.
 - `Get` con `AsQueryable().Where(...).ToListAsync()` → entidades **tracked** por defecto.
- **Anti-patrón típico**: el service `Update` hace `SingleAsync` (trae tracked), modifica propiedades y luego llama `_libraryContext.Libraries.Update(...)` — el `Update` es redundante porque el cambio ya se detecta por el tracker, pero no rompe. Es señal de no confiar en el tracker.
- **Detached**: entidades desconectadas del contexto (creadas en el controller, como `new Book { ... }`). Para que EF las persista, el service las `Add` s.

3.12 Tracking vs NoTracking

- `AsNoTracking()` : lecturas sin vigilancia → más rápidas, ideales para solo-lectura. `AsTracking()` es lo contrario.
- **En el proyecto**: `Get` de services rastrea por defecto (sin `AsNoTracking`) — para listas de solo lectura es subóptimo pero correcto.
- **En los tests**: se detach manualmente el estado (`entity.State = EntityState.Detached`) para limpiar el tracker entre fixtures.

3.13 Migraciones

- **Qué son**: archivos C# versionados que describen cómo evolucionar el esquema de la BD (`Up()` aplica, `Down()` revierte). Son el "control de versiones" del esquema.
- **Conceptos**: `dotnet ef migrations add Nombre` crea una migración; `dotnet ef database update` la aplica; `dotnet ef migrations list` muestra estado; `dotnet ef migrations remove` la borra (solo si no aplicada).
- **Cómo aparecen**:
 - `Migrations/20260528004745_InitialCreate.cs` — crea `Libraries` , `Books` , la FK `FK_Books_Libraries_LibraryId` con `onDelete: ReferentialAction.Cascade` y el índice `IX_Books_LibraryId` .
 - `Migrations/LibraryContextModelSnapshot.cs` — snapshot del modelo; EF lo compara con las entidades para generar la siguiente migración.
 - `Migrations/*.Designer.cs` — metadatos.

- **En el proyecto:** la migración se aplica **automáticamente al arrancar** vía `db.Database.Migrate()` en `Startup.Configure` (guard: solo si no es SQLite y hay pendientes). Alternativa manual: `dotnet ef database update`.
- **Herramienta:** `dotnet-ef` es una **global tool** instalada con `dotnet tool install --global dotnet-ef --version 8.0.2`. Sin ella, los comandos de migración fallan ("No se encontró la herramienta").
- **Importante:** `Migrate()` vs `EnsureCreated()`:
- `EnsureCreated()`: crea el esquema completo si la BD no existe; **no** usa migraciones ni las marca como aplicadas. Rápido para prototipos/tests.
- `Migrate()`: aplica migraciones pendientes sobre el historial. Es lo correcto para producción.
- En los tests se usa `EnsureCreated()` (SQLite in-memory); en la app real, `Migrate()` (Postgres).

3.14 Provider y Npgsql

- **Qué es:** los providers adaptan EF Core a cada BD. Npgsql es el provider oficial para PostgreSQL.
- **En el proyecto:** `options.UseNpgsql(connectionString, npgsqlOptions => { ... EnableRetryOnFailure(...) })`. `Npgsql.EntityFrameworkCore.PostgreSQL 8.0.2`.
- **Opciones del provider:** `EnableRetryOnFailure(maxRetryCount: 1, maxRetryDelay: TimeSpan.FromSeconds(5))` = estrategia de reintentos ante fallos transitorios (la conexión a la nube se cae a veces).
- **Importante:** el `OnDelete CASCADE`, el `IdentityByDefaultColumn` (SERIAL/IDENTITY de Postgres) y los tipos `text` / `integer` son decisiones del provider Npgsql visibles en la migración.

3.15 PostgreSQL (visión EF)

- **Qué es:** BD relacional open-source robusta. Tipos, índices, constraints, transacciones.
- **En el proyecto:** esquema con tablas `Libraries` y `Books`, FK con cascade, índice en `Books.LibraryId`. La NOT NULL de `Category` (nullable: false) es el motivo del default `string.Empty` en el controller.

3.16 SQL Injection

- **Qué es:** ataque inyectando SQL malicioso vía inputs.
- **Cómo EF lo previene:** al usar LINQ/parametrización (`@p0`), EF **parametriza** siempre. Los valores de `Where(x => x.Name == name)` nunca se concatenan al SQL.
- **Regla:** nunca concatenes strings en SQL; usa LINQ/EF o parámetros explícitos.

3.17 Transacciones

- **Qué es:** conjunto de operaciones que deben aplicarse todas o ninguna (ACID). `SaveChangesAsync` ya envuelve los cambios en una transacción.
- **Aplicación avanzada:** `Database.BeginTransactionAsync()` para orquestar varios SaveChanges. No se usa aquí (cada service guarda su propia unidad).

4. Conocimientos previos necesarios

- Módulo 2 (C#, LINQ, genéricos, async).
- Módulo 3 (DI: cómo se registra el contexto).
- Conceptos mínimos de SQL/BD relacionales (Módulo 6) ayudan, aunque EF abstrae el SQL.

5. Cómo aparece dentro del proyecto

Archivo	Concepto
<code>Data/LibraryContext.cs</code>	<code>DbContext</code> , <code>DbSet</code> , entidades <code>Book</code> / <code>Library</code> , <code>[Key]</code> , navegación <code>virtual</code>
<code>Services/LibraryService.cs</code>	<code>AsQueryable</code> , <code>Where</code> , <code>ToListAsync</code> , <code>AddAsync</code> , <code>SingleAsync</code> , <code>Remove</code> , <code>Update</code> , <code>SaveChangesAsync</code>
<code>Services/BookService.cs</code>	Ídem para <code>Books</code>

Archivo	Concepto
<code>Startup.cs</code>	<code>AddDbContextPool</code> , <code>UseNpgsql</code> , <code>EnableRetryOnFailure</code> , <code>GetConnectionString</code> , <code>GetPendingMigrations</code> , <code>Migrate</code>
<code>Migrations/20260528004745_InitialCreate.cs</code>	<code>CreateTable</code> , PK, FK CASCADE, índice, <code>IdentityByDefaultColumn</code>
<code>Migrations/LibraryContextModelSnapshot.cs</code>	snapshot del modelo EF
<code>LibraryService.Integration.Test/IntegrationTest.cs</code>	<code>DbContextOptionsBuilder<LibraryContext>().UseSqlite(...)</code> , <code>EnsureCreated</code> , <code>ChangeTracker</code> , <code>EntityState.Detached</code> , <code>OpenConnection</code>
<code>docs/DOCUMENTACION-TECNICA.md</code>	bug de versiones EF 6.0 vs 8.0; decisión <code>Migrate()</code> vs <code>EnsureCreated()</code>

6. Nivel de importancia

Fundamental — es la mitad del proyecto.

7. Tiempo recomendado de estudio

30 horas (teoría 12 h + práctica 18 h).

8. Recursos recomendados

- **Documentación oficial:** learn.microsoft.com → "Entity Framework Core" (todos los módulos: DbContext, Change Tracking, Querying, Migrations).
- **Microsoft Learn:** "Add and use EF Core" y "Build web apps with EF Core" (paths oficiales).
- **Libros:** *Entity Framework Core in Action* (Jon Smith, 2ª edición); *Programming Entity Framework Core* (Julia Lerman).
- **Videos:** "Entity Framework Core: getting started" (Microsoft), series de Nick Chapsas sobre EF Core, "IQueryable vs IEnumerable" (Coding with David).
- **Cursos:** Pluralsight "Entity Framework Core 8 Fundamentals".
- **Repositorios:** `dotnet/efcore` (samples en `samples/`); `Zack/EFCoreGuide`.

9. Ejercicios sugeridos

1. (Fácil) Desde la raíz: `dotnet ef migrations script --project HackerRank1` y lee el SQL generado por la migración `InitialCreate`.
2. (Fácil) `dotnet ef migrations list --project HackerRank1` y `dotnet ef migrations remove --project HackerRank1` (solo si puedes regenerarla) — practica el ciclo add/remove.
3. (Medio) Agrega una columna `PublishedYear` a `Book`, crea una migración, aplícala y luego reviértela con `Down()`.
4. (Medio) Escribe una consulta LINQ que use `Include` para traer `Library` con sus `Books` y observa la diferencia de SQL con logging (`EnableSensitiveDataLogging`).
5. (Medio) Instrumenta el proyecto para loguear el SQL generado (opción `LogTo` del contexto) y observa qué SQL produce cada service.
6. (Difícil) Implementa `AsNoTracking` en `Get` y mide/observa el comportamiento del Change Tracker (usa el código de los tests que hace `Detach`).
7. (Difícil) Reproduce el bug real del proyecto: apuntar el proyecto de tests a EF Core 6.0 con la API en 8.0 → `MissingMethodException` `ConventionSet.get_ModelFinalizingConventions()`; corrígelo alineando versiones.
8. (Difícil) Escribe una migración manual compleja (índice compuesto, constraint unique) y verifica con `dotnet ef migrations script`.
9. (Difícil) Simula el N+1: carga 10 libros y accede a `book.Library` sin `Include`; cuenta las queries en el log.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Materializar demasiado pronto	<code>.ToList()</code> antes de <code>Where</code> → filtrar en memoria	Mantener <code>IQueryable</code> hasta el final
<code>Category</code> null contra columna NOT NULL	<code>DbUpdateException</code> al guardar	Default <code>string.Empty</code> (como el proyecto) o columna nullable
Guardar <code>DbContext</code> como singleton	Errores de concurrencia/tracking entre requests	Registrar Scoped (o pool)
<code>dotnet ef</code> sin la global tool	"No se pudo ejecutar la herramienta"	<code>dotnet tool install --global dotnet-ef --version 8.0.2</code>
Mezclar <code>EnsureCreated</code> y <code>Migrate</code>	Tests que fallan o esquema doble	Tests → <code>EnsureCreated</code> ; app real → <code>Migrate</code> (guard del proyecto)
Llamar <code>Update</code> sobre entidad ya trackeada	UPDATE redundante / doble tracking	Confiar en el Change Tracker; <code>Update</code> solo para detached
Cambiar la BD rompe los tests	Tests con SQLite vs Npgsql	No usar features de Npgsql en código de dominio; inyectar contexto
Ignorar el provider	Migraciones con sintaxis de otro motor	Usar el provider correcto (Npgsql) al generar migraciones

MÓDULO 6 — PostgreSQL y Supabase

1. Nombre del módulo

PostgreSQL, SQL, Supabase (BD en la nube), connection strings, pooling y User Secrets

2. Objetivo de aprendizaje

El estudiante comprenderá el modelo relacional detrás del proyecto, qué es PostgreSQL, cómo se conecta una app .NET a una BD en la nube (Supabase) mediante connection strings, qué significan las opciones de la cadena (SSL, Pooling, puerto, host pooler) y por qué el secreto real vive en User Secrets y no en el repositorio.

3. Conceptos fundamentales

3.1 Bases de datos relacionales y SQL

- **Qué es:** modelo que organiza datos en **tablas** con filas/columnas y relaciones (1:N, N:M) mediante **foreign keys**.
- **Conceptos SQL que debes dominar:** `CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`, `WHERE`, `JOIN`, `PRIMARY KEY`, `FOREIGN KEY`, `ON DELETE CASCADE`, `INDEX`, `transaction` (`BEGIN/COMMIT/ROLLBACK`).
- **En el proyecto:** las tablas `Libraries` y `Books` con la FK `Books.LibraryId → Libraries.Id ON DELETE CASCADE`.

3.2 PostgreSQL

- **Qué es:** sistema de gestión de BD relacional open-source, con ACID, tipos ricos, JSONB, extensiones (PostGIS, pgvector).
- **Por qué se usa:** robusto, gratuito, cloud-ready, estándar de la industria moderna.
- **Detalles en el proyecto:** columnas `integer` (con `IdentityByDefaultColumn` = autoincremento estilo `GENERATED BY DEFAULT AS IDENTITY`), `text`, FK con cascade, índice en `LibraryId`.

3.3 FK y ON DELETE CASCADE

- **Qué es:** la FK garantiza que `Book.LibraryId` siempre apunte a una `Library` existente. `ON DELETE CASCADE` significa que al borrar una librería, **la BD borra sus libros automáticamente**.
- **En el proyecto:** definido en la migración `InitialCreate` (`onDelete: ReferentialAction.Cascade`). Por eso `DELETE /api/libraries/1` no necesita borrar libros a mano.

3.4 Supabase

- **Qué es:** plataforma backend-como-servicio (BaaS) open-source que incluye PostgreSQL gestionado, Auth, Storage, Realtime y APIs.
- **En el proyecto:** se usa **solo como hosting de PostgreSQL** (proyecto "vacío" creado en el panel para obtener credenciales).
- **Credenciales:** `Host=aws-1-us-west-2.pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.gyktxhyeyisdbafvvp`.
- **El "host pooler":** Supabase ofrece dos endpoints: directo y *pooler* (para conexiones con pooling, típico de apps serverless y drivers con pool). Aquí se usa el pooler.
- **Seguridad:** acceso solo vía SSL (`SSL Mode=Require`); la contraseña es "secret" de nivel BD.

3.5 Connection strings

- **Qué es:** cadena con la configuración de conexión: `Host;Port;Database;Username;Password;SSL Mode;Trust Server Certificate;Pooling`.
- **En el proyecto** (`appsettings.json`): `Host=aws-1-us-west-2.pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.gyktxhyeyisdbafvvp;Password=[SUPABASE-PASSWORD]; SSL Mode=Require;Trust Server Certificate=true; Pooling=false`
- **Opciones clave:**
- `SSL Mode=Require`: obliga a cifrar la conexión (TLS).

- `Trust Server Certificate=true` : no valida la cadena de certificado (aceptable para lab, débil para prod).
- `Pooling=false` : no usar pooling del lado Npgsql. **Bug histórico del proyecto**: el separador era una coma (,) en vez de punto y coma (;), lo que rompía el parsing de Npgsql. Los delimitadores de opciones son ; .
- **Npgsql connection string docs**: separador ; , pares `clave=valor` , valores con ' para espacios.

3.6 Pooling de conexiones

- **Qué es**: reutilizar conexiones TCP a la BD en lugar de abrir/cerrar una por operación (abrir es caro).
- **Doble pooling en este stack**: 1) pool del driver Npgsql (controlado por `Pooling=true/false` y `Maximum Pool Size`) y 2) pool de DbContext de EF (`AddDbContextPool<...>(poolSize: 20)`).
- **En el proyecto**: el driver trae `Pooling=false` (cada operación abre conexión nueva — simple pero más lento), mientras EF usa un pool de 20 contextos. Para un lab está bien; en producción querías `Pooling=true` con `Maximum Pool Size` adecuado.

3.7 Configuración externa y el principio 12-factor

- **Qué es**: la config sensible no vive en el código ni en el repo; viene del entorno (variables, secretos, servicios de config).
- **En el proyecto**: la contraseña real NO está en el repo (placeholder `[SUPABASE-PASSWORD]`); vive en User Secrets (local) o variables de entorno. Los beneficios/costos están documentados en el repo (sección 6.4).

3.8 User Secrets (desarrollo)

- **Qué es**: mecanismo de .NET para guardar secretos de desarrollo fuera del repo, en `%APPDATA%\Microsoft\UserSecrets\{UserSecretsId}\secrets.json` .
- **Las 5 piezas del mecanismo** (documentado en `docs/REPORTE-USER-SECRETS.md`): 1. `UserSecretsId` en el `.csproj` — el "nombre del cajón" (`9cbbfeab-44b3-447c-8aa1-4d5adea68ef6`). Único dato que sube a git. 2. El cajón físico en `%APPDATA%` (fuera del repo) → git nunca lo ve. 3. `ASPNETCORE_ENVIRONMENT=Development` en `launchSettings.json` — llave que activa la carga. 4. `Host.CreateDefaultBuilder(args)` en `Program.cs` — agrega el proveedor User Secrets automáticamente cuando es Development. 5. `Configuration.GetConnectionString("DefaultConnection")` en `Startup.cs` — consume la cadena fusionada.
- **Orden de precedencia (el último gana)**: `appsettings.json` (placeholder) → `appsettings.Development.json` → **User Secrets** (password real, pisa el placeholder) → variables de entorno → args.
- **Comandos**: `dotnet user-secrets init` `dotnet user-secrets set "ConnectionStrings:DefaultConnection" "...password real..."` . La clave usa : como separador de ruta (`ConnectionStrings:DefaultConnection`), exactamente lo que busca `GetConnectionString` .
- **Limitación crítica**: User Secrets **solo se cargan en Development** → en producción la BD no conectaría si la config depende de User Secrets (riesgo documentado en el repo).

3.9 Variables de entorno

- **Qué es**: mecanismo portable de configuración (`ConnectionStrings__DefaultConnection` , `ASPNETCORE_ENVIRONMENT`). Los __ dobles mapean a : en la jerarquía IConfiguration.
- **Para qué sirven**: la vía correcta en producción/CI (Módulos 13 y 14).

3.10 Verificación de conexión

- **Comandos útiles**: `dotnet ef migrations list --project HackerRank1` (conecta y lee historial — prueba viva de conexión + auth), `SELECT 1` con `psql` /pgAdmin.
- **En el proyecto**: se validó la conexión así antes de aplicar `InitialCreate` .

4. Conocimientos previos necesarios

- Módulo 2 (CLI de .NET, csproj).
- Módulo 3 (Configuration, IConfiguration, entornos).
- Módulo 5 (connection string → UseNpgsql, migraciones).

5. Cómo aparece dentro del proyecto

Archivo	Concepto
appsettings.json	connection string con placeholder [SUPABASE-PASSWORD]
HackerRank1.csproj	<UserSecretsId>9cbbfeab-...</UserSecretsId>
Properties/launchSettings.json	ASPNETCORE_ENVIRONMENT=Development (activa User Secrets)
Program.cs	Host.CreateDefaultBuilder (carga User Secrets)
Startup.cs	Configuration.GetConnectionString("DefaultConnection") → UseNpgsql
Migrations/20260528004745_InitialCreate.cs	esquema real aplicado en Supabase (identity, FK cascade)
docs/REPORTE-USER-SECRETS.md	la cadena completa de 5 piezas
docs/DOCUMENTACION-TECNICA.md	bug del separador , vs ; ; costos de no tener credenciales en el repo

6. Nivel de importancia

Importante — necesitas dominarlo para arrancar el proyecto en cualquier máquina.

7. Tiempo recomendado de estudio

12 horas.

8. Recursos recomendados

- **Documentación oficial:** PostgreSQL docs ("Tutorial"), Npgsql connection string docs, Microsoft Learn "Configuration in ASP.NET Core" (incluye User Secrets).
- **Supabase:** docs.supabase.com → "Connect to PostgreSQL".
- **Microsoft Learn:** "Store app secrets in development in ASP.NET Core".
- **Libros:** *PostgreSQL: Up and Running* (Regina Obe); *SQL in 10 Minutes* (Ben Forta, para SQL).
- **Videos:** "PostgreSQL in 100 seconds" (Fireship); "Connection pooling explained" (Hussein Nasser).
- **Cursos:** "SQL for beginners" (freeCodeCamp/Khan Academy); Supabase Fundamentals (app.supabase.com).
- **Herramientas:** pgAdmin, DBeaver, `psql`, Supabase SQL Editor.

9. Ejercicios sugeridos

1. (Fácil) Conecta con DBeaver/psql a tu BD Supabase y explora las tablas `Libraries` y `Books`.
2. (Fácil) Ejecuta en el SQL Editor: `INSERT INTO "Libraries" ("Name","Location") VALUES ('A','B') RETURNING "Id";` y un `SELECT` con JOIN.
3. (Medio) Borra una librería que tenga libros y verifica en BD que los libros se borraron (CASCADE).
4. (Medio) Corrige el bug histórico: escribe una connection string con `,` en vez de `;` y verifica el error de Npgsql; luego corrígela.
5. (Medio) Crea tu propio proyecto Supabase, configura User Secrets y arranca la API con tu BD. Documenta los pasos.
6. (Difícil) Explica por qué `Host.CreateDefaultBuilder` carga User Secrets solo en Development y demuéstalo cambiando el entorno a Production (observa el fallo de conexión).
7. (Difícil) Prueba `Pooling=true;Maximum Pool Size=5` bajo 50 requests concurrentes vs `Pooling=false`; mide tiempo y errores.
8. (Difícil) Redacta un mini-guion de "rotación de secretos": cómo cambiar la password de Supabase sin tocar el código y cuántos lugares deben actualizarse.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Separador incorrecto <code>,</code> en la connection string	Npgsql lanza "keyword not supported" o cae el arranque	Usar <code>;</code> como separador de opciones
Commitear la password real	<code>git grep "Password="</code> encuentra el secreto	User Secrets/variables de entorno; sanear historial si ya se expuso
Creer que User Secrets funciona en producción	La BD no conecta en prod	Usar variables de entorno / Secret Manager de la nube en prod
<code>dotnet run</code> en la raíz de la solución	"No se encontró un proyecto para ejecutar"	<code>dotnet run --project HackerRank1</code>
Ignorar SSL	Npgsql rechaza la conexión a Supabase sin SSL	<code>SSL Mode=Require</code>
Desconocer el <code>__</code> en variables de entorno	Variables que no se leen	<code>ConnectionStrings__DefaultConnection</code> mapea a <code>ConnectionStrings:DefaultConnection</code>

MÓDULO 7 — Autenticación y JWT

1. Nombre del módulo

Autenticación HTTP, JWT, Claims, Roles, Bearer, Middleware de auth y criptografía

2. Objetivo de aprendizaje

El estudiante entenderá de punta a punta cómo funciona la autenticación stateless de esta API: cómo se emite el token (`/login`), qué contiene, cómo se firma (HMAC-SHA256), cómo se valida en cada request (middleware) y cómo se autoriza el acceso a endpoints protegidos (`[Authorize]`).

3. Conceptos fundamentales

3.1 Autenticación vs Autorización

- **Autenticación:** *¿quién eres?* (probar identidad). En el proyecto: presentar `admin / 1234` en `/login` → recibir un JWT.
- **Autorización:** *¿qué puedes hacer?* (permisos). En el proyecto: `[Authorize]` decide si el request con un token válido entra.
- **En el pipeline:** `UseAuthentication()` (identifica) → `UseAuthorization()` (permite o niega).

3.2 Autenticación stateless (sin estado)

- **Qué es:** el servidor no guarda sesiones; la identidad viaja con cada request en el token. El servidor solo verifica la firma.
- **Ventajas:** escala horizontal, sin tablas de sesión, funciona para cualquier cliente (web/móvil/script).
- **Desventajas:** la revocación es difícil (un token robado vale hasta que expire); el token debe ser corto de vida y pequeño.

3.3 JWT (JSON Web Token)

- **Qué es:** estándar (RFC 7519) de token autocontenido. Tres partes separadas por puntos: `header.payload.signature`, codificadas en Base64Url.
- **Header:** `{"alg":"HS256","typ":"JWT"}`.
- **Payload:** los *claims* (`sub`, `email`, `role`, `exp`, `iss`, `aud`).
- **Signature:** firma criptográfica del header+payload.
- **En el proyecto** (`Helpers/TokenGenerator.cs`):
 - Algoritmo **HS256** (HMAC-SHA256) → firma simétrica.
 - Claims: `NameIdentifier` (id), `Email`, `Role`.
 - `issuer: MyApp`, `audience: localhost:80`, `expires: DateTime.UtcNow.AddHours(1)`.
 - `JwtSecurityTokenHandler().WriteToken(token)` produce la cadena.
- **El token NO es cifrado:** header y payload son *solo* Base64Url; cualquier persona los puede leer. La **firma** garantiza que no fueron alterados y que el emisor es quien dice ser. ¡Nunca pongas secretos en un JWT!

3.4 Claims

- **Qué es:** declaraciones sobre el sujeto: `Claim(ClaimTypes.NameIdentifier, ...)`, `Claim(ClaimTypes.Email, ...)`, `Claim(ClaimTypes.Role, ...)`.
- **Para qué sirve:** transportar identidad y permisos dentro del token; el middleware las expone en `HttpContext.User` para usarlas en autorización.
- **En el proyecto:** el `User` autenticado (`admin`) obtiene `Id=1`, `Email=admin`, `Role=admin` → se convierten en claims del token.

3.5 Criptografía de la firma: HMAC-SHA256 y claves simétricas

- **Qué es:** HMAC (Hash-based Message Authentication Code) combina una **clave secreta simétrica** con una función hash (SHA-256) para producir una firma que solo el poseedor de la clave puede crear/verificar.
- **Simétrico:** la **misma clave** firma y verifica. Por eso el `SecretKey` debe conocerse solo en el servidor.

- **En el proyecto:**
- Generación: `new SigningCredentials(key, SecurityAlgorithms.HmacSha256)` donde `key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings.SecretKey))`.
- Validación: `TokenValidationParameters` con `IssuerSigningKey = new SymmetricSecurityKey(...)`.
- **Detalle de seguridad:** el `SecretKey` actual está en `appsettings.json` (`a_very_long_super_secret_key_here`) — aceptable para el lab, **nunca** para producción (Módulo 13).
- **Contraste:** algoritmos *asimétricos* (RS256) usan clave privada para firmar y pública para verificar — preferibles en producción con varios emisores.

3.6 Issuer, Audience y Lifetime

- **Issuer (iss):** quién emitió el token (`MyApp`).
- **Audience (aud):** para quién está destinado (`localhost:80`).
- **Lifetime (exp , nbf):** expiración (`AddHours(1)`) y no-válido-antes-de.
- **En el proyecto** — validación estricta en `Startup.cs`: `csharp ValidateIssuerSigningKey = true, // verifica la firma con la clave ValidateIssuer = true, ValidIssuer = jwtSettings.Issuer, ValidateAudience = true, ValidAudience = jwtSettings.Audience, ValidateLifetime = true, ClockSkew = TimeSpan.Zero`
- **ClockSkew:** margen de reloj entre servidores. `TimeSpan.Zero` = sin tolerancia (estricto; en producción un skew de 30s-2min es típico).

3.7 Bearer tokens y el header Authorization

- **Qué es:** esquema de autenticación HTTP: `Authorization: Bearer <token>`.
- **En el proyecto:** `JwtBearerDefaults.AuthenticationScheme` configura el handler para leer ese header, validar el token y poblar `HttpContext.User`.
- **Firma incorrecta, token vencido, issuer/audience distintos** → el middleware responde `401`.

3.8 Middleware de autenticación (cómo se valida cada request)

- **Qué hace:** en cada request, el middleware JWT: 1) extrae el header, 2) valida firma/fechas/issuer/audience contra `TokenValidationParameters`, 3) construye un `ClaimsPrincipal`, 4) lo asigna a `HttpContext.User`.
- **En el proyecto:** `services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(option => { option.TokenValidationParameters = ... })` y `app.UseAuthentication()` en el pipeline.

3.9 Autorización con [Authorize] y Roles

- **Qué es:** `[Authorize]` sobre un endpoint exige un usuario autenticado; `[Authorize(Roles = "admin")]` exige el claim de rol.
- **En el proyecto:** el endpoint `GET /api/libraries/{id}/books` tenía `[Authorize]` (probado con `401` sin token). `[AllowAnonymous]` en `/login` (el login no puede exigir token). En el estado final, los books quedaron públicos porque el lab no los exige autenticados (decisión documentada).
- **Diferencia:** `[Authorize]` se apoya en el resultado del middleware de autenticación; sin `UseAuthentication()` no funciona.

3.10 El flujo completo del login en el proyecto

1. `POST /login` con `{ "email": "admin", "password": "1234", "role": "admin" }`.
2. `AuthController.Login(User user) → IAuthenticationService.AuthenticateAsync(email, password)`.
3. `AuthenticationService` compara hardcoded (`email=="admin" && password=="1234"`) → devuelve `User` o `null`.
4. Si `null` → `401 Unauthorized()`.
5. Si válido → `TokenGenerator.GenerateToken(validuser, jwtSettings)` → JWT.
6. Respuesta `200` con `{ "token": "eyJ..." }` (record `TokenResponse`). - **Riesgo conocido:** credenciales hardcodedas (`admin / 1234`) y `role` enviado por el cliente (nunca debe confiarse en el `role` del cliente; debe salir de la fuente de identidad).

3.11 Cómo se usa el token desde un cliente

- El cliente guarda el token y lo envía: `Authorization: Bearer eyJ...`. Swagger permite "Authorize" para inyectarlo. Sin él → `401`; con él y endpoint `[Authorize]` → `200`.

4. Conocimientos previos necesarios

- Módulo 1 (headers, status codes, stateless).
- Módulo 3 (middleware, pipeline, DI, configuración).
- Concepto básico de hash y clave (matemática simple de "firma").

5. Cómo aparece dentro del proyecto

Archivo	Concepto
<code>Controllers/AuthController.cs</code>	<code>[HttpPost("/login")]</code> , <code>[AllowAnonymous]</code> , <code>Unauthorized()</code> , <code>TokenResponse</code>
<code>Services/AuthenticationService.cs</code>	validación de credenciales hardcodeadas, devuelve <code>User</code> o <code>null</code>
<code>DTO/User.cs</code>	body del login (email, password, role nullable)
<code>Helpers/TokenGenerator.cs</code>	<code>claims</code> , <code>SymmetricSecurityKey</code> , <code>SigningCredentials(HS256)</code> , <code>JwtSecurityToken</code> , <code>WriteToken</code>
<code>Entities/JwtSettings.cs</code>	<code>Issuer</code> , <code>Audience</code> , <code>SecretKey</code>
<code>Startup.cs</code>	<code>AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(...)</code> , <code>TokenValidationParameters</code> , <code>AddAuthorization</code> , <code>app.UseAuthentication()</code> , <code>app.UseAuthorization()</code> , <code>services.AddSingleton(jwtSettings)</code> , <code>AddScoped<IAuthenticationService, ...></code>
<code>appsettings.json</code>	sección <code>JwtSettings</code>
<code>Controllers/LibrariesController.cs</code> <code>/ BooksController.cs</code>	<code>using Microsoft.AspNetCore.Authorization</code> ; <code>[Authorize]</code> (books, en su momento)

6. Nivel de importancia

Importante — el flujo de auth está presente en el proyecto y cualquier API real lo requiere.

7. Tiempo recomendado de estudio

15 horas.

8. Recursos recomendados

- **Documentación oficial:** jwt.io (playground y conceptos); "JSON Web Token" RFC 7519; Microsoft Learn "Authenticate users with JWT in ASP.NET Core".
- **Microsoft Learn:** "ASP.NET Core security" (module de autenticación/autorización).
- **Libros:** *Pro ASP.NET Core Identity* (Smith & McFetridge); *JWT Handbook* (Sebastián Peyrott, Auth0 — gratis en la web).
- **Videos:** "JWT — How it works" (YouTube, varios buenos), "JWT is Dangerous" (cómo NO usarlo), Nick Chapsas sobre JWT en ASP.NET Core.
- **Cursos:** Auth0 "Learn JWT" (auth0.com/learn).
- **Herramientas:** jwt.io para decodificar, Postman para el flujo, <https://jwt.ms>.

9. Ejercicios sugeridos

1. (Fácil) Haz `POST /login` con las credenciales correctas; pega el token en jwt.io y **decodifica** las tres partes. Identifica `iss` , `aud` , `exp` , `sub` , `email` , `role` .
2. (Fácil) Modifica un carácter del token y pruébalo en un endpoint protegido → `401` (la firma no valida).
3. (Medio) Llama a `/login` sin el campo `role` → `400` . Explica por qué (nullable/validation).
4. (Medio) Con el token vencido (espera 1h o genera uno con 1 minuto) prueba el endpoint → `401` .

5. (Medio) Agrega `[Authorize]` a `GET /api/libraries` y prueba con/sin token; luego revierte.
6. (Difícil) Cambia el algoritmo a RS256 (par de llaves) y valida el token con la clave pública. Documenta las diferencias con HS256.
7. (Difícil) Agrega una política de autorización `[Authorize(Policy = "RequireAdmin")]` basada en el claim de rol.
8. (Difícil) Implementa la lectura del token desde `HttpContext.User` dentro de un controller y extrae `NameIdentifier`, `Email`, `Role`.
9. (Difícil) Redacta un análisis de riesgo: ¿qué pasaría si el `SecretKey` se filtra? Propón mitigaciones (rotación, clave asimétrica, expiración corta).

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Poner secretos en el payload del JWT	Cualquiera los decodifica en jwt.io	El token es legible; solo firma. Guarda secretos en la BD
<code>SecretKey</code> corto/débil	Firma HS256 con clave trivial	Mínimo 32 bytes (256 bits) y generado aleatoriamente
ClockSkew enorme o fechas en local	Tokens que "expiran" raro	Usar <code>DateTime.UtcNow</code> y <code>ClockSkew</code> razonable
Olvidar <code>UseAuthentication()</code>	<code>[Authorize]</code> devuelve 401 sin razón	Montar el middleware antes de Endpoints
Confiar en el <code>role</code> del body	Ciente manda <code>role: admin</code>	El rol sale de la fuente de identidad, nunca del request
Configurar JWT dos veces	<code>Scheme already exists: Bearer</code>	Registrar una sola vez (guard del proyecto)
Emitir tokens de vida larga	Robo prolongado	<code>exp</code> corto + refresh tokens en sistemas reales
Creer que el token es cifrado	Leer claims sensibles	Pensar en el JWT como "firma + claims legibles"

MÓDULO 8 — Swagger y OpenAPI

1. Nombre del módulo

OpenAPI, Swagger, Swashbuckle y testing manual de APIs

2. Objetivo de aprendizaje

El estudiante entenderá qué es la especificación OpenAPI, cómo Swagger la genera y la expone como UI interactiva, y cómo usar Swagger UI / cURL / Postman como herramienta de trabajo diaria para probar y consumir la API.

3. Conceptos fundamentales

3.1 OpenAPI Specification (antes Swagger Spec)

- **Qué es:** estándar (YAML/JSON) para describir una API: endpoints, métodos, parámetros, bodies, status codes, esquemas JSON y seguridad. Es el "contrato legible por máquina" de la API.
- **Para qué sirve:** documentación viva, generación de clientes SDK, mocking, contratos entre equipos.
- **En el proyecto:** Swashbuckle genera el `swagger.json` a partir de los controllers (convenciones de routing + `[ApiController]` + DTOs).

3.2 Swagger UI

- **Qué es:** interfaz web interactiva que consume el OpenAPI: lista endpoints, permite probarlos (con autorización), muestra esquemas.
- **En el proyecto:** en `Development` (`/swagger`): `UseSwagger()` (sirve el JSON en `/swagger/v1/swagger.json`) y `UseSwaggerUI()` (interfaz HTML). Configurado con `SwaggerEndpoint` y título "LibraryService API v1".
- **Detalle:** `launchUrl: swagger` en `launchSettings.json` abre Swagger al arrancar.

3.3 Swashbuckle.AspNetCore

- **Qué es:** el paquete NuGet que integra la generación de OpenAPI + UI a ASP.NET Core (paquete meta: Swashbuckle.AspNetCore.Swagger + .SwaggerGen + .SwaggerUI).
- **En el proyecto:** `services.AddSwaggerGen(c => c.SwaggerDoc("v1", new OpenApiInfo { ... })).`

3.4 Testing manual de APIs (cURL / Postman / Thunder Client)

- **Qué es:** disparar requests y verificar respuestas fuera de los tests automatizados.
- **Cuándo se usa:** durante el desarrollo, para debugging, para validar contratos antes de escribir tests.
- **En el proyecto** (bitácora real): se probaron `/swagger/v1/swagger.json`, `/login`, `GET /api/libraries`, `GET /api/libraries/1/books` (401/200), etc. Swagger UI es la herramienta principal del lab.
- **Conceptos:** método + URL + headers (`Authorization`, `Content-Type`) + body JSON + lectura de status code y body.

3.5 Documentación rica (opcional)

- `[ProducesResponseType]`, `[SwaggerOperation]`, descripciones XML: enriquecen el OpenAPI. No se usan aquí, pero son la vía estándar para documentar APIs reales.

4. Conocimientos previos necesarios

- Módulo 1 (HTTP completo).
- Módulo 3 (controllers, routing).
- Módulo 7 (para probar endpoints con Bearer).

5. Cómo aparece dentro del proyecto

Archivo	Concepto
Startup.cs	AddSwaggerGen , SwaggerDoc("v1", OpenApiInfo) , UseSwagger() , UseSwaggerUI(...)
HackerRank1.csproj	<PackageReference Include="Swashbuckle.AspNetCore" Version="6.5.0" />
Properties/launchSettings.json	launchUrl: "swagger"
Controllers/*.cs	generan el OpenAPI automáticamente (rutas, verbos, DTOs, status codes implícitos)
docs/DOCUMENTACION-TECNICA.md	GET /swagger/v1/swagger.json → 200 (verificación)

6. Nivel de importancia

Importante — es la interfaz de trabajo diaria sobre esta API.

7. Tiempo recomendado de estudio

6 horas.

8. Recursos recomendados

- **Documentación oficial:** swagger.io, learn.openapis.org; Microsoft Learn "Swagger/Swashbuckle in ASP.NET Core".
- **Microsoft Learn:** "Document a web API with Swagger".
- **Libros:** *Designing Web APIs* (Brenda Jin et al.) — incluye diseño de contratos.
- **Videos:** "Intro to OpenAPI" (Swagger), "What is OpenAPI" (Zach Wilson).
- **Cursos:** Postman Academy (gratis).
- **Herramienta:** editor.swagger.io (diseñar API desde YAML), Postman, cURL.

9. Ejercicios sugeridos

1. (Fácil) Abre `/swagger/v1/swagger.json` y localiza el esquema JSON de `Book` y el endpoint POST de books.
2. (Fácil) Usa el botón "Authorize" de Swagger UI con el token del login y ejecuta un endpoint.
3. (Medio) Descarga el `swagger.json` y cárgalo en editor.swagger.io; visualiza la API como especificación.
4. (Medio) Prueba la misma operación desde Swagger UI, cURL y Postman; compara resultados.
5. (Difícil) Agrega `[ProducesResponseType]` y descripciones XML a `BooksController.Add` y observa cómo cambia el `swagger.json`.
6. (Difícil) Genera un cliente HTTP de la API a partir del OpenAPI (herramientas como `NSwag` / `openapi-generator`) y úsalo desde una consola.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Swagger solo en Development y quieren probarlo en prod	404 en <code>/swagger</code> en prod	Decidir conscientemente exponerlo o no (riesgo de info-leak)
Endpoint invisible en Swagger	Faltan <code>[HttpX]</code> o rutas ambiguas	Revisar routing y verbos
Body del POST vacío	400 "invalid JSON"	Enviar <code>Content-Type: application/json</code> + body válido
No autorizar Swagger con el token	401 en endpoints protegidos	Botón Authorize con <code>Bearer <token></code>
Documentación desactualizada	El OpenAPI no refleja el código	El OpenAPI generado siempre refleja el código (esa es la ventaja)

MÓDULO 9 — Testing

1. Nombre del módulo

Testing: xUnit, FluentAssertions, tests de integración, WebApplicationFactory, SQLite in-memory y mocking

2. Objetivo de aprendizaje

El estudiante entenderá la pirámide de testing, cómo funciona xUnit (proyecto, fixtures, `[Fact]`), cómo FluentAssertions hace legibles las aserciones, cómo los tests de integración levantan la app real con `WebApplicationFactory<Program>` sustituyendo la BD por SQLite in-memory, y cuándo y cómo mockear.

3. Conceptos fundamentales

3.1 ¿Por qué testear?

- **Qué es:** ejecutar el software con entradas conocidas y verificar salidas esperadas, de forma automatizada y repetible.
- **Para qué sirve:** detectar regresiones, permitir refactors seguros, documentar el comportamiento esperado (¡los tests de este proyecto SON el contrato de la API!), y dar confianza.
- **En el proyecto:** los 3 tests de integración son la especificación ejecutable de los requisitos (los códigos 201/404/204 del README).

3.2 Pirámide de testing

- **Unit tests:** prueban una unidad aislada (método/clase), rápido, sin BD ni red.
- **Integration tests:** prueban varias piezas juntas (app + BD + HTTP). Aquí, la API completa contra SQLite in-memory.
- **End-to-end (E2E):** prueban el sistema completo (incl. UI).
- **Regla:** muchas unit, algunas integration, pocas E2E.
- **En el proyecto:** solo integration (3 tests), lo cual es correcto para el tamaño del lab.

3.3 xUnit

- **Qué es:** framework de testing (.NET).
- **Conceptos:**
- `[Fact]` : test sin parámetros (los 3 del proyecto).
- `[Theory]` + `[InlineData]` : test parametrizado.
- `IClassFixture<T>` : comparte una instancia de `T` (fixture) entre tests de la misma clase → aquí `IClassFixture<WebApplicationFactory<Program>>` comparte el factory del host entre los 3 tests.
- `Assert` clásico vs `FluentAssertions` (3.4).
- **En el proyecto** (`LibraryService.Integration.Test`):
- `namespace LibraryService.Tests`
- `public class IntegrationTests : IClassFixture<WebApplicationFactory<Program>>` .
- Constructor recibe el `factory` por inyección (xUnit la construye).
- El `HttpClient` se crea con `_factory.WithWebHostBuilder(...).CreateClient()` .

3.4 FluentAssertions

- **Qué es:** librería de aserciones fluidas: `response.StatusCode.Should().BeEquivalentTo(StatusCodes.Status201Created); books.Count.Should().Be(2)` .
- **Ventaja:** legibilidad tipo frase, mensajes de error descriptivos.
- **En el proyecto:** usada en los 3 tests; versionada en 5.0.0.

3.5 WebApplicationFactory y TestServer

- **Qué es:** clase de `Microsoft.AspNetCore.Mvc.Testing` que levanta la app real (la de `Program`) en un `TestServer` en memoria (sin puertos) y expone un `HttpClient`.
- **Para qué sirve:** testear el pipeline completo (routing, middleware, controllers, servicios, BD) tal como corre en producción.
- **En el proyecto:**
- `IClassFixture<WebApplicationFactory<Program>>` — arranca el host una vez.
- `_factory.WithWebHostBuilder(builder => builder.UseStartup<Startup>().ConfigureServices(...))` — personaliza el arranque para los tests.
- `CreateClient()` — el cliente HTTP conectado al TestServer.
- **Detalle:** `Program` debe ser `public` (en este proyecto sí lo es) para que el factory lo referencie.

3.6 Sustitución de la BD: SQLite in-memory vs EF InMemory

- **SQLite in-memory** (`Microsoft.EntityFrameworkCore.Sqlite`, `DataSource=:memory:`): BD SQL *real* en memoria → respeta constraints (NOT NULL, FK, tipos) como el motor real. Es la elección correcta para tests de integración.
- **EF InMemory** (`Microsoft.EntityFrameworkCore.InMemory`): solo en memoria del proveedor; NO es SQL relacional, no respeta constraints ni se puede asumir equivalencia SQL. Útil para unit tests rápidos.
- **En el proyecto:** SQLite in-memory (con `UseSqlite`, `OpenConnection()`, `EnsureCreated()`, `SaveChanges()`, y detach del ChangeTracker entre usos). El package de EF InMemory está en el csproj pero no se usa en los tests actuales.
- **Por qué SQLite y no la BD real:** rapidez, determinismo, sin credenciales en CI. Tradeoff: el código no debe depender de features de Npgsql (decisión documentada en la bitácora).

3.7 Reemplazar el DbContext por DI (la clave de los tests)

- **Qué hace el test:** `csharp services.RemoveAll(typeof(LibraryContext)); services.AddSingleton(context); // el contexto SQLite preparado` Al quitar el registro original (que apuntaba a Npgsql) y añadir el contexto SQLite como Singleton, **toda la app** (controllers y services) usa el contexto de prueba vía el mismo DI.
- **Interacción con el guard de ConfigureServices:** como el contexto se reemplaza *después* de que `ConfigureServices` de Startup se ejecutó, y el guard evita la doble configuración, el flujo funciona sin duplicar esquemas JWT.

3.8 Arrange-Act-Assert (AAA)

- **Qué es:** estructura de todo test: 1) preparar (seed, DTO), 2) actuar (llamada HTTP), 3) verificar (assert).
- **En el proyecto:** `SeedLibrary()` (arrange) → `Client.PostAsync(...)` (act) → `StatusCode.Should().Be...` (assert).

3.9 Naming y estructura de tests

- **Convención:** `TestAddBook_Ok_GetBook_NotFound` = "qué se prueba_caso".
- **En el proyecto:** `TestAddBook_Ok_GetBook_NotFound`, `TestGetBooks_Ok_NotFound`, `TestDeleteLibrary`.
- **Fixtures de datos:** `SeedLibrary()` siembra 4 librerías; `SeedBook` usa la API (no la BD) — interesante: el seed vía API valida también el endpoint.

3.10 Mocking (Moq / NSubstitute) y test doubles

- **Qué es:** sustituir dependencias (servicios, DbSets) por falsificaciones programadas para aislar la unidad bajo test.
- **Conceptos:** `Mock<T>`, `Verify`, `Setup`, `ReturnsAsync`. Tipos de doubles: fake, stub, mock, spy.
- **En el proyecto:** NO se usan mocks — los tests son de integración con la app real y una BD real (SQLite). Se prefiere el contexto real al mock porque probar los mocks de `DbSet` es frágil.
- **Cuándo mockear:** unit tests de servicios/controllers; cuando la dependencia es cara o no disponible.

3.11 Cobertura y el warning xUnit1031

- **Qué es:** la cobertura mide cuántas líneas ejecutan los tests. Útil como señal, no como meta.
- **En el proyecto:** el test usa `response.Content.ReadAsStringAsync().Result` → warning `xUnit1031` (no bloquee async en tests). La solución es `await` (el `ReadBody<T>` de `HttpResponseExtensions` existe para eso, aunque no se usa en todos los tests).

3.12 El ciclo test-first y TDD (fundamento)

- **Qué es:** escribir el test que falla → implementar → verde (red-green-refactor). Los tests dados en el lab juegan ese papel: son el contrato que la implementación debe cumplir.
- **En el proyecto:** los tests ya existían ("Read-Only Files" según README) y la implementación se ajustó a ellos.

4. Conocimientos previos necesarios

- Módulos 1 y 3 (HTTP y arranque de la app — los tests prueban ambas cosas).
- Módulo 5 (DbContext, EnsureCreated, ChangeTracker, SQLite).
- Módulo 2 (async, LINQ, generics).

5. Cómo aparece dentro del proyecto

Archivo	Concepto
<code>LibraryService.Integration.Test/IntegrationTest.cs</code>	<code>IClassFixture<WebApplicationFactory<Program>></code> , <code>[Fact]</code> , AAA, <code>WithWebHostBuilder</code> , <code>RemoveAll</code> , <code>AddSingleton(context)</code> , SQLite in-memory, <code>EnsureCreated</code> , detach, <code>SeedLibrary</code> / <code>SeedBook</code> , asserts FluentAssertions
<code>LibraryService.Integration.Test/Extensions/HttpResponseExtensions.cs</code>	<code>ReadBody<T></code> con Newtonsoft
<code>LibraryService.Integration.Test/LibraryService.Integration.Test.csproj</code>	xunit, FluentAssertions 5.0.0, Mvc.Testing 8.0.16, EFCore.Sqlite 8.0.2, EFCore.InMemory 8.0.2, Newtonsoft.Json, MSTest.TestFramework (usado vía using, no vía atributos), <code>IsTestProject</code> , ProjectReference a la API
<code>Startup.cs</code>	guard de idempotencia + guard SQLite (para coexistir con los tests)
<code>docs/DOCUMENTACION-TECNICA.md</code>	bugs reales de testing (versiones EF 6 vs 8, <code>Migrate</code> vs <code>EnsureCreated</code> , doble <code>ConfigureServices</code>)

6. Nivel de importancia

Fundamental — los tests son el contrato de esta API y la habilidad de testear es imprescindible.

7. Tiempo recomendado de estudio

20 horas.

8. Recursos recomendados

- **Documentación oficial:** learn.microsoft.com → xUnit Guide (.NET); "Integration tests in ASP.NET Core" (Microsoft Learn).
- **Microsoft Learn:** "Test an ASP.NET Core web API" (module).
- **Libros:** *The Art of Unit Testing* (Roy Oshero); *Unit Testing Principles, Practices, and Patterns* (Vladimir Khorikov).
- **Videos:** "xUnit for beginners" (Nick Chapsas); "Integration Testing ASP.NET Core" (Nick Chapsas); "Testcontainers for .NET" (avanzado, BD real en contenedores).
- **Cursos:** Pluralsight "Testing in ASP.NET Core".
- **Repositorios:** `dotnet/EntityFramework.Docs` (samples de testing con SQLite/InMemory); `testcontainers/testcontainers-dotnet`.

9. Ejercicios sugeridos

1. (Fácil) Ejecuta `dotnet test` y explica qué pasa: qué se levanta, qué BD se usa, cuántos tests corren.
2. (Fácil) Cambia un assert (`Status201Created` → `Status200OK`) y observa el mensaje de fallo de FluentAssertions; luego reviértelo.

3. (Medio) Agrega un test `TestDeleteLibrary_NotFound` que borre una librería inexistente y verifique `404`.
4. (Medio) Escribe un unit test con Moq del `BooksService` (mockea `DbSet<Book>` con datos en memoria) — compara con el approach de integración.
5. (Medio) Convierte un `[Fact]` en `[Theory]` con `[InlineData]` para el caso de librerías inexistentes (varios ids).
6. (Difícil) Sustituye SQLite por `UseInMemoryDatabase` y ejecuta los tests; documenta qué cambia (o qué rompe, p.ej. la FK CASCADE).
7. (Difícil) Agrega cobertura de cobertura (`dotnet test --collect:"XPlat Code Coverage"`) y reporta el % del proyecto.
8. (Difícil) Reproduce el bug real: en el proyecto de tests usa EF 6.0 → `MissingMethodException` ; alinea a 8.0.2 y explica la causa (dependencia transitiva vs directa).
9. (Difícil) Implementa el patrón Testcontainers para correr los tests contra PostgreSQL real en Docker (preparación del Módulo 15).

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Bloquear con <code>.Result</code> en tests	Warning <code>xUnit1031</code> , tests lentos	<code>await</code> (<code>ReadBody<T></code> del proyecto)
Mezclar versiones EF (6 vs 8)	<code>MissingMethodException ... ConventionSet</code>	Alinear versiones (bug real del repo)
<code>Migrate()</code> en tests con <code>EnsureCreated()</code>	Conflictos de esquema	Guard del proyecto: auto-migrar solo si no es SQLite
Registrar el contexto dos veces	El contexto de producción "gana"	<code>RemoveAll(typeof(LibraryContext))</code> antes de <code>AddSingleton</code>
Testear contra BD real	Tests frágiles, dependen de red/credenciales	BD en memoria (SQLite) o Testcontainers
Crear que EF InMemory es igual a SQL	Tests que pasan en memoria y fallan en Postgres	Usar SQL relacional (SQLite) para integración
Sin arreglos de versiones al cambiar SDK	Builds rotos en CI	Congelar versiones en <code>.csproj</code> (como el proyecto)

MÓDULO 10 — Git y GitHub

1. Nombre del módulo

Git: control de versiones, commits, Conventional Commits, ramas y Pull Requests

2. Objetivo de aprendizaje

El estudiante entenderá el flujo completo de versionado que usa este repositorio: cómo se clona, cómo se hizo el historial de commits (con estilo Conventional Commits), por qué `.gitignore` protege secretos, y cómo colaborar con ramas y pull requests en GitHub.

3. Conceptos fundamentales

3.1 Control de versiones y Git

- **Qué es:** sistema que registra el historial completo de cambios de archivos (quién, qué, cuándo). Git es distribuido: cada clon tiene el historial completo.
- **Para qué sirve:** volver atrás, colaborar, revisar cambios, releases reproducibles.
- **En el proyecto:** el historial real es: `199fc31 docs: agregar guia en PDF de la ruta del proyecto y user secrets e95aee8 docs: agregar guia visual de User Secrets cbe5a1b feat: implementar endpoints de libraries y books con tests en verde bd3861b docs: documentar procesos y actualizar README con setup local bb96648 chore: configurar conexion a Supabase con user-secrets 55cead1 my first upload`
- **Conceptos:** repositorio, working tree (archivos en disco), **index/staging** (área de preparación), **commit** (snapshot), **HEAD** (puntero al último commit), **refs** (ramas/tags).

3.2 Flujo básico (working directory → staging → commit)

- `git add <archivo>`: mueve cambios al staging.
- `git commit -m "mensaje"`: guarda el snapshot.
- `git status`: estado actual.
- `git diff`: cambios sin stagear; `git diff --cached`: stageados.
- `git log --oneline`: historial.
- **En el proyecto:** los commits del repo siguen este flujo (ej. el commit `bb96648` incluyó `.csproj` y `appsettings.json`).

3.3 Conventional Commits

- **Qué es:** convención de mensajes `tipo(scope): descripción`. Tipos: `feat`, `fix`, `docs`, `chore`, `refactor`, `test`, `style`, `perf`, `ci`, `build`, `revert`.
- **Para qué sirve:** historial legible, semver automático, changelogs generados.
- **En el proyecto:** `feat:`, `docs:`, `chore:` — conforme al estándar.

3.4 .gitignore

- **Qué es:** lista de archivos/carpetas que Git ignora.
- **En el proyecto** (`/Paradigma-lab1/.gitignore`): ignora `bin/`, `obj/` (outputs de build), `.vs/`, `.idea/`, `.vscode/`, `*.user`, `*.log`, `appsettings.*.local.json`.
- **Por qué importa:** los secretos y artefactos de build nunca deben subir. Nota: el archivo **no** ignora `secrets.json` explícitamente porque vive fuera del repo (en `%APPDATA%`).
- **Peligro real:** si un secreto se commitó alguna vez, `gitignore` no lo borra del historial — hay que sanear (filter-repo / BFG).

3.5 Remotos y GitHub

- **Qué es:** un remoto (`origin`) es otra copia del repo (GitHub). `git push origin HEAD`, `git pull`, `git fetch`, `git clone`.
- **En el proyecto:** `origin/main`; el push del commit de credenciales se hizo con `git push origin HEAD`.

3.6 Ramas y merge

- **Qué es:** rama = puntero móvil a commits. `main` es la principal. Feature branches aíslan trabajo.
- **Comandos:** `git checkout -b feature/x`, `git merge`, `git rebase`.
- **Por qué importan:** desarrollo paralelo, revisión, protección de `main` (branch protection).

3.7 Pull Requests (PRs)

- **Qué es:** mecanismo de GitHub para proponer cambios de una rama a otra: diffs, comentarios, revisiones, checks de CI, merge.
- **Cuándo se usan:** colaboración; en repos con protección de rama, es la única vía a `main`.
- **Partes de una PR:** título, descripción, commits, review (approve/request changes), CI checks, merge (merge/squash/rebase).

3.8 Resolver conflictos

- **Qué es:** cuando dos ramas cambian las mismas líneas, Git no puede mergear solo → marca `<<<<<<`, `=====`, `>>>>>>`.
- **Cómo se resuelve:** editar el archivo, stage, commit del merge.

3.9 Buenas prácticas de historial

- Commits pequeños y atómicos; mensajes claros; no commitear binarios/secretos; `.gitignore` antes de `git add .`; revisar con `git diff` antes de commitear; nunca `--force` sin necesidad.

4. Conocimientos previos necesarios

- Uso básico de terminal.
- Ningún módulo técnico previo es obligatorio (se puede aprender en paralelo).

5. Cómo aparece dentro del proyecto

Archivo	Concepto
<code>.git/</code>	repositorio
<code>.gitignore</code>	<code>bin/</code> , <code>obj/</code> , <code>.vs/</code> , <code>*.user</code> , etc.
<code>git log --oneline</code> (historial real)	commits en estilo Conventional Commits
Commit <code>bb96648</code>	"chore: configurar conexion a Supabase con user-secrets" — incluyó <code>UserSecretsId</code> en csproj y connection string con placeholder
<code>origin/main</code>	remoto y rama principal
<code>docs/REPORTE-USER-SECRETS.md</code>	sección "Verificación" con <code>git grep -in "Password="</code> , <code>git show HEAD:...</code> , <code>git ls-files</code>

6. Nivel de importancia

Importante — es la herramienta de trabajo cotidiana; el repo es la entrega del lab.

7. Tiempo recomendado de estudio

10 horas.

8. Recursos recomendados

- **Documentación oficial:** git-scm.com/doc (Pro Git — libro gratuito); Conventional Commits (conventionalcommits.org).
- **Microsoft Learn:** "Introduction to Git" y "GitHub best practices".

- **Libros:** *Pro Git* (Chacon & Straub, gratis); *Git Pocket Guide*.
- **Videos:** "Git & GitHub Crash Course" (freeCodeCamp); "Git for Professionals" (ThePrimeagen).
- **Cursos:** GitHub Skills (skills.github.com — interactivos y gratuitos).
- **Herramientas:** GitHub CLI (`gh`), GitKraken/SourceTree para GUI.

9. Ejercicios sugeridos

1. (Fácil) `git log --oneline` sobre el repo y traduce cada mensaje a su tipo Conventional Commit.
2. (Fácil) Crea una rama `feature/experimento`, haz un commit y mergea de vuelta a `main`.
3. (Medio) Practica `git reset --soft/--mixed/--hard` en un repo de prueba; documenta qué pasa con el staging y el working tree en cada uno.
4. (Medio) Crea un PR real desde una rama a `main` en un repo tuyo; pide revisión y mergea con squash.
5. (Medio) Genera un conflicto a propósito (dos ramas editan la misma línea) y resuélvelo.
6. (Difícil) Demuestra con `git grep "Password="` que el repo no tiene secretos; luego agrega un `.gitignore` más estricto para tu proyecto.
7. (Difícil) Reescribe el mensaje del commit más antiguo con `git rebase -i` y observa cómo cambia el historial (¡en un repo de práctica!).
8. (Difícil) Practica recuperar un commit "perdido" con `git reflog`.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Committear secretos	<code>git grep "Password="</code> encuentra la password	Quitar del working tree + sanear historial (filter-repo)
<code>git add .</code> ciego	Basura en staging (bin/obj)	<code>.gitignore</code> + revisar <code>git status</code>
Mensajes vagos ("fix", "update")	Historial inútil	Conventional Commits
Trabajar todo en <code>main</code>	Sin PRs ni revisión	Feature branches + PR
Mergear con conflictos mal resueltos	Código roto "sin querer"	Resolver con cuidado y revisar el diff
<code>git push --force</code> destructivo	Historia reescrita en remoto	Evitar; usar <code>--force-with-lease</code> si es necesario

MÓDULO 11 — Azure DevOps

1. Nombre del módulo

Azure DevOps: Backlog, PBIs, Features, Tasks y Sprints (gestión ágil del trabajo)

2. Objetivo de aprendizaje

El estudiante entenderá el modelo de gestión de trabajo ágil de Azure DevOps (Azure Boards): tipos de Work Items (Epic, Feature, PBI, Task, Bug), el backlog, la estimación (Story Points), los sprints y cómo descomponer el trabajo de este proyecto en Work Items.

3. Conceptos fundamentales

3.1 Azure DevOps

- **Qué es:** plataforma de Microsoft para todo el ciclo de vida del software: Azure Boards (gestión ágil), Azure Repos (git), Azure Pipelines (CI/CD), Azure Test Plans, Azure Artifacts.
- **Para qué sirve:** planificar, desarrollar, integrar y desplegar en un solo lugar. El proyecto la menciona como herramienta de gestión (el lab se organizó en tareas).
- **En el proyecto:** no hay código de Azure DevOps en el repo (`.github/workflows/` está vacío), pero la gestión del trabajo se modela con sus conceptos.

3.2 Work Items (ítems de trabajo)

- **Qué son:** las unidades de trabajo trazables. En el modelo "Basic"/"Scrum":
- **Epic:** iniciativa grande (varios Features).
- **Feature:** funcionalidad de negocio (varios PBIs).
- **PBI (Product Backlog Item):** requerimiento de usuario en el backlog.
- **Task:** trabajo técnico concreto derivado de un PBI.
- **Bug:** defecto.
- **En el proyecto:** los endpoints son PBIs; su implementación (service, controller, test) son Tasks; arreglar la connection string fue un Bug; el guard de idempotencia fue una Task técnica.

3.3 Backlog y Product Backlog

- **Qué es:** lista priorizada de PBIs/Features que el equipo debe entregar.
- **Priorización:** MoSCoW, valor vs esfuerzo, dependencias.
- **En el proyecto** (backlog real): PBI "DELETE library", PBI "POST book", PBI "GET books con 404", PBI "User Secrets".

3.4 Sprints e iteraciones

- **Qué es:** ciclos de trabajo de duración fija (1-4 semanas) con un objetivo; al final debe haber entregables.
- **Sprint Planning:** qué se compromete. **Daily:** seguimiento. **Review:** demo. **Retro:** mejora.
- **En el proyecto:** el lab se completó en iteraciones cortas (Fase 1 análisis, Fase 2 entorno, Fase 3 endpoints, Fase 4 versionado — documentadas en la bitácora).

3.5 Estimación: Story Points y Planning Poker

- **Qué es:** estimación relativa de esfuerzo (Fibonacci: 1,2,3,5,8,13), no de horas.
- **Para qué sirve:** velocidad del equipo, pronóstico de sprints.
- **Cómo estimar un endpoint:** complejidad de lógica, de integración (BD), de tests, de configuración.

3.6 Acceptance Criteria y Definition of Done

- **Acceptance Criteria:** condiciones verificables de un PBI ("POST book a librería inexistente → 404").
- **Definition of Done (DoD):** checklist transversal ("compila, tests verdes, sin secretos, revisado").
- **En el proyecto:** los criterios del README + tests de integración funcionan como Acceptance Criteria ejecutables.

3.7 Boards, sprints y burndown

- **Qué es:** tableros (To Do / In Progress / Done), gráficos de avance (burndown), filtros por persona, áreas e iteraciones.

3.8 Relación con CI/CD (Azure Pipelines)

- **Qué es:** pipelines `azure-pipelines.yml` para build + test + deploy (ampliar en Módulo 14). Conceptos: triggers, stages, jobs, steps, artifacts.

4. Conocimientos previos necesarios

- Concepto de proyecto y entregables.
- Módulo 9 ayuda a entender "Acceptance Criteria ejecutables = tests".

5. Cómo aparece dentro del proyecto

Elemento	Concepto
<code>docs/DOCUMENTACION-TECNICA.md</code> secciones 8-10	descomposición del trabajo en tareas con orden de dependencias
README "Requirements"	PBIs con Acceptance Criteria (201/404/204)
<code>LibraryService.Integration.Test/IntegrationTest.cs</code>	DoD ejecutable (tests = criterios de aceptación)
<code>.github/workflows/</code> (vacío)	aún no hay CI configurado (campo de mejora)

6. Nivel de importancia

Complementario — no es código, pero es el proceso que organiza el trabajo de un equipo.

7. Tiempo recomendado de estudio

8 horas.

8. Recursos recomendados

- **Documentación oficial:** learn.microsoft.com → "Azure Boards", "Agile and Scrum terms".
- **Microsoft Learn:** "Manage project work with Azure Boards".
- **Libros:** *Scrum: The Art of Doing Twice the Work in Half the Time* (Jeff Sutherland); *Essential Scrum* (Kenneth Rubin).
- **Videos:** "Azure Boards walkthrough" (Microsoft); "Scrum in under 5 minutes".
- **Cursos:** Azure DevOps learning paths (learn.microsoft.com).
- **Herramienta:** cuenta gratuita de Azure DevOps.

9. Ejercicios sugeridos

1. (Fácil) Crea una organización/ proyecto en Azure DevOps con proceso Scrum.
2. (Fácil) Modela el backlog del lab: 1 Feature "Library CRUD", 3-4 PBIs, 8-10 Tasks con estimación en Story Points.
3. (Medio) Define Acceptance Criteria para el PBI "GET books" basándote en el contrato de los tests.
4. (Medio) Simula un sprint de 1 semana: planifica, mueve tarjetas y crea un burndown.
5. (Difícil) Conecta Azure Boards a un repo de GitHub y vincula commits a Work Items por `#id` o `AB#id`.
6. (Difícil) Redacta tu DoD personal para "endpoint terminado" y úsalo en tu próximo ejercicio.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Confundir PBI con Task	Tareas sin requerimiento de usuario	PBI = "qué", Task = "cómo"
Sprints sin DoD	"Terminado" es ambiguo	Definir DoD escrito y compartido
Estimación en horas rígida	Quema de horas frustrante	Story Points relativos
No vincular tests a criterios	Features "verdes" pero sin test	Los criterios deben ser verificables (tests)
Backlog sin prioridad	Trabajo arbitrario	Priorizar por valor y dependencias

MÓDULO 12 — Buenas prácticas de desarrollo

1. Nombre del módulo

SOLID, Clean Code, Async/Await, Naming, DTOs, Validación, Manejo de errores y Logging

2. Objetivo de aprendizaje

El estudiante aplicará principios de diseño y calidad de código al escribir y refactorizar C#/ASP.NET Core: identificará smell codes en este proyecto, los corregirá y escribirá código limpio desde el inicio.

3. Conceptos fundamentales

3.1 SOLID

- **S — Single Responsibility (SRP):** una clase tiene una sola razón para cambiar. `LibrariesController` solo maneja HTTP; `LibrariesService` solo acceso a datos; `TokenGenerator` solo firma tokens.
- **O — Open/Closed:** abiertas a extensión, cerradas a modificación. Añadir un nuevo proveedor de auth no debería modificar `AuthenticationService` (usar interfaces/polimorfismo).
- **L — Liskov Substitution:** una subclase debe poder sustituir a su base sin romper el comportamiento.
- **I — Interface Segregation:** interfaces pequeñas y específicas. `ILibrariesService` expone solo lo que usa el controller.
- **D — Dependency Inversion:** depender de abstracciones (controllers dependen de `ILibrariesService`, no de la clase concreta).
- **En el proyecto:** se cumple razonablemente (interfaces, DI). Fallas: entidades con responsabilidad de datos mezcladas con `[Key]` (acceptable), `AuthenticationService` con lógica hardcodeada (anti-SRP pero es scaffolding).

3.2 Clean Code

- **Qué es:** código legible que "se lee como prosa": nombres claros, funciones pequeñas, sin duplicación, intención explícita.
- **Reglas clave:** nombres que dicen el *qué*; funciones que hacen *una* cosa; comentarios que explican el *por qué* (no el *qué*); evitar magia numérica; YAGNI (no agregues lo que no necesitas); DRY con juicio.
- **En el proyecto:** nombres claros (`_librariesService`, `bookForm`, `createdBook`). Mejoras posibles: el `Update` del service con `Update()` redundante; comentarios en inglés/ortografía del scaffolding.

3.3 Naming

- **Convenciones:** PascalCase (clases/métodos), camelCase (variables/parámetros), prefijo `_` para campos privados, `I` para interfaces, sufijos `Controller` / `Service`, `Async` para métodos async, tipos con nombre descriptivo (`BookForm`).
- **En el proyecto:** `_libraryContext`, `ILibrariesService`, `TokenGenerator`, `Task<IActionResult> Login(User user)`.

3.4 Async/Await best practices

- `async` en toda la cadena (no intercalar sync/async).
- Nunca `.Result` / `.Wait()` (deadlocks/warnings — el `xUnit1031` del proyecto).
- Nombre de métodos async con sufijo `Async`.
- Cancelación: `CancellationToken` en operaciones de BD y HTTP (los services no lo aceptan — mejora).
- `ConfigureAwait(false)` solo cuando corresponde (librerías), no en código de app por defecto.
- **En el proyecto:** toda la cadena controller→service→EF es async y nombrada `Async`. El test con `.Result` es la mala práctica a corregir.

3.5 DTOs (profundización)

- **Qué son:** objetos de transferencia. Separar el modelo de entrada (request) del de salida (response) del interno (entity).
- **Beneficios:** validación controlada, no exponer la entidad completa, evolucionar el contrato sin tocar el dominio.

- **En el proyecto:** `BookForm` (entrada). Debilidades: `User` mezcla entrada (email/password) y salida (role); `Library` / `Book` (entidades) se exponen directamente como respuesta → acoplamiento contrato-eficiencia. Mejora sugerida: `LibraryResponse` , `BookResponse` .
- **Mapeo:** manual hoy; herramientas: AutoMapper (más boilerplate), implicit operators, records.

3.6 Validación

- **Qué es:** verificar que los datos de entrada cumplen reglas ANTES de usarlos.
- **Mecanismos en ASP.NET Core:** `[Required]` , `[MaxLength]` , `[Range]` , etc. + validación automática de `[ApiController]` → `400` con `ProblemDetails` . También `[FromBody]` explícito, `ModelState` .
- **En el proyecto:** la validación implícita por nullable (`Category` / `Role` obligatorios → bug 400) fue el caso real. No hay validaciones explícitas de negocio (ej. `Name` no vacío) — mejora.
- **Regla:** validar en el borde de entrada (controller/DTO), no en el service a medias.

3.7 Manejo de errores (error handling)

- **Estrategia:**
- Errores *esperados* (recurso no existe) → códigos HTTP: `NotFound()` , `BadRequest()` .
- Errores *inesperados* (BD caída) → excepción + middleware de manejo global + logging + `500` .
- `ExceptionHandler` / filtro `ExceptionHandler` : capturar, loggear, responder genérico (no filtrar stack traces en prod).
- **En el proyecto:** no hay middleware global de errores; el pipeline solo usa `UseDeveloperExceptionPage` en Development. En Production devolvería `500` por defecto sin JSON estructurado (mejora: `UseExceptionHandler`).
- `DeveloperExceptionPage` : solo Development — nunca en producción (expone stack traces = riesgo).

3.8 Logging

- **Qué es:** registro de eventos (`ILogger<T>`). Niveles: `Trace/Debug/Information/Warning/Error/Critical` .
- **Reglas:** loggear eventos significativos y errores con contexto; nunca credenciales/PII; no loggear en loops calientes sin nivel bajo.
- **En el proyecto:** la infraestructura existe (`Logging` en `appsettings.Development.json`) pero ningún service/controller usa `ILogger` — mejora clara. El `AddDbContextPool` puede loggear SQL con `EnableSensitiveDataLogging` (solo dev).

3.9 Nullable y manejo de null (defensivo)

- `string?` , `??` , `?.` , `??=` para código null-safe. Verificación de `library == null` en controllers (el patrón 404 del proyecto).

3.10 El patrón "find-then-404" (respuesta HTTP correcta)

- **En el proyecto:** cada acción que actúa sobre un recurso por id hace: buscar → si null → `404` ; si existe → operar. Consistente en `LibrariesController` , `BooksController` .

4. Conocimientos previos necesarios

- Módulos 2 y 3 (C# y ASP.NET Core) — no puedes aplicar buenas prácticas sin el lenguaje/framework.
- Módulo 4 (arquitectura) para las decisiones de diseño.

5. Cómo aparece dentro del proyecto

Archivo	Buenas prácticas (y smell codes)
<code>Services/LibraryService.cs</code>	interfaces + DI (DIP); <code>Update</code> con <code>.Update()</code> redundante (tracker) — smell menor
<code>Controllers/BooksController.cs</code>	mapeo manual DTO→entidad; default <code>Category ?? string.Empty</code> (defensivo); 404-check
<code>Controllers/LibrariesController.cs</code>	patrón find-then-404 consistente
<code>Services/AuthenticationService.cs</code>	credenciales hardcodeadas (anti-buena práctica de seguridad; ver Módulo 13)

Archivo	Buenas prácticas (y smell codes)
<code>Controllers/AuthController.cs</code>	devuelve entidad <code>User</code> en validación interna; sin loguear intentos fallidos
<code>DTO/BookForm.cs</code>	<code>[JsonPropertyName]</code> (System.Text.Json); <code>string?</code> correcto
<code>LibraryService.Integration.Test/IntegrationTest.cs</code>	<code>.Result</code> (warning xUnit1031) — mala práctica async
<code>docs/DOCUMENTACION-TECNICA.md</code>	23 warnings (CS8618, CS8603/8604, xUnit1031) como deuda a limpiar

6. Nivel de importancia

Importante — diferencia entre "código que funciona" y "código que otro mantiene".

7. Tiempo recomendado de estudio

15 horas.

8. Recursos recomendados

- **Documentación oficial:** Microsoft Learn "C# best practices", "Fundamentals of error handling", "Logging in .NET".
- **Libros:** *Clean Code* (Robert C. Martin); *Clean Craftsmanship* (ídem); *The Pragmatic Programmer* (Hunt & Thomas).
- **Videos:** "SOLID principles" (freeCodeCamp / Nick Chapsas); "Async await best practices" (Nick Chapsas).
- **Cursos:** Pluralsight "Clean Code" tracks.
- **Herramientas:** analyzers de Roslyn (`.editorconfig` , SonarQube/SonarLint).

9. Ejercicios sugeridos

1. (Fácil) Clasifica cada clase del proyecto según los 5 principios SOLID; marca dónde se viola y por qué.
2. (Fácil) Corrige los warnings `CS8618` del proyecto inicializando propiedades.
3. (Medio) Refactoriza el test para eliminar `.Result` usando `await` y `ReadBody<T>`.
4. (Medio) Agrega validación `[Required]` / `[MaxLength]` a `BookForm.Name` y verifica el `400` automático.
5. (Medio) Crea DTOs de respuesta (`LibraryResponse` , `BookResponse`) y mapea en los controllers; compara con exponer entidades.
6. (Difícil) Implementa un `ExceptionHandler` que loguee con `ILogger` y devuelva `500` JSON en Production, `500` con detalles en Development.
7. (Difícil) Agrega `CancellationToken` a toda la cadena async de los services y controllers.
8. (Difícil) Agrega `ILogger<T>` a `BooksService` y loguea creación/borrado de libros con contexto (sin datos sensibles).

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
<code>.Result</code> / <code>.Wait()</code> en app o tests	Warnings, deadlocks	<code>await</code> en toda la cadena
Sin manejo de errores global	<code>500</code> HTML con stack trace en prod	<code>UseExceptionHandler</code> + logging
Exponer entidades directamente	Cambio de BD rompe el contrato JSON	DTOs de respuesta
Validación en capas equivocadas	Servicios validando "de nuevo"	Validar en el borde (DTO/controller)
Loguear secretos/PII	Passwords o emails en logs	Sanear antes de loguear
Nombres confusos	<code>x</code> , <code>temp</code> , <code>data</code>	Nombrar por intención
Comentarios que repiten el código	<code>// incrementa i</code>	Comentarios de <i>por qué</i> , no de <i>qué</i>

MÓDULO 13 — Seguridad

1. Nombre del módulo

Seguridad en APIs: secretos, OWASP Top 10, SQL Injection, validaciones y seguridad de JWT

2. Objetivo de aprendizaje

El estudiante identificará los riesgos de seguridad presentes en este proyecto (y en cualquier API), entenderá el OWASP Top 10 aplicado a una Web API .NET, sabrá por qué EF Core previene SQL Injection, cómo se gestionan secretos y cómo endurecer la autenticación JWT.

3. Conceptos fundamentales

3.1 Seguridad de aplicaciones web — visión general

- **Qué es:** proteger la app, sus datos y a sus usuarios de ataques. Es transversal: no es "un módulo al final", es parte de cada decisión.
- **Principios:** menor privilegio, defensa en profundidad, no confiar en la entrada del cliente, secretos fuera del código, cifrado en tránsito y en reposo.

3.2 OWASP Top 10 (aplicado a este proyecto)

- **A01 — Broken Access Control:** endpoints protegidos incorrectamente. En el proyecto: `GET books` quedó público (decisión de lab); si se requiere auth, controlar bien. La regla es: **auth por defecto, abrir explícitamente**.
- **A02 — Cryptographic Failures:** claves débiles o expuestas. En el proyecto: `SecretKey` en `appsettings.json` y credenciales hardcodeadas.
- **A03 — Injection (SQL/XSS):** EF Core parametriza (seguro frente a SQL Injection); riesgo en SQL manual/concatenado.
- **A04 — Insecure Design:** patrones inseguros por diseño (devolver entidades completas, `role` que llega del cliente).
- **A05 — Security Misconfiguration:** `Trust Server Certificate=true`, `DeveloperExceptionPage` en prod, CORS demasiado abierto.
- **A06 — Vulnerable Components:** versiones desactualizadas de NuGet (este repo mezcló EF 6.0 y 8.0).
- **A07 — Auth Failures:** credenciales hardcodeadas (`admin / 1234`).
- **A08 — Integrity failures:** firmas (JWT HS256) y actualizaciones.
- **A09 — Logging/monitoring failings:** no hay logs de intentos de login fallidos (si existe el endpoint, deberían auditarse).
- **A10 — SSRF:** peticiones a URLs del cliente (no aplica directamente aquí, pero conviene conocerlo).

3.3 Gestión de secretos (profundización)

- **Qué es:** secretos = passwords, connection strings, claves de firma, tokens. Regla: **nunca** en el repo ni en código.
- **En el proyecto:** password de Supabase en User Secrets (dev); `SecretKey` JWT aún en `appsettings.json` (deuda). La lección del lab: los secretos son una decisión de arquitectura inicial ("es mucho más difícil sanear un secreto ya expuesto en git").
- **Jerarquía correcta:** dev → User Secrets; CI → secretos de pipeline; prod → Secret Manager del proveedor (Azure Key Vault, AWS Secrets Manager) o variables de entorno cifradas.

3.4 SQL Injection

- **Qué es:** inyección de código SQL malicioso vía inputs no saneados.
- **Ejemplo peligroso:** `"SELECT * FROM Books WHERE Name='" + name + "'" → name = "'; DROP TABLE Books;--"`.
- **Por qué EF Core es seguro:** traduce LINQ a SQL parametrizado (`WHERE "Name" = @p0`). El valor nunca se concatenó.
- **Regla:** nunca concatenar inputs en SQL; usar EF/LINQ o parámetros (`NpgsqlParameter`).

3.5 Validación de entrada (límites)

- **Qué es:** validar tamaño, tipo, formato, rango y reglas de negocio ANTES de procesar.
- **En el proyecto:** la validación automática `[ApiController]` devuelve `400`. Faltan: `[MaxLength]`, `[Required]` en `BookForm.Name`, `[EmailAddress]`, `[Range]`.

- **Regla de oro:** nunca confíes en el body; valida todo en el borde.

3.6 Seguridad de JWT (endurecimiento)

- `SecretKey` ≥ 32 bytes aleatorios, guardado en secreto (no en `appsettings.json`).
- Expiración corta + refresh tokens.
- Preferir **RS256** (asimétrico) en producción para revocación/rotación y separación de emisor/verificador.
- `ValidateIssuer/Audience/Lifetime` activados (el proyecto lo hace).
- No meter datos sensibles en claims.
- No aceptar `role` del cliente: el rol debe salir de la fuente de identidad.
- Protección contra "alg confusion" (forzar algoritmo esperado), revisión de `typ`, listas de revocación si es necesario.

3.7 TLS/SSL y cifrado

- **En tránsito:** HTTPS para la API; `SSL Mode=Require` para la BD.
- **En reposo:** encriptación de BD (Supabase ofrece), secretos cifrados.
- `Trust Server Certificate=true`: aceptable solo para lab; en prod usar CA válida.

3.8 CORS (Cross-Origin Resource Sharing)

- **Qué es:** mecanismo del navegador que permite/niega peticiones entre orígenes. El servidor responde headers `Access-Control-Allow-Origin`.
- **En el proyecto:** política "Frontend" permite solo `http://localhost:5173` (Vite). Correcto y restrictivo. Riesgo: abrir a `*` en prod.
- **Errores comunes:** olvidar CORS (la app web no puede llamar), o abrir `AllowAnyOrigin` + `AllowAnyHeader` + `AllowAnyMethod` indiscriminadamente.

3.9 Autenticación robusta (más allá del lab)

- Hashing de passwords (BCrypt/PBKDF2/Argon2id), nunca texto plano ni MD5/SHA1.
- Credenciales en BD, no hardcodeadas.
- Rate limiting en `/login` (anti fuerza bruta).
- Lockout tras intentos fallidos, logs de auditoría.

3.10 Logging seguro y auditoría

- Loguear intentos de login (éxito/fallo) con IP y timestamp, sin passwords.
- Nunca loguear tokens completos, connection strings ni datos personales.

4. Conocimientos previos necesarios

- Módulos 1, 3, 7 (HTTP, pipeline, JWT) y 6 (secretos) — la seguridad se apoya en todos ellos.

5. Cómo aparece dentro del proyecto

Archivo	Riesgo / control
<code>appsettings.json</code>	<code>SecretKey</code> y placeholder en repo (control: nada sensible, pero la clave JWT no debería estar)
<code>docs/REPORTE-USER-SECRETS.md</code>	decisión correcta de secretos (password fuera del repo)
<code>Services/AuthenticationService.cs</code>	credenciales hardcodeadas (<code>admin</code> / <code>1234</code>) — riesgo A07
<code>Controllers/AuthController.cs</code>	<code>/login</code> sin rate limiting, sin logs de auditoría
<code>DTO/User.cs</code>	<code>Role</code> lo envía el cliente — riesgo de escalada

Archivo	Riesgo / control
<code>Startup.cs</code>	CORS restrictivo (bien); <code>DeveloperExceptionPage</code> solo dev (bien); sin <code>UseExceptionHandler</code> en prod (mejora)
<code>appsettings.json</code> connection string	<code>Trust Server Certificate=true</code> (solo lab)
<code>Startup.cs</code> / EF Core	queries parametrizadas (seguro contra SQL Injection)
<code>.github/workflows/</code> (vacío)	sin gates de seguridad (dependabot, secret scanning)

6. Nivel de importancia

Importante — un endpoint mal protegido anula todo el resto.

7. Tiempo recomendado de estudio

12 horas.

8. Recursos recomendados

- **Documentación oficial:** OWASP Top 10 (owasp.org), OWASP API Security Top 10, Microsoft Learn "ASP.NET Core security topics".
- **Microsoft Learn:** "Introduction to web security" y "Secure ASP.NET Core web apps".
- **Libros:** *The Web Application Hacker's Handbook*; *OWASP Testing Guide*.
- **Videos:** "SQL Injection explained" (Hacksplained), "JWT Security" (varios); curso gratuito de Application Security en educative.io.
- **Cursos:** PortSwigger Web Security Academy (gratis, interactivo).
- **Herramientas:** OWASP ZAP, Burp Suite Community, `dotnet security-scan`, Dependabot.

9. Ejercicios sugeridos

1. (Fácil) Revisa el repo con `git grep -in "password\|secret"` y clasifica lo encontrado.
2. (Fácil) Verifica que `DELETE /api/libraries/1` y `GET books` no expongan datos por error (status codes correctos).
3. (Medio) Implementa hashing de passwords con BCrypt y muévelo a `AuthenticationService` (sin hardcode).
4. (Medio) Agrega `[Required]` y `[MaxLength(200)]` a `BookForm.Name` y verifica el `400`.
5. (Medio) Mueve el `SecretKey` de JWT a User Secrets/variables de entorno y prueba el login.
6. (Difícil) Agrega rate limiting en `/login` (paquete `AspNetCoreRateLimit` o middleware propio).
7. (Difícil) Simula SQL Injection contra un endpoint que uses SQL manual (crea uno con `FromSqlRaw` a propósito) y demuestra la diferencia con LINQ.
8. (Difícil) Implementa RS256 para el JWT y rota las claves; documenta el proceso.
9. (Difícil) Loguea auditoría de login (éxito/fallo, IP, timestamp) sin datos sensibles.
10. (Difícil) Configura Dependabot en el repo y revisa sus alertas de seguridad.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Credenciales en el código/repo	<code>git grep</code> encuentra passwords	User Secrets / env / vault + sanear historial
<code>SecretKey</code> JWT en <code>appsettings.json</code>	Se sube a GitHub (repo público)	Guardar en secretos; rotar inmediatamente si se expuso
CORS abierto a <code>*</code>	Cualquier origen puede llamar	Restringir orígenes conocidos
<code>DeveloperExceptionPage</code> en prod	Stack traces públicos	Solo en Development
Confiar en el <code>role</code> del body	Escalada de privilegios	Rol desde la fuente de identidad

Error	Cómo detectarlo	Cómo solucionarlo
SQL concatenado	SQL Injection	EF Core/LINQ o parámetros
Validar poco o tarde	Entrada basura en la BD	[Required] , [MaxLength] , etc. en el borde
Sin rate limiting en login	Fuerza bruta	Limitar intentos

MÓDULO 14 — Despliegue y CI/CD

1. Nombre del módulo

Entornos, configuración por entorno, variables de entorno, publicación, CI/CD y 12-factor

2. Objetivo de aprendizaje

El estudiante entenderá cómo llevar una API .NET de local a producción: diferencias entre entornos, cómo la configuración cambia por entorno, cómo publicar la app (`dotnet publish`), y qué es un pipeline de CI/CD (GitHub Actions / Azure Pipelines) — incluyendo por qué la configuración actual del proyecto **no** funciona en producción.

3. Conceptos fundamentales

3.1 Entornos (Development, Staging, Production)

- **Qué es:** `ASPNETCORE_ENVIRONMENT` selecciona el conjunto de configuraciones: `appsettings.{Environment}.json`, comportamientos (Swagger solo dev), secretos.
- **En el proyecto:** `Development` activa: `DeveloperExceptionPage`, Swagger, User Secrets, y los guards.
- **Problema real documentado:** User Secrets no se cargan en Production → la BD no conectaría si solo se usa User Secrets. La vía para prod son **variables de entorno** o el secret manager de la nube.

3.2 Variables de entorno como configuración

- **Qué es:** mecanismo del SO/plataforma para config: `ASPNETCORE_ENVIRONMENT`, `ConnectionStrings__DefaultConnection`, `JwtSettings__SecretKey`.
- **Por qué:** portable, no versionada, segura, por-entorno. Es la recomendación 12-factor.
- **Cómo funciona la precedencia:** `appsettings.json` → `appsettings.{env}.json` → User Secrets (solo dev) → **variables de entorno** → args.

3.3 Los 12 factores (12-factor app)

- **Qué es:** metodología para apps cloud-native: código único versionado, dependencias declaradas, **configuración por entorno**, servicios desechables, sin estado local, puertos expuestos, paralelismo, etc.
- **En el proyecto:** se respeta parcialmente (config externa para la password; código único; stateless). Faltan: secretos de producción, migraciones como paso de despliegue explícito, telemetría.

3.4 Publicación (.NET publish)

- **Qué es:** `dotnet publish -c Release` produce la carpeta `publish/` lista para ejecutar (dlls + archivos de config).
- **Modos:** framework-dependent (requiere runtime instalado) vs self-contained (incluye runtime). `--no-self-contained` (por defecto) es lo típico para contenedores.
- **En el proyecto:** no hay script de publish ni contenedor todavía; se ejecuta con `dotnet run` (dev).

3.5 Migraciones en despliegue

- **Qué es:** cómo se aplican los cambios de esquema en producción. Opciones: `dotnet ef database update` en el pipeline, `db.Database.Migrate()` al arrancar (lo que hace este proyecto) o scripts SQL revisados manualmente.
- **Riesgo de `Migrate()` al arrancar en producción:** si dos instancias arrancan a la vez → carrera; si la migración falla → la app no levanta. En proyectos serios se aplican las migraciones como paso explícito del pipeline.

3.6 CI (Integración Continua)

- **Qué es:** cada push a la rama dispara build + tests automáticos.

- **En el proyecto:** `.github/workflows/` está **vacío** → no hay CI. Esta es la mejora evidente: un workflow que haga `dotnet restore`, `dotnet build`, `dotnet test`.
- **Beneficios:** feedback temprano, gates de calidad, repetibilidad.

3.7 CD (Despliegue Continuo / Delivery)

- **Qué es:** pipeline que publica y despliega la app (a un servidor, contenedor, Azure, etc.).
- **Stages típicos:** Build → Test → Publish → Deploy.
- **En el proyecto:** no implementado; los secretos de producción irían en el pipeline (secrets de GitHub/ADO).

3.8 GitHub Actions (flujo concreto)

- **Qué es:** CI/CD en GitHub: archivos `.github/workflows/*.yaml` con jobs/steps.
- **Ejemplo mínimo para este proyecto:** job en `ubuntu-latest` con `actions/checkout`, `setup-dotnet` (versión 8.x), `dotnet restore`, `dotnet build -c Release`, `dotnet test` (los tests usan SQLite in-memory → corren sin credenciales).

3.9 Azure Pipelines (alternativa)

- **Qué es:** `azure-pipelines.yaml` con stages (build, test, deploy). Conectado a Boards/Repos del Módulo 11.

3.10 Contenedores y orquestación (intro)

- **Qué es:** empaquetar la app + runtime en una imagen (Docker) ejecutable en cualquier lado (Módulo 15 lo amplía).
- **Para qué sirve:** despliegues reproducibles, escalado, entornos idénticos.

4. Conocimientos previos necesarios

- Módulos 2, 3 y 6 (build, hosting, configuración, secretos).
- Módulo 10 (Git) para los triggers de CI.

5. Cómo aparece dentro del proyecto

Archivo	Concepto
<code>Properties/launchSettings.json</code>	<code>ASPNETCORE_ENVIRONMENT=Development</code>
<code>appsettings.Development.json</code>	config específica de dev (logging)
<code>appsettings.json</code>	config base con placeholder
<code>Program.cs</code>	<code>Host.CreateDefaultBuilder</code> (carga por entorno)
<code>.github/workflows/</code> (vacío)	sin CI/CD configurado
<code>docs/DOCUMENTACION-TECNICA.md</code>	"User Secrets no se cargan en Production → BD no conectaría"

6. Nivel de importancia

Complementario — este lab no despliega, pero cualquier API real necesita este conocimiento. Importante si quieres profesionalizarte.

7. Tiempo recomendado de estudio

10 horas.

8. Recursos recomendados

- **Documentación oficial:** [learn.microsoft.com](https://learn.microsoft.com/en-us/azure/devops/pipelines/) "Publish and deploy .NET apps", "Configure environments"; [docs.github.com](https://docs.github.com/en/actions/) "GitHub Actions".

- **Microsoft Learn:** "Automate workflows with GitHub Actions"; "Azure Pipelines" learning path.
- **Libros:** *12-Factor Apps* (12factor.net, leer gratis); *Continuous Delivery* (Humble & Farley).
- **Videos:** "GitHub Actions Tutorial" (freeCodeCamp); "Publish .NET to Azure App Service".
- **Cursos:** GitHub Actions "Hello World" (skills.github.com).
- **Herramientas:** GitHub CLI, Azure CLI, Docker Desktop.

9. Ejercicios sugeridos

1. (Fácil) Crea un workflow de GitHub Actions que corra `dotnet build` y `dotnet test` en cada push a `main`. Verifica en Actions.
2. (Fácil) Ejecuta `dotnet publish -c Release` y explica qué hay en `publish/`.
3. (Medio) Haz que el workflow pase la connection string real como variable de entorno/secreto para una ejecución de smoke test.
4. (Medio) Implementa un segundo job que publique el artefacto (zip) del build.
5. (Difícil) Despliega la app en Azure App Service (o Render/Railway) usando variables de entorno para BD y JWT; configura la BD de prod por separado.
6. (Difícil) Mueve las migraciones al pipeline (ejecutar `dotnet ef database update` como paso de despliegue) en lugar de `Migrate()` al arrancar.
7. (Difícil) Agrega un stage de deploy a un contenedor (Docker + registry) — preparación para Módulo 15.
8. (Difícil) Escribe un documento de "promoción de entornos": cómo cambia cada configuración de Dev → Staging → Prod.

10. Errores comunes

Error	Cómo detectarlo	Cómo solucionarlo
Depender de User Secrets en prod	BD que no conecta	Variables de entorno / vault en cada entorno
Migraciones "al arrancar" en cluster	Carreras entre instancias	Migraciones como paso del pipeline
No testear en CI	Tests que solo pasan localmente	CI con <code>dotnet test</code> desde el primer push
Subir secretos a los logs del pipeline	Passwords visibles	Secrets del pipeline, nunca en <code>echo</code>
Publicar con credenciales embebidas	<code>appsettings.json</code> con password	Placeholder + env vars
Swagger abierto en prod	Info-leak de endpoints	Activar solo en dev/staging

MÓDULO 15 — Temas avanzados

1. Nombre del módulo

CQRS, Vertical Slice, MediatR, caching, Docker, Redis, observabilidad, arquitectura hexagonal y microservicios

2. Objetivo de aprendizaje

El estudiante conocerá el "siguiente nivel" de arquitectura y operación: hacia dónde evolucionaría este proyecto si creciera. No se implementan aquí, pero se explican lo suficiente para decidir cuándo aplicarlos y reconocerlos en el código ajeno.

3. Conceptos fundamentales

3.1 CQRS (Command Query Responsibility Segregation)

- **Qué es:** separar las operaciones de **lectura** (queries) de las de **escritura** (commands). Lecturas con modelos optimizados; escrituras con reglas de dominio.
- **En este proyecto:** hoy los services hacen GET y POST/DELETE sobre el mismo modelo. Con CQRS, `GetBooks` usaría un read-model y `AddBook` un command.
- **Cuándo usarlo:** cuando la lectura y la escritura tienen necesidades muy distintas (volumen, optimización, complejidad de dominio). Para un CRUD pequeño es sobre-ingeniería.

3.2 Vertical Slice Architecture

- **Qué es:** organizar el código por *funcionalidad* (un slice por feature con su propio request/handler/response), en lugar de por capas técnicas (Módulo 4).
- **Beneficios:** cambios locales, alineación con CQRS/MediatR, menos "búsqueda entre capas".
- **En este proyecto:** la evolución natural sería crear slices `Books.AddBook`, `Libraries.DeleteLibrary`, etc., cada una con su handler y DTO.

3.3 MediatR

- **Qué es:** librería que implementa el patrón **mediator** (objetos intermedios). Dispatcheas un `IRequest<T>` y un `IRequestHandler<TRequest, TResponse>` lo procesa.
- **Para qué sirve:** desacoplar controllers de services; pieza clave de CQRS/Vertical Slice.
- **En este proyecto:** `BooksController` llamaría `await _mediator.Send(new AddBookCommand(...))` en vez de `_booksService.Add(...)`.

3.4 Caching

- **Qué es:** guardar respuestas/datos costosos en memoria o en caché distribuida para no recalcular.
- **Tipos:** in-memory (`IMemoryCache`), distribuido (Redis), HTTP caching (headers), response caching.
- **Cuándo:** datos que cambian poco y se leen mucho (ej. catálogo de librerías).
- **Riesgos:** datos obsoletos (invalida caché), inconsistencia tras escrituras.

3.5 Docker

- **Qué es:** contenedores — empaquetan la app + runtime + config en una imagen reproducible.
- **En este proyecto:** un `Dockerfile` con base `mcr.microsoft.com/dotnet/sdk:8.0` (build) y `mcr.microsoft.com/dotnet/aspnet:8.0` (runtime); el `docker-compose.yml` podría levantar API + Postgres local.
- **Beneficio:** "funciona en mi máquina" se convierte en "funciona en cualquier máquina" — y habilita Testcontainers (Módulo 9).

3.6 Redis

- **Qué es:** almacén de datos en memoria (key-value) usado como caché distribuida, cola de mensajes, sesiones, rate limiting.

- **Para qué sirve:** caché compartida entre instancias; supera la caché in-memory cuando la app escala horizontalmente.

3.7 Observabilidad: logs, métricas, trazas (OpenTelemetry)

- **Qué es:** la capacidad de entender el estado del sistema por sus *señales*: logs estructurados, métricas (cuántas requests, latencia, errores) y trazas distribuidas (el recorrido de un request entre servicios).
- **OpenTelemetry:** estándar abierto que instrumenta tu app y exporta a backends (Jaeger, Prometheus, Grafana, Azure Monitor).
- **En este proyecto:** hoy no hay telemetría; los logs van a consola. El siguiente paso sería `ILogger` + OpenTelemetry.

3.8 Arquitectura Hexagonal (Puertos y Adaptadores)

- **Qué es:** el dominio está en el centro; "puertos" (interfaces) y "adaptadores" (implementaciones) conectan el mundo exterior (BD, HTTP, colas). Dependencias hacia adentro.
- **Diferencia con Clean Architecture:** es el mismo espíritu con énfasis en los *adaptadores* (drivers/driven).
- **En este proyecto:** el adaptador de persistencia sería Npgsql; el puerto, `ILibraryRepository` o el propio `LibraryContext` detrás de una interfaz.

3.9 Microservicios

- **Qué es:** partir el sistema en servicios pequeños, despleables y escalables independientemente, cada uno con su BD.
- **En contraste:** este proyecto es un **monolito** (una API + una BD) — correcto para su tamaño.
- **Tradeoffs:** microservicios = operación compleja (red, consistencia distribuida, observabilidad, despliegues); monolito modular es el punto intermedio recomendado.
- **Regla práctica:** empieza monolito; divide cuando haya una razón de negocio/escala concreta.

3.10 Patrón Gateway / BFF (contexto)

- Si hubiera microservicios, un API Gateway centraliza auth, routing y rate limiting; un BFF (backend-for-frontend) adapta respuestas por cliente. Útil para el panorama.

3.11 Contratos y versionado de APIs

- **Qué es:** versionar el API (`v1` , `v2`) para no romper consumidores cuando cambia el contrato.
- **En este proyecto:** Swagger doc es `v1` ; la ruta no tiene `/v1` en controllers (solo en la doc).

4. Conocimientos previos necesarios

- Todos los módulos anteriores (son los cimientos que estos patrones extienden).

5. Cómo aparece dentro del proyecto

Elemento	Relación
<code>Services/*.cs</code>	candidatos a convertirse en handlers MediatR (Vertical Slice)
<code>Controllers/*.cs</code>	dónde entraría <code>IMediator.Send(...)</code>
<code>Startup.cs</code>	dónde se registraría MediatR, Redis, OpenTelemetry
Swagger <code>v1</code>	concepto de versionado
<code>.github/workflows/</code> vacío	punto de partida para CI/CD y despliegue por contenedores
<code>docs/DOCUMENTACION-TECNICA.md</code>	la sección "Observaciones y riesgos" es la semilla de mejoras operativas

6. Nivel de importancia

Avanzado — no se necesitan para el lab; se necesitan para el siguiente nivel profesional.

7. Tiempo recomendado de estudio

15 horas.

8. Recursos recomendados

- **Documentación oficial:** learn.microsoft.com "CQRS pattern", "MediatR" (GitHub [jbogard/MediatR](https://github.com/jbogard/MediatR)), docs.docker.com, redis.io, opentelemetry.io.
- **Libros:** *Designing Data-Intensive Applications* (Kleppmann); *Building Microservices* (Sam Newman); *Patterns, Principles and Practices of Domain-Driven Design* (Millet & Tune).
- **Videos:** "CQRS in practice" y "Vertical Slice Architecture" (Jimmy Bogard, NDC); "What is Docker?" (TechWorld with Nana); "Microservices" (Martin Fowler talks).
- **Cursos:** Pluralsight "Building Distributed Applications with .NET"; Docker "Getting Started" (docs).
- **Repositorios:** [jbogard/ContosoUniversity](https://github.com/jbogard/ContosoUniversity) (CQRS+Vertical Slice+MediatR), [dotnet-architecture/eShopOnContainers](https://github.com/dotnet-architecture/eShopOnContainers) (microservicios), [testcontainers/testcontainers-dotnet](https://github.com/testcontainers/testcontainers-dotnet).

9. Ejercicios sugeridos

1. (Fácil) Refactoriza `BooksController.Add` para usar MediatR (`AddBookCommand` + handler). Mantén los tests en verde.
2. (Medio) Extrae un read-model para `GetBooks` (CQRS de lectura) y compara con el query actual.
3. (Medio) Agrega `IMemoryCache` para el catálogo de librerías con invalidation al hacer PUT/DELETE.
4. (Medio) Crea un `Dockerfile` multi-stage para la API y una imagen que corra los tests.
5. (Difícil) Levanta Redis en Docker y usa `IDistributedCache` con un serializador JSON para las librerías.
6. (Difícil) Instrumenta con OpenTelemetry (traces + métricas) y exporta a Jaeger en Docker; observa los spans de un request.
7. (Difícil) Usa Testcontainers para correr los tests de integración contra PostgreSQL real en Docker.
8. (Difícil) Diseña (en papel) la división de este monolito en 2 microservicios y argumenta si vale la pena.
9. (Difícil) Versiona la API a `/v2` con un cambio rompedor y mantén `/v1` funcionando.

10. Errores comunes

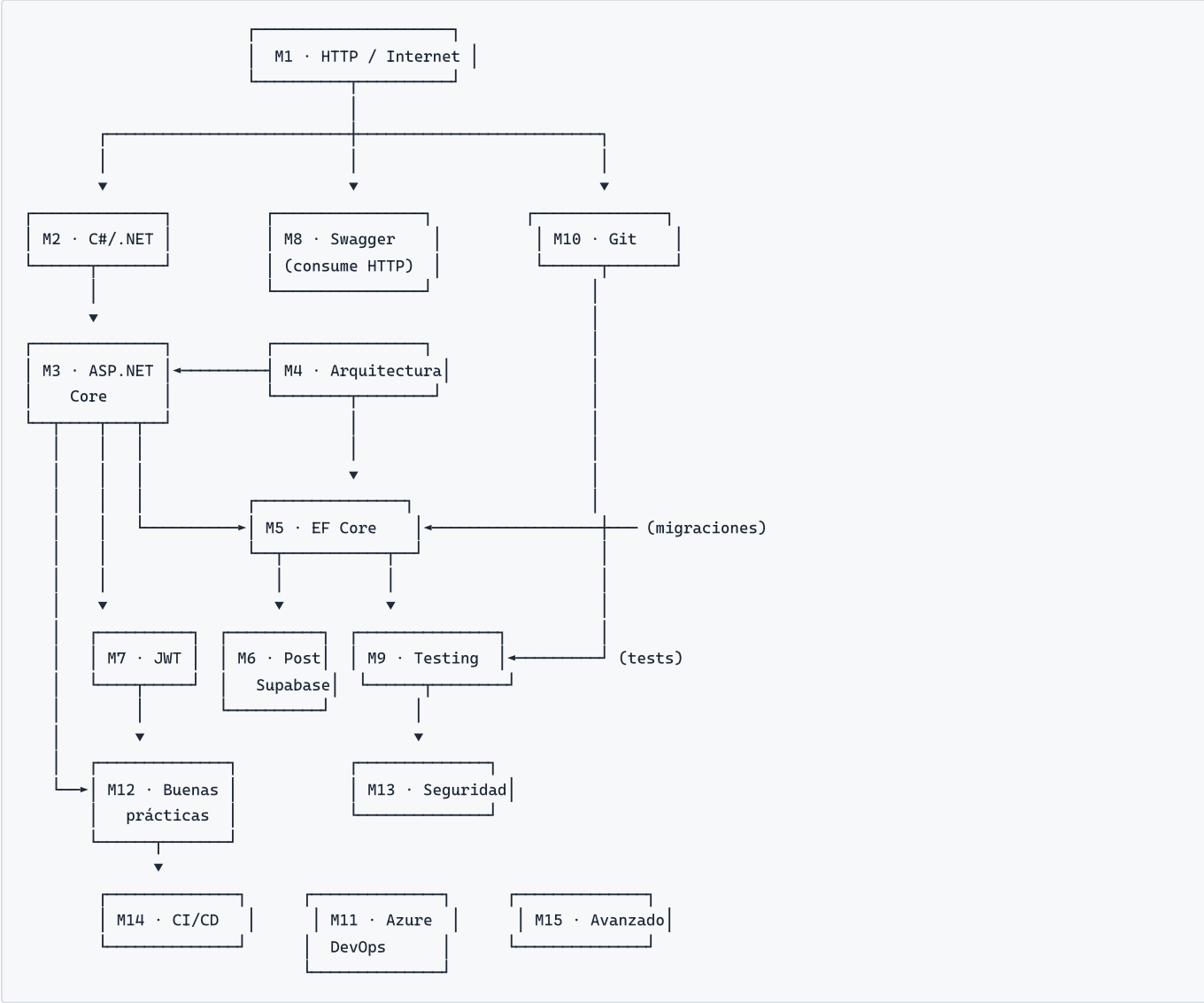
Error	Cómo detectarlo	Cómo solucionarlo
CQRS/MediatR en un CRUD trivial	Cientos de handlers para 3 endpoints	Aplicar solo con complejidad real
Microservicios por moda	Un "monolito distribuido"	Empezar monolito; dividir con razón de negocio
Caché sin invalidación	Datos obsoletos	Invaldar/expirar explícitamente
Docker sin multi-stage	Imágenes gigantes con SDK	Build en sdk, runtime en aspnet
Observabilidad "a lo bruto"	Sin trazas ni métricas útiles	OpenTelemetry con contexto
Vertices de rendimiento sin medición	"Optimizaciones" sin base	Medir antes de optimizar

MAPA CONCEPTUAL

Cómo leer este mapa

Cada nodo es un módulo/concepto. **A → B** significa "B depende de / se apoya en A". Las flechas indican el orden mental correcto: no estudies el destino antes que el origen.

Vista general (las 5 grandes familias)



Dependencias detalladas por módulo

Módulo	Depende de	Alimenta a
M1 · HTTP	— (base)	M3, M7, M8, M9
M2 · C#/.NET	— (base)	M3, M4, M5, M7, M9, M12
M3 · ASP.NET Core	M1, M2	M4, M7, M9, M12, M14
M4 · Arquitectura	M2, M3	M12, M15
M5 · EF Core	M2, M3, M6 (parcial)	M9, M12, M14, M15

Módulo	Depende de	Alimenta a
M6 · PostgreSQL/Supabase	M2, M3, M5	M14, M13
M7 · JWT	M1, M3	M9, M13, M15
M8 · Swagger	M1, M3, M7	M9 (smoke tests manuales)
M9 · Testing	M1, M2, M3, M5	M14 (CI), M12
M10 · Git	— (paralelo)	M14 (CI), M11
M11 · Azure DevOps	M10	M14 (pipelines)
M12 · Buenas prácticas	M2, M3, M4	M15 (calidad)
M13 · Seguridad	M1, M3, M6, M7	M14, M15
M14 · Despliegue	M2, M3, M6, M10, M12	M15
M15 · Avanzado	Todos	— (expansión)

Mapa de archivos → módulos (cómo encaja cada archivo del proyecto)

HackerRank1.sln	
└─ HackerRank1/	M2, M3, M4
└─ Program.cs	M3 (hosting legacy), M6 (User Secrets)
└─ Startup.cs	M3 (DI/pipeline), M5 (DbContext/Migrate), M7 (JWT), M8 (Swagger), M13 (CORS)
└─ appsettings.json	M6, M7, M13
└─ appsettings.Development.json	M3 (entornos), M14
└─ HackerRank1.csproj	M2 (NuGet/versiones), M6 (UserSecretsId)
└─ Properties/launchSettings.json	M3 (entornos/puertos), M6
└─ Controllers/	M1, M3, M4
└─ AuthController.cs	M7
└─ LibrariesController.cs	M1, M3, M4
└─ BooksController.cs	M1, M3, M4, M12
└─ Services/	M3, M4, M5, M7
└─ LibraryService.cs	M5 (LINQ/ChangeTracker), M4
└─ BookService.cs	M5
└─ AuthenticationService.cs	M7 (hardcoded), M13 (riesgo)
└─ Data/LibraryContext.cs	M5
└─ DTO/	M4, M12
└─ Entities/JwtSettings.cs	M7
└─ Helpers/TokenGenerator.cs	M7 (HS256)
└─ Migrations/	M5, M6
└─ 20260528004745_InitialCreate.cs	M5 (FK CASCADE), M6
└─ LibraryService.Integration.Test/	M9
└─ IntegrationTest.cs	M9 (WebApplicationFactory, SQLite in-memory)
└─ Extensions/HttpResponseExtensions.cs	M9
docs/	M10 (commit docs), M11 (proceso)
.github/workflows/ (vacío)	M14 (CI pendiente)

RUTA DE APRENDIZAJE

Orden exacto recomendado (215 h)

La ruta intercala teoría (T), práctica con el proyecto real (P) y proyecto propio (PP = "Librería Plus", ver sección de proyectos prácticos). El orden minimiza fricción: cada paso construye sobre el anterior.

Paso	Módulo	Horas	Qué haces
1	M1 · HTTP	8	Teoría de HTTP + probar endpoints del proyecto en Swagger
2	M10 · Git	10	Clonar, historial, ramas, Conventional Commits (puede correr en paralelo)
3	M2 · C#/.NET	25	Lenguaje y plataforma; leer todo el código del proyecto en C#
4	M3 · ASP.NET Core	20	Explicar <code>Program.cs</code> / <code>Startup.cs</code> línea por línea
5	M4 · Arquitectura	12	Analizar capas, justificar el NO-Repository del proyecto
6	M5 · EF Core	30	Entidades, LINQ, migraciones, ChangeTracker; aplicar <code>InitialCreate</code>
7	M6 · PostgreSQL/Supabase	12	Connection strings, User Secrets; arrancar la API contra Supabase
8	M7 · JWT	15	<code>/login</code> , decodificar token, validación Bearer
9	M8 · Swagger	6	Probar toda la API desde Swagger/Postman
10	M9 · Testing	20	Correr/ampliar los 3 tests; entender SQLite in-memory
11	M12 · Buenas prácticas	15	Auditar smells del proyecto y refactorizar
12	M13 · Seguridad	12	Endurecer secretos, JWT y validaciones del proyecto
13	M11 · Azure DevOps	8	Modelar el backlog del lab en Boards
14	M14 · Despliegue	10	CI con GitHub Actions; publish; entornos
15	M15 · Avanzado	12	MediatR/Vertical Slice sobre el proyecto; Docker; OTel
—	Evaluación final	4	Examen universitario completo (sección final)

Reglas de la ruta: 1. No pases al paso n+1 sin resolver los ejercicios marcados como "Medio" del paso n. 2. Cada 3 pasos, vuelve al proyecto real y **reescribelo de memoria** (sin mirar): esa es la prueba real de dominio. 3. Los pasos 2 y 6-10 son los más profundos: no los saltes "porque ya sé algo".

CHECKLIST DE DOMINIO

Marca cada ítem: ☐ No estudiado → ☐ En progreso → ☒ Dominado .

Módulo 1 — HTTP

- ☐ Entiendo request/response y sus partes (línea, headers, body)
- ☐ Domino métodos HTTP y su semántica (idempotencia)
- ☐ Reconozco códigos 2xx/4xx/5xx y sé cuándo usar 200/201/204/400/401/404
- ☐ Explico stateless y por qué se usa JWT
- ☐ Leo JSON y sé qué es serialización/deserialización
- ☐ Explico HTTPS/TLS y sus opciones en la connection string
- ☐ Uso cURL/Postman/Swagger para probar endpoints

Módulo 2 — C#/.NET

- ☐ Distingo SDK/runtime/CLR y sé qué es un ensamblado
- ☐ Uso `dotnet` CLI (new/build/run/test/restore/add)
- ☐ Conozco .sln, .csproj, NuGet y dependencias transitivas
- ☐ Explico clases, interfaces, records y genéricos
- ☐ Escribo LINQ (Where/Select/Any/Contains/FirstOrDefault/ToListAsync)
- ☐ Uso async/await sin bloquear (evito `.Result`)
- ☐ Explico nullable reference types y `??` / `?.`
- ☐ Manejo excepciones (throw, try/catch) y sé cuándo usarlas

Módulo 3 — ASP.NET Core

- ☐ Diferencio hosting moderno vs legacy (Program + Startup)
- ☐ Explico los 8 pasos del pipeline del proyecto en orden
- ☐ Registro servicios con Transient/Scoped/Singleton y conozco las reglas de lifetime
- ☐ Uso IConfiguration y entornos (Development/Production)
- ☐ Entiendo routing y `[Route]` / `[HttpGet]` / `[ApiController]`
- ☐ Devuelvo Ok/NotFound/NoContent/Unauthorized/CreatedAtAction correctamente
- ☐ Explico el guard de idempotencia de `ConfigureServices`

Módulo 4 — Arquitectura

- ☐ Dibujo el flujo Controller → Service → DbContext → BD
- ☐ Comparo capas, Clean, Hexagonal y Vertical Slice
- ☐ Justifico por qué este proyecto NO usa Repository Pattern
- ☐ Defino tradeoffs de una decisión arquitectónica
- ☐ Explico DTOs y su rol en el contrato de la API

Módulo 5 — EF Core

- ☐ Explico ORM, DbContext como Unit of Work + Change Tracker
- ☐ Diferencio IQueryable vs IEnumerable (ejecución diferida)
- ☐ Uso AsNoTracking, Include y conozco el problema N+1
- ☐ Explico EntityState (Added/Modified/Deleted/Detached)
- ☐ Creo y aplico migraciones (`dotnet ef migrations add/update/list`)

- ☐ Explico EnsureCreated vs Migrate y los guards del proyecto
- ☐ Sé cómo el provider Npgsql configura PostgreSQL (cascade, identity)

Módulo 6 — PostgreSQL/Supabase

- ☐ Escribo connection strings Npgsql válidas (separador `;`)
- ☐ Explico SSL Mode, Trust Server Certificate y Pooling
- ☐ Configuro User Secrets y explico sus 5 piezas
- ☐ Explico por qué los secretos no suben a git
- ☐ Entiendo la precedencia de configuración (el último gana)
- ☐ Sé qué falla en Production si solo uso User Secrets

Módulo 7 — JWT

- ☐ Explico autenticación vs autorización
- ☐ Decodifico un JWT y distingo header/payload/signature
- ☐ Explico HS256, HMAC y claves simétricas vs asimétricas
- ☐ Entiendo claims, issuer, audience, lifetime y ClockSkew
- ☐ Configuro AddJwtBearer + TokenValidationParameters
- ☐ Uso `[Authorize]`, `[AllowAnonymous]` y roles
- ☐ Explico los riesgos (secretos en payload, clave débil, role del cliente)

Módulo 8 — Swagger

- ☐ Explico OpenAPI vs Swagger UI vs Swashbuckle
- ☐ Configuro Swagger solo en Development
- ☐ Uso Swagger con autorización Bearer
- ☐ Genero/leo `swagger.json`

Módulo 9 — Testing

- ☐ Explico la pirámide de testing
- ☐ Escribo tests xUnit (`[Fact]`, `[Theory]`, fixtures)
- ☐ Uso FluentAssertions
- ☐ Explico WebApplicationFactory y TestServer
- ☐ Sustituyo el DbContext por SQLite in-memory vía DI
- ☐ Explico por qué SQLite in-memory y no EF InMemory para integración
- ☐ Aplico AAA y buenas prácticas de nombrado
- ☐ Entiendo cuándo mockear (Moq) y cuándo no

Módulo 10 — Git/GitHub

- ☐ Uso add/commit/status/diff/log
- ☐ Escribo Conventional Commits
- ☐ Creo ramas, PRs y resuelvo conflictos
- ☐ Uso .gitignore correctamente
- ☐ Verifico con `git grep` que no hay secretos en el repo

Módulo 11 — Azure DevOps

- ☐ Distingo Epic/Feature/PBI/Task/Bug

- ☐ Modelo un backlog con prioridad y estimación
- ☐ Defino Acceptance Criteria y DoD
- ☐ Planifico un sprint y entiendo burndown

Módulo 12 — Buenas prácticas

- ☐ Explico y aplico los 5 principios SOLID
- ☐ Escribo código con nombres claros y funciones pequeñas
- ☐ Aplico async/await de forma correcta en toda la cadena
- ☐ Uso DTOs y validación en el borde
- ☐ Manejo errores esperados (HTTP) e inesperados (excepciones + logging)
- ☐ Logueo con `ILogger<T>` sin exponer secretos

Módulo 13 — Seguridad

- ☐ Enumero riesgos del OWASP Top 10 aplicados al proyecto
- ☐ Explico por qué EF Core previene SQL Injection
- ☐ Gestiono secretos (User Secrets, env vars, vault)
- ☐ Endurezco JWT (clave fuerte, expiración, RS256, role del servidor)
- ☐ Configuro CORS restrictivo
- ☐ Valido entradas y nunca confío en el cliente

Módulo 14 — Despliegue

- ☐ Explico entornos y precedencia de configuración
- ☐ Publico con `dotnet publish`
- ☐ Creo un workflow de GitHub Actions (build + test)
- ☐ Conozco por qué `Migrate()` al arrancar es riesgoso en producción
- ☐ Explico los 12 factores aplicados a este proyecto

Módulo 15 — Avanzado

- ☐ Explico CQRS y Vertical Slice
 - ☐ Refactorizo un controller a MediatR
 - ☐ Uso IMemoryCache/IDistributedCache
 - ☐ Creo un Dockerfile multi-stage y docker-compose
 - ☐ Explico OpenTelemetry (logs, métricas, trazas)
 - ☐ Comparo monolito vs microservicios y justifico la elección
-

AUTOEVALUACIÓN POR MÓDULO

Reglas: responde **sin** mirar el código. Marca tus respuestas y verifica contra el proyecto al final. Cada módulo tiene ≥ 10 preguntas.

Módulo 1 — HTTP (autoevaluación)

1. ¿Cuáles son las 4 partes de un request HTTP? ¿Y de un response?
2. ¿Qué diferencia hay entre `GET /api/libraries` y `GET /api/libraries/5`?
3. ¿Qué status code devuelve el proyecto en: (a) POST book a librería existente, (b) POST book a librería inexistente, (c) DELETE librería existente, (d) DELETE librería inexistente, (e) login sin `role`?
4. ¿Qué significa que HTTP es stateless y qué problema genera para la autenticación?
5. ¿Qué contiene el header `Authorization: Bearer <token>` y en qué endpoint del proyecto se usa?
6. ¿Por qué `DELETE` es idempotente y `POST` no? Demuéstralo con los endpoints del proyecto.
7. ¿Qué diferencia hay entre el `404` de "librería inexistente" y el `401` de "sin token"?
8. ¿Qué papel juega el `Content-Type: application/json` en un POST?
9. ¿Qué es una URL? Descompón `https://localhost:7098/api/libraries/1/books?x=1`.
10. ¿Por qué el JSON `{ "id": 3 }` usa camelCase en esta API?
11. ¿Qué headers genera la política CORS "Frontend" y para qué origen?
12. ¿Qué implican `SSL Mode=Require` y `Trust Server Certificate=true` en la connection string?

Módulo 2 — C#/.NET (autoevaluación)

1. ¿Qué diferencia hay entre SDK, runtime y CLR? ¿Cuál instalarías en una máquina de build y cuál en producción?
2. ¿Qué es un ensamblado y cómo se relaciona con un proyecto y una solución?
3. Escribe el comando para (a) compilar, (b) ejecutar el proyecto HackerRank1, (c) ejecutar tests, (d) listar migraciones.
4. ¿Por qué `dotnet run` falla en la raíz sin `--project`?
5. ¿Qué diferencia hay entre `class` y `record`? ¿Dónde hay un record en el proyecto?
6. ¿Qué significa `string? Category` y qué pasaría si fuera `string Category` con `[ApiController]`?
7. ¿Qué es un método async? ¿Por qué el test con `.Result` es una mala práctica?
8. ¿Qué diferencia hay entre `IEnumerable`, `IQueryable` y `List`? ¿Dónde aparece cada uno en el proyecto?
9. ¿Qué es un delegate y una lambda? Pon un ejemplo del `Startup.cs`.
10. ¿Qué hace `?? throw new InvalidOperationException("Invalid JWT Settings")` y por qué es bueno que la app falle al arrancar?
11. ¿Qué es una interfaz? Nombra 3 del proyecto y su implementación.
12. ¿Qué son los atributos (`[Key]`, `[JsonPropertyName]`) y en qué se diferencian de los comments?

Módulo 3 — ASP.NET Core (autoevaluación)

1. Explica qué hace cada línea de `Program.cs`.
2. ¿Cuáles son las 8 etapas del pipeline en `Startup.Configure` y en qué orden van?
3. ¿Por qué `UseAuthentication` va antes de `UseAuthorization`?
4. ¿Qué diferencia hay entre `ConfigureServices` y `Configure`? ¿Cuál registra y cuál usa el contenedor?
5. ¿Qué son Transient, Scoped y Singleton? ¿Qué lifetime tiene cada servicio del proyecto?
6. ¿Por qué existe el guard `if (services.Any(d => d.ServiceType == typeof(JwtSettings))) return;`?
7. ¿Qué proveedores de configuración se cargan y en qué orden? ¿Quién gana si dos definen lo mismo?
8. ¿Qué hace `env.IsDevelopment()` en `Configure`? ¿Qué cambia entre Development y Production?
9. ¿Qué es el model binding? Pon 3 ejemplos de cómo se llenan los parámetros en los controllers.
10. ¿Qué hace `[ApiController]`? ¿Cómo se relaciona con el error "The Category field is required."?
11. ¿Qué devuelve `Ok()`, `NotFound()`, `NoContent()`, `Unauthorized()`, `CreatedAtAction(...)`? ¿Qué código HTTP produce cada una?

12. ¿Qué hace `AddDbContextPool` ? ¿Por qué `LibraryContext` es Scoped y no Singleton?

Módulo 4 — Arquitectura (autoevaluación)

1. Dibuja el flujo completo de una petición HTTP en este proyecto (sin mirar).
2. ¿Qué responsabilidades tiene cada capa (Controller, Service, DbContext)?
3. ¿Por qué este proyecto NO usa Repository Pattern? Argumenta con el `DbContext` en mente.
4. ¿Cuándo SÍ usarías Repository? Da 2 escenarios concretos.
5. ¿Qué es Clean Architecture y en qué se diferencia de la capas clásicas?
6. ¿Qué es Vertical Slice? ¿Cómo se vería la feature "crear libro" en ese estilo?
7. ¿Qué es un DTO? ¿Por qué `BookForm` existe y `Library` no es un DTO?
8. ¿Qué es la Inversión de Dependencias (DIP)? ¿Dónde la ves en los controllers?
9. ¿Qué es acoplamiento y cohesión? Evalúa el proyecto en ambos ejes.
10. ¿Qué tradeoff tiene la decisión de no separar el proyecto en Domain/Application/Infrastructure?
11. ¿Qué es una entidad "anémica"? ¿Este proyecto tiene dominio anémico? ¿Está mal?
12. ¿Qué cambiarías de la arquitectura si la API creciera a 50 endpoints?

Módulo 5 — EF Core (autoevaluación)

1. ¿Qué es un ORM y qué es un provider? ¿Cuál es el provider de este proyecto?
2. ¿Qué es el DbContext? ¿Por qué se dice que es Unit of Work + Repository?
3. ¿Qué diferencia hay entre `IQueryable` y ejecución? Explica el `if (ids != null && ids.Any())` de `LibraryService.Get`.
4. ¿Qué SQL genera `_libraryContext.Books.Where(b => b.LibraryId == id).ToListAsync()` ?
5. Explica los estados `Added/Modified/Deleted/Detached/Unchanged` con un ejemplo de cada uno en el proyecto.
6. ¿Por qué en `LibrariesService.Update` el `.Update()` es redundante?
7. ¿Qué es `AsNoTracking` y cuándo usarlo? ¿El proyecto lo usa?
8. ¿Qué diferencia hay entre `EnsureCreated()` y `Migrate()` ? ¿Por qué los tests usan uno y la app el otro?
9. ¿Qué contiene una migración? Explica la migración `InitialCreate` (tablas, FK, cascade, índice, identity).
10. ¿Qué es el problema N+1? ¿Cómo se evitaría en este proyecto?
11. ¿Qué comando usas para crear/aplicar/listar migraciones? ¿Qué herramienta global requiere?
12. ¿Cómo evita EF Core la SQL Injection?

Módulo 6 — PostgreSQL/Supabase (autoevaluación)

1. Escribe una connection string Npgsql válida y explica cada opción.
2. ¿Por qué la password no puede subir a git? ¿Qué mecanismo se usa en este proyecto?
3. Nombra las 5 piezas de User Secrets y explica cada una con el archivo real.
4. ¿Qué lee `GetConnectionString("DefaultConnection")` y cómo llega a su valor final (precedencia)?
5. ¿Por qué el separador de opciones de Npgsql debe ser `;` y no `,` ?
6. ¿Qué es el pooler de Supabase y qué diferencia hay con el endpoint directo?
7. ¿Qué es el pooling de conexiones? ¿Qué opciones controlan el pool del driver y el de EF?
8. ¿Qué significa `ON DELETE CASCADE` en la FK `Books.LibraryId` ?
9. ¿Por qué `dotnet ef migrations list` es una buena prueba de conectividad?
10. ¿Qué falla si despliegas la API a Production manteniendo la config actual? ¿Por qué?
11. ¿Cómo se mapea la variable de entorno `ConnectionStrings__DefaultConnection` en IConfiguration?
12. ¿Qué tabla/columna de la BD es NOT NULL y por qué obligó al default `string.Empty` ?

Módulo 7 — JWT (autoevaluación)

1. ¿Qué diferencia hay entre autenticación y autorización? Pon cada una en el pipeline del proyecto.

2. ¿Cuáles son las 3 partes de un JWT y cómo se codifican?
3. ¿Por qué es incorrecto decir "el JWT está cifrado"? ¿Qué garantiza la firma?
4. ¿Qué es HMAC-SHA256 y por qué la clave se llama simétrica?
5. ¿Qué claims contiene el token del proyecto y de dónde salen?
6. ¿Qué valida el middleware JWT con `TokenValidationParameters` ? ¿Qué pasa si el token está vencido?
7. ¿Qué es issuer, audience y ClockSkew? ¿Qué valores usa el proyecto?
8. ¿Por qué `/login` tiene `[AllowAnonymous]` ?
9. ¿Qué riesgo tiene que `role` llegue en el body del login?
10. ¿Qué pasaría si alguien modifica el payload y re-firma con la clave equivocada?
11. ¿Cómo se "consume" el token desde un cliente? ¿Qué header y qué esquema?
12. ¿Qué cambiarías para usar RS256 en vez de HS256?

Módulo 8 — Swagger (autoevaluación)

1. ¿Qué es OpenAPI y qué es Swagger UI? ¿Quién los conecta?
2. ¿Dónde se sirve el `swagger.json` y cómo se configura en el proyecto?
3. ¿Por qué Swagger solo aparece en Development?
4. ¿Cómo pruebas un endpoint `[Authorize]` desde Swagger?
5. ¿De dónde saca Swashbuckle los esquemas de los DTOs?
6. ¿Qué ocurre si un endpoint no tiene `[HttpX]` ? ¿Aparece en Swagger?
7. ¿Qué es `OpenApiInfo` y qué título/versión define el proyecto?
8. ¿Cómo se relaciona el OpenAPI con los tests de integración (contrato)?
9. ¿Qué es editor.swagger.io y para qué sirve con el `swagger.json` descargado?
10. ¿Qué riesgo existe en exponer Swagger en producción?

Módulo 9 — Testing (autoevaluación)

1. ¿Qué es la pirámide de testing y en qué nivel están los 3 tests del proyecto?
2. ¿Qué hace `IClassFixture<WebApplicationFactory<Program>>` ?
3. ¿Cómo se sustituye el `LibraryContext` real por SQLite? ¿Qué dos llamadas clave se hacen?
4. ¿Por qué SQLite in-memory y no EF InMemory para estos tests?
5. ¿Qué es `EnsureCreated` aquí y por qué no choca con `Migrate()` de `Startup` ?
6. Explica el patrón AAA en `TestAddBook_Ok_GetBook_NotFound` .
7. ¿Qué verifica exactamente `TestDeleteLibrary` (paso a paso)?
8. ¿Qué es FluentAssertions y qué ventaja tiene sobre `Assert.Equal` ?
9. ¿Qué es un test double? ¿Por qué este proyecto prefiere BD real (SQLite) a mocks de DbSet?
10. ¿Cuándo usarías Moq? Diseña el setup de un `Mock<ILibrariesService>` .
11. ¿Qué es el warning xUnit1031 y cómo se corrige?
12. ¿Qué papel juegan los tests como "contrato ejecutable" de la API?

Módulo 10 — Git/GitHub (autoevaluación)

1. ¿Qué diferencia hay entre working tree, staging y commit?
2. ¿Qué mensajes de commit usa este repo? Clasifícalos por tipo Conventional Commit.
3. ¿Qué hace `.gitignore` ? Nombra 5 entradas del repo y explica por qué están.
4. ¿Por qué `.gitignore` no protege de un secreto ya commiteado? ¿Cómo se sanearía?
5. ¿Qué es `origin/main` ? ¿Qué hace `git push origin HEAD` ?
6. ¿Qué es una PR y por qué es útil incluso trabajando solo?
7. ¿Cómo se resuelve un conflicto de merge?

8. ¿Qué diferencia hay entre merge y rebase? ¿Cuándo usarías cada uno?
9. ¿Qué comando te muestra el historial en una línea? ¿Y el estado?
10. ¿Qué es `git reflow` y para qué sirve?
11. ¿Cómo verificas que un repo no contiene secretos?
12. ¿Qué es una rama? ¿Por qué no se trabaja directamente en `main` ?

Módulo 11 — Azure DevOps (autoevaluación)

1. ¿Qué es un Work Item? Nombra 5 tipos y su jerarquía.
2. ¿Qué diferencia hay entre PBI y Task? Pon 2 ejemplos del lab en cada tipo.
3. ¿Qué es el backlog y cómo se prioriza?
4. ¿Qué es un Sprint? ¿Qué ceremonias tiene?
5. ¿Qué son Story Points? ¿Por qué se estima con Fibonacci?
6. ¿Qué son los Acceptance Criteria? Redacta los de "DELETE library".
7. ¿Qué es Definition of Done? Escribe una DoD de 5 puntos para un endpoint.
8. ¿Qué es un burndown y qué muestra?
9. ¿Cómo se relaciona Azure Boards con Azure Pipelines?
10. ¿Qué tipos de Boards existen (Basic/Agile/Scrum) y cuál usarías?

Módulo 12 — Buenas prácticas (autoevaluación)

1. Nombra los 5 principios SOLID y encuentra un ejemplo (o violación) de cada uno en el proyecto.
2. ¿Qué es un smell code? Lista 3 presentes en el proyecto (p. ej. `.Result` , `Update()` redundante).
3. ¿Cuándo un comentario es bueno y cuándo es ruido?
4. ¿Por qué no se debe usar `.Result` en una cadena async? ¿Qué solución aplica aquí?
5. ¿Qué es un DTO de salida y por qué exponer entidades es mala idea?
6. ¿Qué es la validación en el borde? ¿Qué atributos agregarías a `BookForm` ?
7. ¿Cuándo debe responder HTTP y cuándo lanzar excepción? Ejemplifica con 404 y BD caída.
8. ¿Qué es `ILogger<T>` ? ¿Qué niveles existen y cuándo usar cada uno?
9. ¿Por qué no se debe loguear una password? ¿Qué loguearías en un intento de login fallido?
10. ¿Qué es el patrón "find-then-404" y dónde está en el proyecto?
11. ¿Qué significa "nombres que expresan intención"? Da 3 nombres del proyecto que lo cumplan.
12. ¿Qué es DRY y por qué aplicarlo "con juicio"?

Módulo 13 — Seguridad (autoevaluación)

1. Enumera 4 riesgos del OWASP Top 10 presentes (o latentes) en este proyecto.
2. ¿Por qué EF Core/LINQ es seguro frente a SQL Injection? ¿Cuándo dejaría de serlo?
3. ¿Qué es un secreto? Nombra los secretos del proyecto y dónde deben vivir en dev/CI/prod.
4. ¿Qué riesgo tiene el `SecretKey` en `appsettings.json` ? ¿Qué hacer si ya se publicó?
5. ¿Por qué no confiar en el `role` que llega en el body del login?
6. ¿Qué es CORS? ¿Por qué la política "Frontend" restringe a `localhost:5173` ?
7. ¿Qué significa menor privilegio? Ejemplifica con el usuario de BD.
8. ¿Por qué `DeveloperExceptionPage` nunca debe activarse en producción?
9. ¿Qué es rate limiting y cómo protege `/login` ?
10. ¿Qué es hashing de passwords? ¿Por qué `admin` / `1234` es inaceptable en producción?
11. ¿Qué endurecimientos de JWT aplicarías (clave, algoritmo, expiración)?
12. ¿Qué es defensa en profundidad? Da 3 capas para esta API.

Módulo 14 — Despliegue (autoevaluación)

1. ¿Qué cambia entre Development y Production en este proyecto?
2. ¿Por qué la app "compila pero no conecta" en producción con la config actual?
3. ¿Qué es `dotnet publish` y qué produce? Diferencia framework-dependent vs self-contained.
4. ¿Qué es CI? Escribe el YAML mínimo de GitHub Actions para este repo.
5. ¿Qué es CD? Nombra los stages típicos.
6. ¿Por qué `db.Database.Migrate()` al arrancar es riesgoso en producción?
7. ¿Dónde deben vivir los secretos en CI (GitHub/Azure)?
8. ¿Qué es el 12-factor? Aplica 3 factores a este proyecto.
9. ¿Qué es un artefacto en un pipeline?
10. ¿Qué alternativa a GitHub Actions conoces (Azure Pipelines)? ¿En qué se parecen?
11. ¿Qué es una imagen Docker y qué base usarías para una app ASP.NET Core 8?
12. ¿Cómo probarías los tests en CI sin credenciales de Supabase?

Módulo 15 — Avanzado (autoevaluación)

1. ¿Qué es CQRS? ¿Cómo dividirías `BooksService.Get` y `BooksService.Add`?
 2. ¿Qué es Vertical Slice y qué archivos tendría la slice "AddBook"?
 3. ¿Qué hace MediatR? ¿Cómo refactorizarías `BooksController.Add`?
 4. ¿Qué es una caché in-memory y una distribuida? ¿Cuándo usarías Redis?
 5. ¿Qué es un contenedor? ¿Qué hace un Dockerfile multi-stage?
 6. ¿Qué es OpenTelemetry y cuáles son sus 3 señales?
 7. ¿Qué es arquitectura hexagonal? ¿Qué son puertos y adaptadores?
 8. ¿Cuándo vale la pena pasar de monolito a microservicios?
 9. ¿Qué es un API Gateway? ¿Para qué sirve un BFF?
 10. ¿Qué es el versionado de API? ¿Cómo versionarías este proyecto a v2?
 11. ¿Qué es Testcontainers y qué problema resuelve respecto a SQLite?
 12. ¿Qué es "monolito modular" y por qué es el punto medio recomendado?
-

PROYECTOS PRÁCTICOS POR MÓDULO

Proyecto acumulativo (recomendado): "Librería Plus". Es un sistema de gestión de librerías **que tú construyes desde cero** y que crece con cada módulo. Es distinto del HackerRank1 (aunque parecido): el objetivo es que lo hagas **sin mirar** el proyecto real. Al final tendrás dos proyectos comparables y la capacidad real de construir el segundo desde cero.

Reglas del proyecto acumulativo: - No copies código del HackerRank1. Si te atascas, lee el código real **después** de intentarlo 20 minutos. - Cada módulo añade una funcionalidad; los módulos siguientes usan lo anterior (espiral de reforzamiento). - Al final, ejecuta la evaluación final con **ambos** proyectos abiertos.

Módulo 1 — Proyecto: "Librería Plus" (semilla HTTP)

Construye el contrato en papel: la lista completa de endpoints de una "Librería Plus" (librerías y libros), con método, URL, body JSON y códigos de respuesta (éxito y error). Crea un archivo `CONTRATO.md`. Luego prueba el contrato contra el HackerRank1 real y marca las diferencias.

Módulo 2 — Proyecto: consola C# de dominio

`dotnet new console -n LibreriaPlus.Domain`. Crea clases `Libreria` y `Libro`, un `record LibreriaDto`, una interfaz `ICatalogoService` con métodos async, LINQ para filtrar por id, y nullable correcto. Consume el HackerRank1 real desde esta consola con `HttpClient` (GET/POST) — así practicas HTTP + C# juntos.

Módulo 3 — Proyecto: la API "Librería Plus" nace

`dotnet new webapi -n LibreriaPlus.Api`. Configura el modelo moderno (`WebApplicationBuilder`) y **luego** muévelo al modelo legacy (`Program` + `Startup`) para practicar ambos. Registra servicios con los 3 lifetimes, monta el pipeline completo (Routing, CORS, Auth, Authorization, Endpoints) y crea controllers `LibreriasController` y `LibrosController` con todos los verbos.

Módulo 4 — Proyecto: arquitectura de Librería Plus

Documenta en `ARQUITECTURA.md`: capas, flujo, decisión explícita sobre Repository (justifica si lo usas o no). Luego **refactoriza el HackerRank1** agregando un Repository `ILibreriaRepository` en una rama aparte y escribe la conclusión comparativa (este es el mejor ejercicio para entender los tradeoffs).

Módulo 5 — Proyecto: EF Core en Librería Plus

Agrega `Microsoft.EntityFrameworkCore` + `Npgsql.EntityFrameworkCore.PostgreSQL` (o SQLite local para no depender de la nube) a Librería Plus. Crea `LibreriaContext` con `DbSet<Libreria>` y `DbSet<Libro>`, relación 1:N con FK CASCADE, y 3 migraciones sucesivas (crear, agregar columna, quitar columna). Aplícalas y reviértelas.

Módulo 6 — Proyecto: Supabase + User Secrets en Librería Plus

Crea tu propio proyecto Supabase. Configura User Secrets (`dotnet user-secrets init/set`) y arranca Librería Plus contra tu BD. Escribe tu propio `REPORTE-USER-SECRETS.md` con las 5 piezas. Verifica con `git grep` que tu repo no tiene la password.

Módulo 7 — Proyecto: JWT en Librería Plus

Agrega `Microsoft.AspNetCore.Authentication.JwtBearer`. Crea `JwtSettings`, un `TokenGenerator` (HS256), un endpoint `POST /login` (credenciales en config o BD, no hardcode), protege un endpoint con `[Authorize]` y otro con rol. Decodifica tu token en `jwt.io` y verifica `iss`, `aud`, `exp`.

Módulo 8 — Proyecto: Swagger en Librería Plus

Agrega Swashbuckle con título/descripción propios, prueba todo desde la UI con el botón Authorize, descarga el `swagger.json` y validalo con editor.swagger.io. Añade `[ProducesResponseType]` a un endpoint y observa el cambio en el JSON.

Módulo 9 — Proyecto: tests de integración para Librería Plus

Agrega un proyecto xUnit + `Microsoft.AspNetCore.Mvc.Testing` + SQLite in-memory. Escribe 6 tests: POST libro 201/404, GET libros 200/404, DELETE librería 204/404, y uno que verifique el CASCADE. Reutiliza el patrón `RemoveAll` + `AddSingleton(context)`. Mide cobertura.

Módulo 10 — Proyecto: git para Librería Plus

Crea el repo de Librería Plus con historia limpia: Conventional Commits, `.gitignore` (bin/obj/vs), rama `feature/auth` para el Módulo 7, PR con squash merge, y verificación final `git grep "Password="` vacío.

Módulo 11 — Proyecto: backlog de Librería Plus en Azure DevOps

Crea una organización de Azure DevOps y modela el backlog completo de Librería Plus: 1 Epic, 3 Features, 6 PBIs con Acceptance Criteria y 15 Tasks estimadas en Story Points. Planifica un sprint y escribe la DoD.

Módulo 12 — Proyecto: auditoría de calidad

Audita Librería Plus con una checklist SOLID + Clean Code: elimina smells (nombres, métodos largos, `.Result`), agrega DTOs de salida, validación `[Required]/[MaxLength]`, `ILogger<T>` en servicios y un `ExceptionMiddleware` que devuelva JSON en prod y detalle en dev. Refactoriza con tests en verde.

Módulo 13 — Proyecto: endurecimiento de seguridad

En Librería Plus: hashea passwords (BCrypt), mueve el `SecretKey` a User Secrets/env, agrega rate limiting en `/login`, logs de auditoría, validación estricta, política CORS restrictiva, y una migración que marque columnas correctamente. Documenta en `SEGURIDAD.md` qué OWASP Top 10 mitigaste.

Módulo 14 — Proyecto: CI/CD de Librería Plus

Crea `.github/workflows/ci.yml` (build + test en `ubuntu-latest`), un job de publish de artefacto, y luego despliega en una plataforma gratuita (Render/Railway/Azure) usando variables de entorno para la connection string y el `SecretKey`. Documenta la promoción Dev→Prod.

Módulo 15 — Proyecto: evolución de Librería Plus

En una rama `feature/avanzado`: refactoriza un controller a MediatR, agrega `IMemoryCache` con invalidación, crea un `Dockerfile` multi-stage y `docker-compose.yml` (api + postgres), y añade OpenTelemetry con export a Jaeger. Termina escribiendo `EVOLUCION.md` con una propuesta de microservicios (o su rechazo argumentado).

EVALUACIÓN FINAL — Examen técnico universitario

Duración: 4 horas (recomendado en un solo bloque). **Ponderación:** Total 100 puntos. **Reglas:** sin internet durante las partes I–III; se permite el proyecto HackerRank1 abierto SOLO en la parte III (práctica) si se indica. Sin consultas externas en I, II, IV y V.

Parte I — Preguntas teóricas (30 puntos · ~45 min)

Responde con 2-4 líneas cada una.

1. (2 p) ¿Qué diferencia hay entre `401 Unauthorized` y `404 Not Found` en el contexto de `GET /api/libraries/{id}/books`?
2. (2 p) ¿Por qué HTTP se considera stateless y qué solución usa este proyecto para la identidad?
3. (2 p) ¿Qué es la ejecución diferida de `IQueryable`? Da un ejemplo del proyecto donde se compone la consulta antes de ejecutarla.
4. (2 p) ¿Qué diferencia hay entre `Migrate()` y `EnsureCreated()`? ¿Cuándo se usa cada uno aquí y por qué?
5. (2 p) Explica Transient, Scoped y Singleton. ¿Qué lifetime tienen `LibrariesService`, `AuthenticationService` y `JwtSettings` y por qué?
6. (2 p) ¿Qué es el Change Tracker? ¿Qué `EntityState` adopta una entidad al hacer `AddAsync`, `Remove` o modificar una trackeada?
7. (2 p) ¿Qué garantiza la firma de un JWT (HS256) y qué NO garantiza?
8. (2 p) ¿Qué es un DTO? ¿Por qué `BookForm` existe y por qué exponer `Book` directamente es discutible?
9. (2 p) ¿Qué hace el guard `if (services.Any(d => d.ServiceType == typeof(JwtSettings))) return;` en `ConfigureServices`?
10. (2 p) ¿Qué es CORS? ¿Qué permite la política "Frontend" y cuál es el riesgo de `AllowAnyOrigin`?
11. (2 p) ¿Por qué EF Core/LINQ evita la SQL Injection? ¿Qué caso concreto de este proyecto lo demuestra?
12. (2 p) ¿Qué es un middleware? Nombra los 6 middleware del pipeline del proyecto en orden.
13. (2 p) ¿Qué diferencia hay entre `AddDbContext` y `AddDbContextPool`?
14. (2 p) ¿Qué es un ensamblado? ¿Qué relación tiene con un proyecto y una solución?
15. (2 p) ¿Por qué se dice que el `DbContext` es a la vez Unit of Work y Repository?

Parte II — Preguntas de análisis (25 puntos · ~50 min)

Responde con párrafos argumentados (5-8 líneas).

1. (6 p) **Arquitectura.** El proyecto NO usa Repository Pattern. Explica con detalle: (a) qué hace hoy `LibrariesService` con el `DbContext`, (b) por qué añadir un `IRepository<T>` sería redundante aquí, (c) en qué escenario lo añadirías, y (d) qué tradeoff tiene la decisión tomada.
2. (6 p) **Configuración y entornos.** `appsettings.json` contiene `Password=[SUPABASE-PASSWORD]` (placeholder) y la password real vive en User Secrets. Explica: (a) la cadena completa de las 5 piezas, (b) la precedencia de proveedores de configuración, (c) por qué esto **falla en producción**, y (d) la solución correcta para prod.
3. (6 p) **Pruebas.** Los tests sustituyen el `LibraryContext` por SQLite in-memory con `RemoveAll` + `AddSingleton` y usan `EnsureCreated`. Analiza: (a) por qué este enfoque prueba "la app real", (b) por qué se prefiere a mockear los services, (c) qué limitaciones tiene frente a PostgreSQL real, y (d) qué aportaría Testcontainers.
4. (7 p) **Contrato REST.** Dado el test `TestDeleteLibrary`, explica la secuencia completa de estados HTTP y por qué el contrato es: DELETE librería existente → `204`; GET books de librería borrada → `404`; DELETE repetido → `404`. Relaciona con el CASCADE de la FK y con la idempotencia de DELETE.

Parte III — Ejercicios prácticos (30 puntos · ~90 min)

Implementa en el proyecto HackerRank1 (en una rama nueva) o en Librería Plus.

1. (8 p) **Endpoint nuevo.** Agrega `GET /api/libraries/{libraryId}/books/{bookId}` que devuelva `200` con el libro si existe, `404` si el libro o la librería no existen. Implementa service + controller. Escribe un test de integración.
2. (7 p) **Validación.** Agrega `[Required]` y `[MaxLength(100)]` a `BookForm.Name` y `[MaxLength(100)]` a `Category`. Verifica con un test que `POST` con `name` vacío devuelva `400`.

3. (8 p) **Migración.** Agrega la columna `PublishedYear` (nullable) a `Book`, crea la migración, revisala en el archivo, aplícala (o pruébala con `--script`) y luego crea una segunda migración que agregue un índice. Explica qué contiene el snapshot.
4. (7 p) **Refactor a MediatR (opcional alternativo: middleware de logging).** *Opción A:* refactoriza `BooksController.Add` a un `AddBookCommand` + handler con MediatR. *Opción B:* crea un middleware que loguee método, ruta, status code y duración de cada request con `ILogger`. Elige una y justifica.

Parte IV — Preguntas tipo entrevista (10 puntos · ~30 min)

Responde como si fueras una entrevista técnica.

1. (2 p) "Explicame qué pasa cuando haces `POST /api/libraries/1/books` con `{"name":"X"}` — desde la red hasta la BD y de vuelta."
2. (2 p) "¿Por qué tu API es `async/await` en toda la cadena? ¿Qué pasa si bloqueas con `.Result`?"
3. (2 p) "¿Cómo sabes que un secreto no está en el repo? ¿Qué harías si descubres que se filtró uno?"
4. (2 p) "¿Cuándo usarías PostgreSQL y por qué EF Core con Npgsql en vez de SQL a mano?"
5. (2 p) "¿Qué harías si tus tests pasan en local pero fallan en CI?"

Parte V — Escenarios reales (5 puntos · ~15 min)

Analiza y decide (2-3 líneas de justificación cada uno).

1. (1 p) Alguien publica el repo en GitHub y un scanner detecta el `SecretKey` del JWT en `appsettings.json`. ¿Qué haces de inmediato y cómo evitas que vuelva a ocurrir?
2. (1 p) En producción, dos instancias de la app arrancan a la vez y ambas ejecutan `db.Database.Migrate()`. ¿Qué puede pasar y cómo lo previenes?
3. (1 p) Un cliente dice que `GET /api/libraries/5/books` devuelve `401` solo en algunos casos. ¿Qué revisas primero (orden de diagnóstico)?
4. (1 p) Los tests pasan con SQLite pero fallan en Supabase al guardar un libro sin `category`. ¿Por qué? (columna NOT NULL).
5. (1 p) El backend se migra de EF Core 6.0 a 8.0 y los tests dejan de compilar con `MissingMethodException`. ¿Cuál es la causa más probable y la solución?

Rúbrica de corrección

Parte	Puntos	Criterio de dominio
I · Teoría	30	≥24 → dominado; 18-23 → repasar módulos señalados; <18 → repetir
II · Análisis	25	≥19 → dominado; 13-18 → repasar
III · Práctica	30	código compila + tests verdes + decisión justificada
IV · Entrevista	10	≥8 → listo para entrevista junior
V · Escenarios	5	≥4 → criterio de producción
Total	100	≥70 → listo para diseñar un proyecto similar de forma autónoma

Interpretación del resultado: - **90-100:** nivel avanzado — puedes plantear mejoras arquitectónicas reales. - **70-89:** nivel esperado al terminar la guía — capaz de construir un proyecto similar desde cero. - **50-69:** revisa los módulos donde fallaste (usa la rúbrica de autoevaluación para localizarlos) y repite la evaluación en 2 semanas. - **<50:** vuelve a estudiar la ruta desde el módulo 5 (EF Core) y los módulos de fundamentos antes de reintentar.

Cierre

Si llegaste hasta aquí y completaste el checklist + la evaluación final con ≥ 70 puntos, estás en condiciones de:

1. Explicar **cada línea** de `Program.cs`, `Startup.cs`, los controllers, los services, el `LibraryContext`, la migración y los tests.
2. Justificar cada decisión de diseño del proyecto (capas, sin Repository, SQLite en tests, User Secrets, JWT HS256).
3. **Diseñar e implementar desde cero** una API Web .NET 8 con PostgreSQL, autenticación JWT, documentación OpenAPI y tests de integración — sin ayuda.

La diferencia entre "leer la guía" y "dominar el proyecto" es la que hay entre mirar y **hacer**. Los ejercicios, el proyecto acumulativo y la evaluación final son el verdadero examen: el código que escribas tú, no el que ya estaba escrito.

Guía generada a partir del estado real del repositorio `Paradigma-lab1` (commit `199fc31`). Última actualización: 2026-08-04.